

目 录

第一部分 基础知识及考点.....3

第一章 微型计算机基础知识.....3

第二章 8086/8088 硬件特性.....10

第三章 8086 指令系统.....26

第四章 汇编语言程序设计.....57

第五章 存储器.....82

第六章 输入输出与中断技术.....94

第七章 可编程接口芯片.....112

第八章 数/模转换及模/数转换技术.....141

第二部分 模拟题.....150

微机原理模拟试题 1.....150

模拟试题 1 参考答案.....157

微机原理模拟试题二.....159

模拟试题 2 答案部分.....166

第一部分 基础知识及考点

第一章 微型计算机基础知识

一、本章考点

本章考察小题

【例】

1. 微型计算机包括几部分?

2. 完成下列数制的转换

(1) $10101101\text{B} = (\quad) \text{D} = (\quad) \text{H}$

(2) $0.11\text{B} = (\quad) \text{D}$

(3) $211.25 = (\quad) \text{B} = (\quad) \text{H}$

(4) $10111.0101\text{B} = (\quad) \text{H} = (\quad) \text{BCD}$

3. 已知 $X = +1011010\text{B}$, $Y = -0011011\text{B}$, 设机器数为 8 位, 分别写出 X, Y 的原码, 反码和补码.

4. 已知 $X = 51$, $Y = -86$, 用补码完成下列运算, 并判断是否产生溢出 (设字长为 8 位).

$X + Y$

$X - Y$

$-X + Y$

$-X - Y$

【答案】

1. 由微处理器, 存储器, 输入输出设备, 总线四部分构成

2. ① 173 AD; ② 0.75; ③ 11010011.01 D3.4; ④ 17.5 100011.0101

3. X: 原码: 0101 1010; 反码: 0101 1010; 补码: 0101 1010

Y: 原码: 1001 1011; 反码: 1110 0100; 补码: 1110 0101

4. X: 原码: 0011 0011; 补码: 0011 0011

Y: 原码: 1101 0110 补码: 1010 1010

只有在同符号数相加或者异符号数相减的情况下, 才有可能产生溢出

① $(X + Y)$ 补 = 1101 1110, 真值为: $-128 + 64 + 16 + 8 + 4 + 2 = -34$

② $-Y = 0101 0110$, $(X - Y)$ 补 = 1000 0101, $X - Y > 0$, 首位应为 0, 所以溢出

③ $(-X)$ 补 = 1100 1101, $(-X + Y)$ 补 = 1 0111 0111, 溢出

④ $(-X - Y)$ 补 = 1100 1101 + 0101 0110, 真值为 34

二、基础知识

1. 微型计算机系统简介

- (1) 微型计算机系统基于微处理器,由微处理器,存储器,输入输出设备,总线四部分构成.
- (2) 微处理器功能:
 - ① 在微处理器与存储器之间,微处理器和输入输出设备之间传送数据;
 - ② 简单的算术和逻辑运算;
 - ③ 通过简单的判定控制程序的流向.
- (3) 存储器:
 - ① 存储器是微型计算机的存储和记忆部件,用以存放程序指令和数据.微型计算机中常用的存储器包括动态随机存取存储器(DRAM),静态随机存取存储器(SRAM),高速缓冲存储器(Cache),只读存储器(ROM),闪存.
 - ② 存储器由一系列的存储单元构成,每个存储单元可以存放 8 位的二进制数.指令和数据都是以二进制数的形式存储在存储单元中.微处理器为每个存储单元分配一个存储地址,并通过地址总线把存储器地址传送给存储器,以此选择存储单元.微处理器通过数据总线从存储器中读取指令和数据,并将指令执行结果存放在存储器中.
 - ③ 微型计算机的存储器也常称为内存储器,主存储器或内存,主存.
 - ④ 存储器中存储单元的数量称为存储容量.微型计算机中存储容量是由微处理器决定的.
- (4) 输入输出系统:
 - ① 微型计算机的输入输出系统包括输入输出接口和输入输出设备.
 - ② 由于 I/O 设备的工作速度,信号类型,信号格式等与主机不同,I/O 设备一般不能直接和主机进行连接,通常需要在主机与 I/O 设备之间设置 I/O 接口.I/O 接口提供信号格式转换,时序匹配,数据缓冲等功能,作为主机与外设之间通信的桥梁.
 - ③ 微型计算机的输入输出设备常见的有显示器,键盘,鼠标,磁盘驱动器,光盘驱动器,打印机,绘图仪,扫描仪和各种串行通信设备.
- (5) 总线
 - ① 微型计算机系统是总线结构的计算机系统.总线是微型计算机系统中互连各部件的一组公用信号线,负责在各部件之间传送地址信息,数据信息和控制信息.
 - ② 微处理器通过总线与存储器,I/O 设备连接成一个整体.
 - ③ 在 CPU,存储器,I/O 接口之间传送信息的总线称为系统总线.

- ④ 系统总线分类:根据传送信息的类别,系统总线又分为地址总线,数据总线和控制总线三种类型.

1° 地址总线

地址总线由微处理器产生,用来向存储器或输入输出接口传送地址信息,是单向总线.

2° 数据总线

数据总线用来传送数据,是双向总线.通过数据总线,微处理器既可以从内存或输入设备输入数据,又可以将数据传送至内存或输出设备.

3° 控制总线

控制总线传送控制信号,时序信号和状态信号.其中有些是微处理器发出的控制存储器或 I/O 操作的信号,有些是存储器或 I/O 接口向微处理器发出的状态信息.所以,控制总线的每一根信号都是单向的,但从整体来看,控制总线是双向的.

2. 计算机中的数制

(1) 数制

① 进位计数制

二进制,十进制,十六进制,都是进位计数制,进位计数制使用位置表示法,都有数码,位权和基数 3 个要素.

1° 十进制

数码为:0,1,2,3,4,5,6,7,8,9

基数为:10

表示形式:D

位权: $10^3, 10^2, 10^1, 10^0, 10^{-1}$

例: $678.34_D = 6 \times 10^2 + 7 \times 10^1 + 8 \times 10^0 + 3 \times 10^{-1} + 4 \times 10^{-2}$ (权表达式)

2° 二进制

数码为:0,1

基数为:2

位权: 2^i

表示形式:B

3° 十六进制

数码为:0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F

基数为:16

位权: 16^i

表示形式:H

4° 基数为 R 的 R 进制:

R 个基本符号:0,1,2,...,n

位权: R^i

任意一个 R 进制数 N 可以表示为:

$$N = a_{n-1} \times R^{n-1} + a_{n-2} \times R^{n-2} + \cdots + a_1 \times R^1 + a_0 \times R^0 + a_{-1} \times R^{-1} + \cdots +$$

$$a_{-m} \times R^{-m} = \sum_{i=-m}^{n-1} a_i \times R^i.$$

其中: a_i 是数码,R 是基数, R^i 是位权

(2) 数制转换

① 非十进制转换为十进制

按权表达式展开,再按十进制运算规则求和,即可得到对应的十进制数.

② 十进制数到非十进制数的转换

1° 转换为二进制,对整数:除 2 取余;对小数:乘 2 取整.

2° 转换为十六进制,对整数:除 16 取余;对小数:乘 16 取整.

3° 二进制数与十六进制数之间的转换

用 4 位二进制数表示 1 位十六进制数

$$\begin{array}{ccc} (0101 & 1010 & . 1000)_2 \\ \downarrow & \downarrow & \downarrow \\ (5 & A & 8)_{16} \end{array}$$

4° 二进制与八进制间的转换

用 3 位二进制数表示 1 位八进制数

3. 二进制整数算数运算

(1) 运算规则

① 加法运算

$$0+0=0 \quad 0+1=1$$

$$1+0=1 \quad 1+1=0 \text{ (有进位)}$$

② 减法运算

$$0-0=0 \quad 1-0=1$$

$$1-1=0 \quad 0-1=1 \text{ (有借位)}$$

③ 乘法运算

$0 \times 0 = 0$

$1 \times 0 = 0$

$0 \times 1 = 0$

$1 \times 1 = 1$

④ 除法运算

$0 \div 1 = 0$

$1 \div 1 = 1$

⑤ 移位运算

无符号数的乘法运算可以转换为加法和左移位的运算.一个二进制数,每左移一位,相当于乘以2,左移 n 位就相当于乘以 2^n .每右移一位,相当于除以2,左移 n 位就相当于除以 2^n .

(2) 无符号整数

一个 n 位无符号二进制数 X ,表示数的范围为: $0 \leq X \leq 2^n - 1$

在无符号数的加减运算中,如果最高位向前有进位(加法)或借位(减法),则运算结果产生溢出.

(3) 带符号整数

① 表示方法:原码,反码,补码

1° 原码

最高位为符号位,数值部分为真值的绝对值. X 的原码记为 $[X]_{\text{原}}$.

一个字节的原码,表示的数据范围为:

11111111~01111111,即 $-127 \sim +127$.

$[+0]_{\text{原}} = 0000\ 0000$

$[-0]_{\text{原}} = 1000\ 0000$

2° 反码

对于正数,其反码形式与原码相同,最高位为符号位,其余位为数值位.

对于负数,将其原码除符号位之外的其余各位按位取反

反码中0有 $+0$ 与 -0 之分

一个字节的带符号二进制数的反码所能表示的数值范围为 $-127 \sim +127$

3° 补码

正数的补码与其原码相同,最高位为符号位,其他各位为数值位.负数的补码为它的反码加1.

补码的几个特点:

0的补码表示是唯一的.没有 $+0$ 和 -0 之分,

一个字节的有符号二进制数的补码所能表示的范围为:

$-128 \sim +127$

补码的位权

无符号数:

128	64	32	16	8	4	2	1
-----	----	----	----	---	---	---	---

有符号数:

-128	64	32	16	8	4	2	1
------	----	----	----	---	---	---	---

1000 0001 为-127 的补码

1000 0000 为-128 的补码

补码转换为十进制数:

位权相加

$(11101010)_2 = (-128 + 64 + 32 + 8 + 2)_{10} = (-22)_{10}$

或者,补码再次取补得到它的原码,再转换成十进制数

运算规则

补码的加法规则:: $[X+Y]_{\text{补}} = [X]_{\text{补}} + [Y]_{\text{补}}$

补码的减法规则:: $[X-Y]_{\text{补}} = [X]_{\text{补}} - [Y]_{\text{补}} = [X]_{\text{补}} + [-Y]_{\text{补}}$

通过引进补码,可将减法运算转换为加法运算.

补码运算的结果仍然是补码,需要将其转换为十进制数.

② 带符号数的溢出

有符号数的表示范围:

字节类型 (8 位) 的有符号数:

原码:1111 1111~0111 1111(-127~127)

反码:1000 0000~0111 1111(-127~127)

补码:1000 0000~0111 1111(-128~127)

在运算中判断带符号数的溢出:

8 位二进制补码所能表示的数值范围为-128~+127.当补码运算结果超出这个表示范围时,便产生溢出.

4. 浮点数

表示方法:浮点数采用科学计数法,任何二进制数 N 都可表示为:

$$N = \pm S \times 2^{\pm j}$$

例:实数 14.375 转换为二进制数 1110.011:

$$1110.011 = 1.110011 \times 2^3$$

5. 字符型数据

(1) ASCII 码

ASCII 码表中:

0~20H (32)是控制字符或通讯专用字符;

21H~7EH (33~126)为可打印 ASCII 码;

数字 0~9:30H~39H,对应的十进制数为 48~57.

26 个大写字母 A~Z:41H~5AH,对应的十进制数为 65~90.

26 个小写字母 a~z:61H~7AH,对应的十进制数为 97~122.

(2) BCD 码

十进制数	0	1	2	3	4	5	6	7	8	9
BCD 码	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001

BCD 码与二进制数之间的转换:

BCD 码转换为十进制数,再与二进制数之间进行转换.

6. 基本逻辑运算

(1) “与”运算 ($Y=A \wedge B$)

(2) “或”运算 ($Y=A \vee B$)

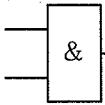
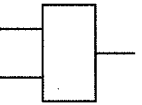
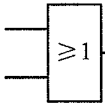
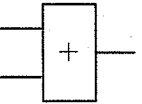
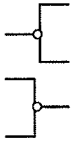
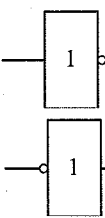
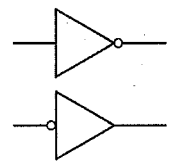
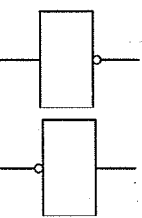
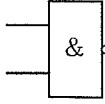
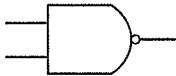
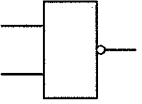
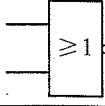
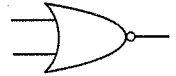
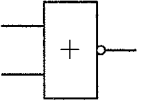
(3) “非”运算 $Y=A$

(4) “异或”运算 $Y=\overline{A}B + A\overline{B} = A \oplus B$

(5) “同或”运算 $Y=AB + \overline{A}\overline{B} = A \odot B$

逻辑门符号表示如下图:

表 C1 基本逻辑门电路图形符号

序号	名称	GB/T 4728.12-1996		国外流行图形符号	曾用图形符号
		限定符号	国标图形符号		
1	与门	&			
2	或门	≥ 1			
3	非门	 逻辑非入与出			
4	与非门				
5	或非门				

第二章 8086/8088 硬件特性

一、本章考点

本章基本都在小题部分出题,掌握 8086/8088 引脚图的部分与总线时序部分;各个寄存器相关知识,重点掌握 FLAG 寄存器标志位.

【例】

1. 列出 8086/8088 微处理器的区别.
2. 当 ALE 有效时,8088 的地址/数据总线上将出现什么信息?
3. 8086/8088 的/RD 引脚上是逻辑 0 意味着什么?
4. 当 MN/(MX)接地时选择的是哪种工作模式?
5. 8086/8088 的/WR 选通信号指示了有关 8086/8088 的什么操作?
6. 当 DT/(R)为逻辑 1 时,它指示了有关 8086/8088 的什么操作?
7. 解释/LOCK 引脚的操作.
8. 当 8086/8088 的 HOLD 输入被置为逻辑 1 电平时,将发生什么情况?
9. 8086 微处理器的哪些总线是典型的用于多路分离的?
10. 8086/8088CPU 为什么对存储器采用分段管理? 分段管理的方法是什么?
11. 8088 微处理器上经过多路分离的/BHE 信号有什么用途?
12. 一个总线周期等于几个时钟周期?
13. 如果 8086/8088 的 CLK 输入是 4MHz,那么一个总线周期是多少?
14. 若 READY 引脚接地,则它在 8086/8088 的总线周期上引入什么状态?
15. 当 8086/8088 以最大模式工作时,8288 总线控制器主要提供什么功能?
16. 8086 微处理器可以寻址多少字节存储器?

【答案】

1. 二者的主要区别:
 - (1) 8086 的数据总线宽度为 16 位,而 8088 的数据总线宽度为 8 位.
 - (2) 两款微处理器内部都有指令队列,8086 的指令队列长度为 6 字节,而 8088 的指令队列长度为 4 字节.
 - (3) 8088 的/SS0 引脚取代了 8086 的/BHE/S7 引脚,IO/ (M) 引脚取代了 8086 (IO) /M 引脚.

如果抗扰能力从 400mV 下降到 350mV,则 8086/8088TTL 兼容.

2. 地址信息

3. /RD 为读信号,当它为逻辑 0 时有效,CPU 通过数据总线接收来自存储器或

I/O 设备的数据.

4. 最大模式

5. /WR: 写选通信号,输出,三态.指示 8086/8088 正在输出数据给存储器或 I/O 设备.在/WR 为低电平有效,数据总线包含给存储器或 I/O 设备的有效数据.

6. DT/(/R)=1 表明 CPU 数据总线正在传送数据

7. /LOCK:

锁定输出信号,用来锁定外围设备对系统总线的控制权

该引脚通过在指令前加前缀 LOCK 激活

8. 微处理器停止执行软件,并将其地址,数据,控制总线置成高阻抗状态

9. 多路分离 8088

两片 74LS373 透明锁存器来分离地址/数据总线 AD7—AD0 及地址/状态线 A19/S6—A16/S3

10. 8086/8088CPU 内部寄存器的宽度为 16 位,单个寄存器不能寻址 1M 的地址空间,所以它采用了分段寻址的管理方法,使用两个寄存器表示物理地址.

8086/8088CPU 将 1M 的内存空间划分成一些连续的区域,称为段.每段的,最大长度为 64K,并要求段的起始地址必须能被 16 整除.段的起始地址的高 16 位称为段基址,存于段寄存器中.段内各存储单元地址相对于段起始地址的位移量,称为段内偏移量.

物理地址=段基址 X10H+段内偏移量.

11. 在 8086 中,在读操作或写操作期间允许高 8 位数据总线 D15—D8 有效

12. 4 个

13. 250 ns

14. READY: 就绪输入信号,用于在微处理器时序中插入等待状态.若该引脚被置为逻辑 0,则微处理器进入等待状态并保持空闲.

15. 状态位 S2, S1 和 S0 通常由 8288 总线控制器译码指示当前总线周期的功能.

16. 1MB

二、基础知识点

1. 8086/8088 基本介绍

8088 与 8086 同属于第三代 16 位微处理器,都有最大和最小两种工作模式,

支持完全相同的指令系统。

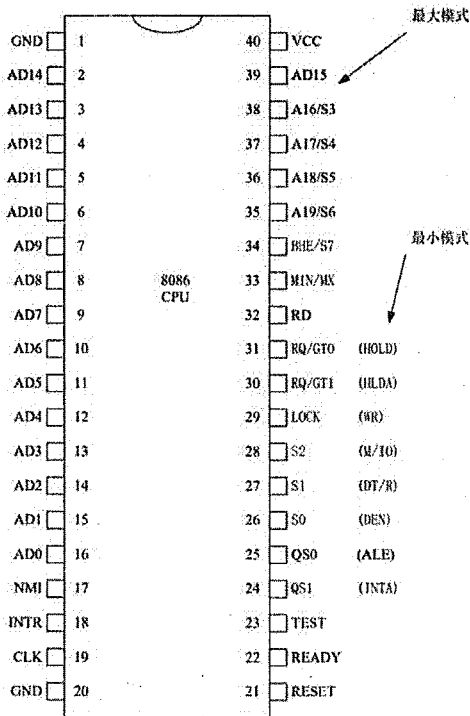
二者的主要区别:

- (1) 8086 的数据总线宽度为 16 位,而 8088 的数据总线宽度为 8 位。
- (2) 两款微处理器内部都有指令队列,8086 的指令队列长度为 6 字节,而 8088 的指令队列长度为 4 字节。③8088 的/SS0 引脚取代了 8086 的/BHE/S7 引脚,/IO/ (M) 引脚取代了 8086 (IO) /M 引脚。

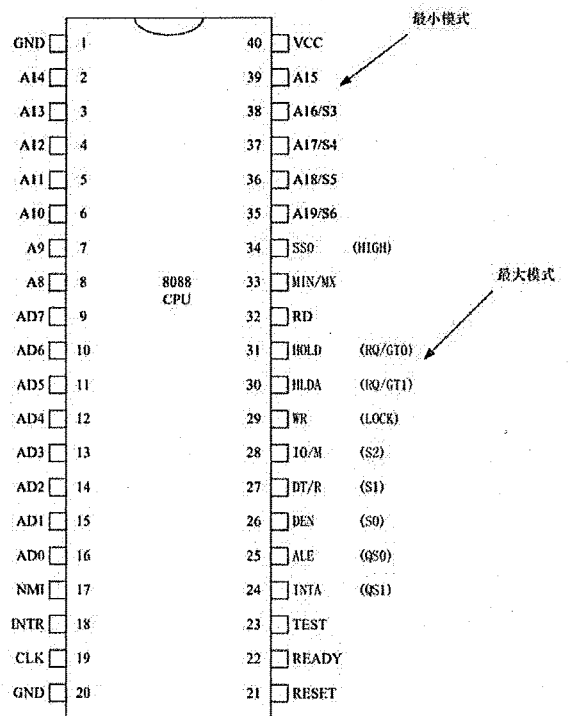
如果抗扰能力从 400mV 下降到 350mV,则 8086/8088TTL 兼容。

2. 8086/8088 引脚图及功能

(1) 8086/8088 引脚图



8086 处理器的引脚图



8088 处理器的引脚图

(2) 基本介绍:

二者均封装在 40 引脚的双列直插封装(DIP)中

8086 是具有 16 位数据总线的 16 位微处理器(AD0—AD15),而 8088 是具有 8 位数据总线的 16 位微处理器(AD0—AD7)

(3) 工作模式:

最小模式:系统控制总线的信号由芯片本身的引脚直接引出

最大模式:系统控制总线由 8088/8086 和总线控制器 8288 共同构成

3. 8088 引脚及其功能

MN//MX:最小/最大模式引脚,为微处理器选择最小模式或最大模式工作方式,高电平最小,低电平(接地)最大

CLK: 时钟引脚,为微处理器提供基本的定时信号时钟信号占空比必须为33%(即时钟周期的 1/3 为高电平,而 2/3 为低电平)

Vcc:电源输入提供+5.0V, $\pm 10\%$

GND:接地引脚接地

(1) 最小模式

① 地址数据分时复用控制信号

1° AD7~AD0:8088 地址/数据分时复用总线(address/data bus),ALE 有效输出地址的低 8 位,传送数据信号时双向,三态.

2° A15~A8:8 位地址信号,输出,三态.在整个总线周期内提供存储器高 8 位地址.

3° A19/S6~A16/S3:分时复用地 址/状态总线(address/status bus),输出高四位地址信号,三态.提供地址信号 A19~A16 及状态位 S6~S3.
S6=0,S5=IF,

状态位S3和S4的功能

S4	S3	功能
0	0	附加段
0	1	堆栈段
1	0	代码段或不用
1	1	数据段

4° ALE:地址锁存允许,三态,输出.高电平表明 8088 的地址/数据总线包含地址信息

5° IO//M:输出,三态.该引脚选择存储器或 I/O 端口,即微处理器地址总线是存储器地址还 I/O 端口地址.高:I/O 接口;低:内存

② 数据传送控制信号

1° /RD:读信号,输出,三态.当它为低电平时有效,CPU 通过数据总线接收来自存储器或 I/O 设备的数据.

2° /WR:写选通信号,输出,三态.指示 8086/8088 正在输出数据给存储器或 I/O 设备.在/WR 为低电平有效,数据总线包含给存储器或 I/O 设备的有效数据.

3° DT/ (R):数据传送/接受信号,表明 CPU 数据总线正在传送(DT/R=1)或接受(DT/R=0)数据

4° /DEN:数据使能信号,三态,输出.低电平有效,表明数据总线上有有效数据

③ 中断相关控制信号

1° INTR:可屏蔽中断请求信号,输入,用来申请一个硬件中断.当 IF=1 时,若 INTR 保持高电平,则 8088 在当前指令执行完毕后就进入中断响应周期 (/INTA 变有效) .

2° /INTA:可屏蔽中断响应信号,输出.响应 INTR 输入.该引脚常用来选通中断向量码以响应中断请求.

3° NMI:非屏蔽中断输入信号.与 INTR 信号类似,但 NM 中断不必检查 IF 标志位是否为 1.

④ DMA 操作相关信号

1° HOLD:保持输入信号,用来请求直接存储器存取(DMA)若 HOLD 信号为逻辑 1,微处理器停止执行软件,并将其地址,数据,控制总线置成高阻抗状态

2° HLDA:保持响应信号,指示 8088 已进入保持状态

⑤ 其他信号

1° SS0:8088 微处理器状态线,该信号 IO/M 及 DT/R 组合在一起,译码当前总线周期的不同功能

(8088)使用 SS0 的总线周期状态

IO/M	DT/R	SS0	功能
0	0	0	中断响应
0	0	1	读存储器
0	1	0	写存储器
0	1	1	暂停
1	0	0	取操作码
1	0	1	读 I/O
1	1	0	写 I/O
1	1	1	无效状态

2° RESET:复位信号,是输入信号.若该引脚保持 4 个时钟周期以上的高电平,则导致微处理器复位;一旦 8086 或 8088 复位,则它从存储单元 FFFF0H 开始执行指令,并使 IF 标志位清零,禁止中断.

3° READY:就绪输入信号,用于在微处理器时序中插入等待状态.若该引脚被置为逻辑 0,则微处理器进入等待状态并保持空闲.

4° /TEST:这是一个测试输入信号,用 WAIT 指令来测试,若/TEST 为逻辑 0,则 WAIT 指令的功能相当于 NOP 空操作指令若/TEST 为逻辑 1,则 WAIT 指令重复测试 TEST 引脚,该引脚大多与 8087 算术协处理器相连.

(2) 最大模式

与最小模式相同引脚功能基本相同.

① S2, S1 和 S0:

这些状态位指示当前总线周期的功能,它们通常由 8288 总线控制器译码

总线控制器(8288)使用 S2,S1,S0 产生的总线控制功能

S2	S1	S0	功能
0	0	0	中断响应
0	0	1	读存储器
0	1	0	写存储器
0	1	1	暂停
1	0	0	取操作码
1	0	1	读 I/O
1	1	0	写 I/O
1	1	1	无效状态

② QS1 和 QS0:

队列状态位,表明内部指令队列的状态

这些引脚被算术协处理器(8087)访问

队列状态位

QS1	QS0	功能
0	0	队列空闲
0	1	操作码的第一个字节
1	0	队列空
1	1	操作码的后续字节

③ /LOCK:

锁定输出信号,用来锁定外围设备对系统总线的控制权

该引脚通过在指令前加前缀 LOCK 激活

④ RQ/GT1 和 RQ/GT0:

请求/同意 在最大模式下请求直接存储器存取(DMA)

双向的,既可以用于请求 DMA 操作,又可用于同意 DMA 操作

⑤ HIGH 始终为高电平

4. 8086 的引脚及其功能

基本与 8088 相同,针对不同的引脚加以说明

- (1) $\overline{M}/IO$: 选择存储器或 I/O 端口,该引脚指示,微处理器地址总线是存储器地址还是 I/O 端口地址
- (2) AD7—AD0:8086/8088 地址/数据总线,构成地址/数据多路复用总线
- (3) AD15—AD8:8086 地址/数据总线,构成了 8086 的高 8 位地址/数据多路复用总线
- (4) $(\overline{BHE})/S7$:高 8 位总线允许引脚,用在 8086 中,在读操作或写操作期间允许高 8 位数据总线 D15—D8 有效,状态位 S7 始终为逻辑 1.

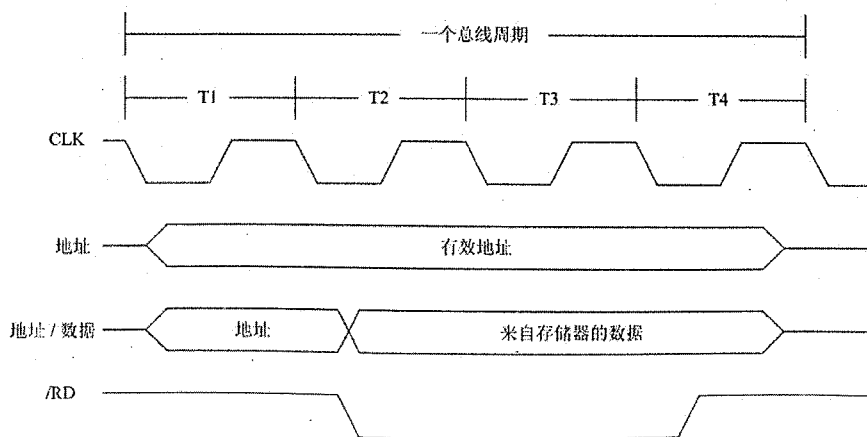
5. 8086/8088 的总线时序

时序:微处理器各引脚在时间上的先后关系称为时序.时序有两种不同的度量方法:时钟周期与总线周期.

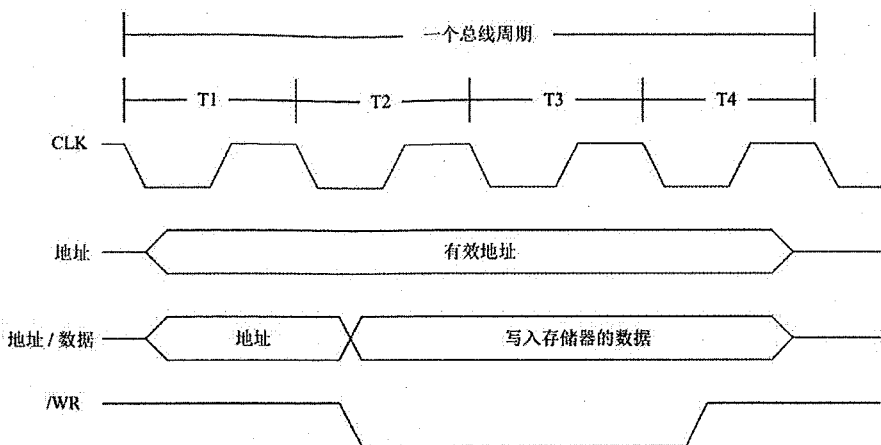
(1) 度量方法

- ① 时钟周期:每个时钟脉冲的持续时间,是计算机内部最小的计量单位,用 T_i 表示.时钟脉冲由时钟发生器发生.808 标准脉冲频率 5MHz,及时钟周期为 200ns ($T=1/f$).
- ② 总线周期:CPU 进行一次读或写所需时间称为一个总线周期.一个总线周期需要四个时钟周期构成.8088 一个标准总线周期为 800ns ($4/f$)

(2) 读/写总线周期时序图及解释



简化的 8086/8088 读总线周期



简化的 8086/8088 写总线周期

① T1 期间:存储器或 I/O 端口的地址通过地址总线和地址/数据总线被送出

② T2 期间:8086/8088 微处理器发送/RD 或/WR 信号及/DEN 信号,在写操作的情况下,要写入的数据出现在数据总线上
在 T2 结束时采样 READY 信号

(8088 允许存储器存储时间为 460ns,如果存储器或 I/O 设备读取时间超过这一数值,则 READY 信号低电平,在 T2 结束时检测,在 T3 后插入一个等待时间 T_w (题中会给出); T_w 开始前,CPU 再次检测 READY 信号,如果仍为低电平,在 T_w 后再插入一个 T_w ,直至检测为高电平.)
如果 READY 此时是低电平,则 T3 之后将会出现一个等待状态
这一时钟周期允许存储器有时间存取数据

③ T3 期间:如果总线周期正好是一个读总线周期,则在 T3 结束时采样数据总线

④ 在 T4 期间,所有总线信号变为无效,为下一个总线周期做准备
这个时间也是 8086/8088 采样数据总线读取存储器或 I/O 中数据的时间

另外,此时 /WR 信号的后沿传送数据给存储器或 I/O,当 /WR 信号回到逻辑 1 电平时,存储器或 I/O 被激活,写入数据

6. 8088/8086 系统总线的形成 (一般来说不常考)

(1) 总线缓冲与锁存

在 8086/8088 微处理器能与存储器或 I/O 端口一起使用前,其多路复用总线必须分离.地址线,数据线都是独立的,所以必须将地址线,数据线

分离,且地址信息锁存来保持稳定。

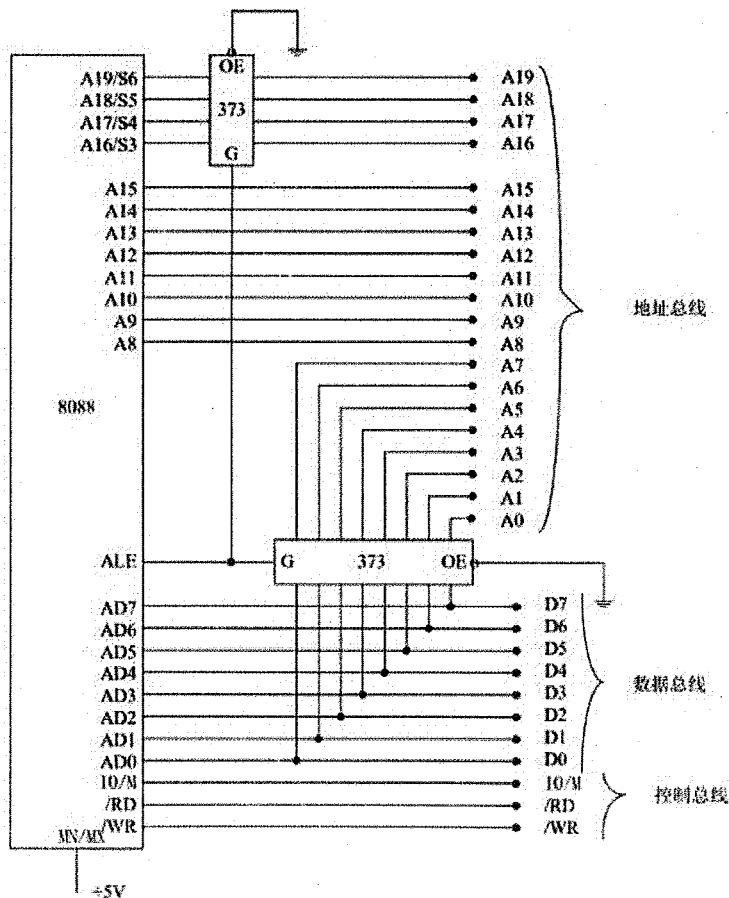
由于最大扇出为 10,所以若系统超过 10 个器件,则必须经过缓冲

7. 多路分离 8086/8088

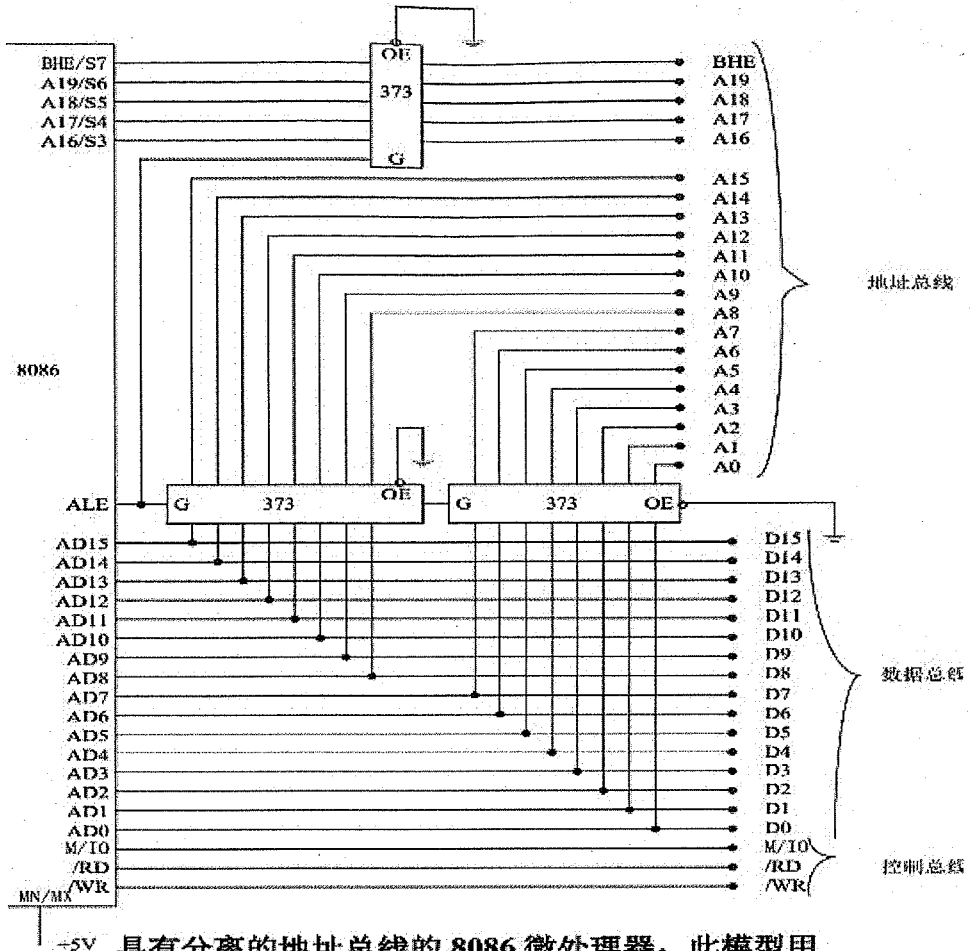
多路分离 8088

两片 74LS373 透明锁存器来分离地址/数据总线 AD7—AD0 及地址/状态线 A19/S6—A16/S3

ALE: 变为逻辑 1 时,将输入传送到输出.同时,使得锁存器记忆 ALE 变到逻辑 0 时的输入



具有分离的地址总线的 8088 微处理器, 此模型用于许多基于 8088 的系统



8. 缓冲系统

(1) 介绍

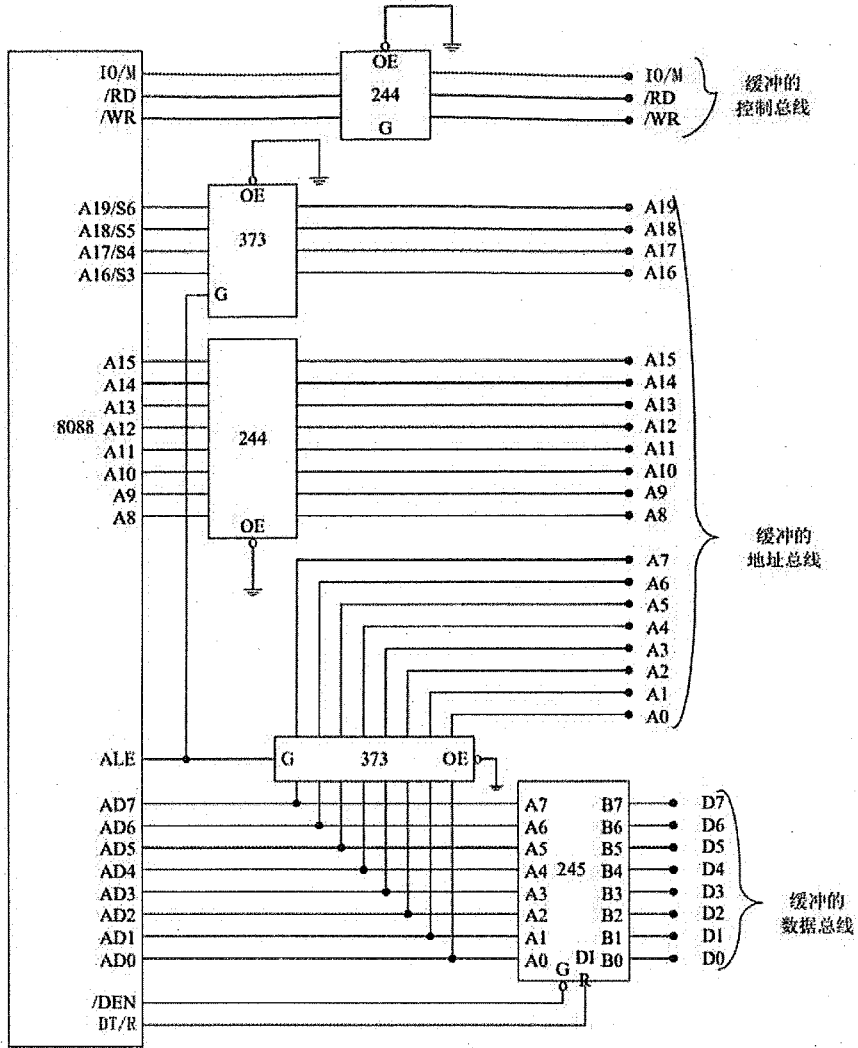
最大负载是 10

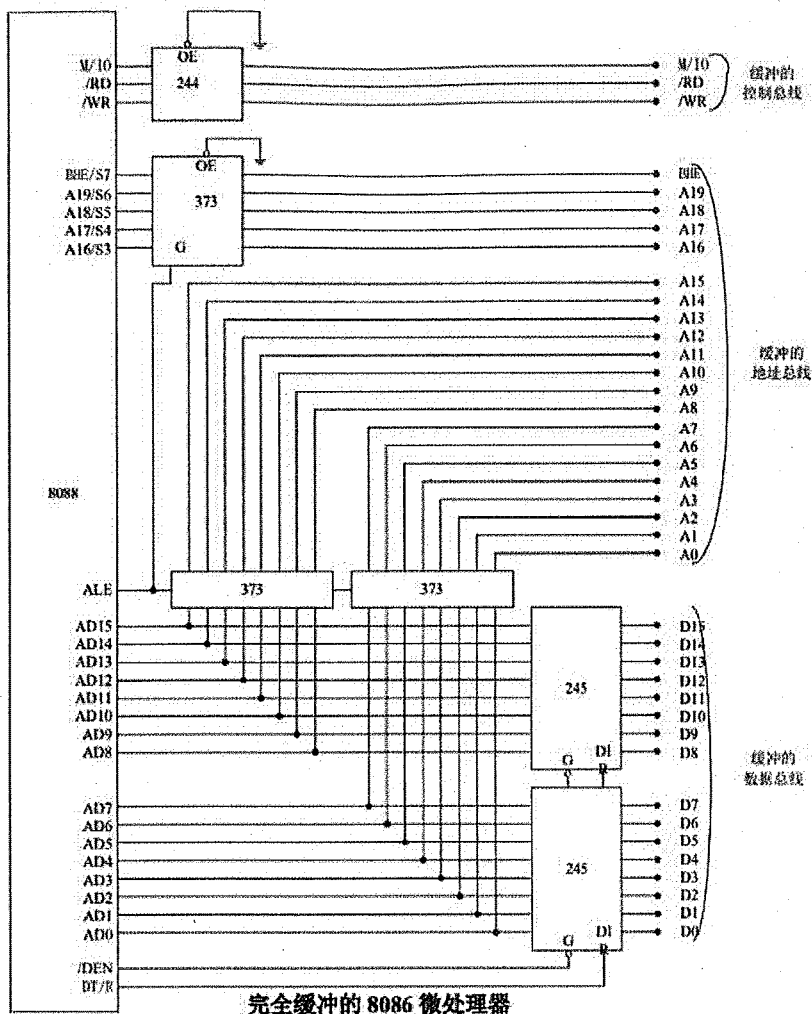
经过分离的引脚由 74LS373 锁存器缓冲

74LS244 八缓冲器

74LS245 八双向总线缓冲器

(2) 8088/8086 完全缓冲





9. 存储器分段管理

(1) 物理地址

概念:8088 有 20 根地址线,可表示的地址范围为 00000H~FFFFFH 即最大内存空间为 1MB.每个存储单元对应一个 20 位的地址,这个地址称为存储单元的物理地址.每个存储单元都有唯一的一个物理地址.

(2) 存储器分段管理

① 分段管理原因

因为 8088CPU 内部寄存器的宽度为 16 位,只能存放 16 位的二进制数据,可表示范围为 0000H~FFFFH (64K).16 位的寄存器要存放内存单元的 20 位物理地址,采用存储器分段管理方式.

② 分段原则

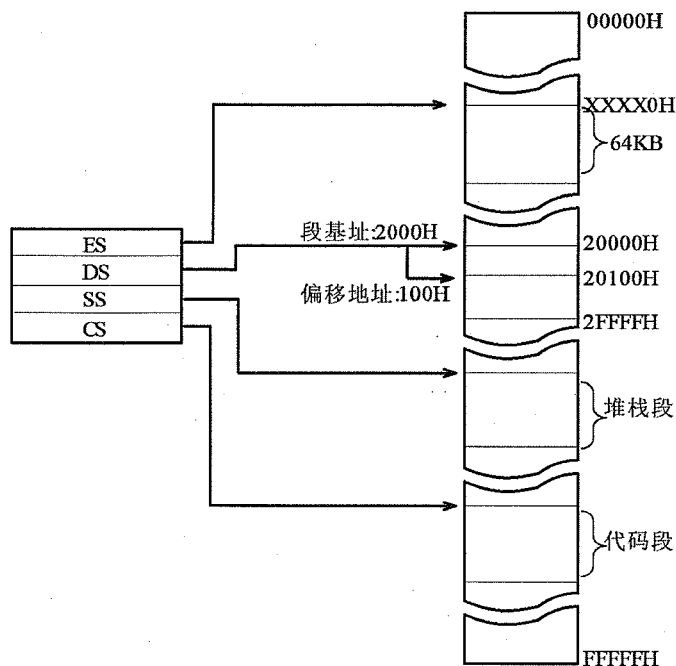
8088 将可直接寻址的 1MB 的内存空间划分成一些连续的区域,称为

段.每段的长度最大为 64KB,并要求段的起始地址必须能被 16 整除,形式如 XXXX0H.这种形式的地址,最低 4 位二进制数为 0000,所以,只要存储高 16 位就可以据此得到段的起始地址.分段管理要求每个段都由连续的存储单元构成,并且能够独立寻址,而且段和段之间允许重叠.

③ 段基址与偏移地址

8088 将 XXXXH 称为段基址,存储在段寄存器 CS, DS, SS, ES 中.

段内各存储单元地址相对于该段起始单元地址的位移量称为段内偏移量.段内偏移量从 0 开始,取值范围 0~FFFFH.



④ 逻辑地址

程序中使用的存储器地址是由段基址和段内偏移地址组成,称为逻辑地址.逻辑地址通常写成 XXXXH:YYYYH 的形式,其中 XXXXH 为段基址,YYYYH 为段内偏移地址.

⑤ 段基址和偏移地址与物理地址之间的关系:

物理地址 = 段基址 $\times$ 10H + 段内偏移地址

⑥ 存储器分段管理的作用

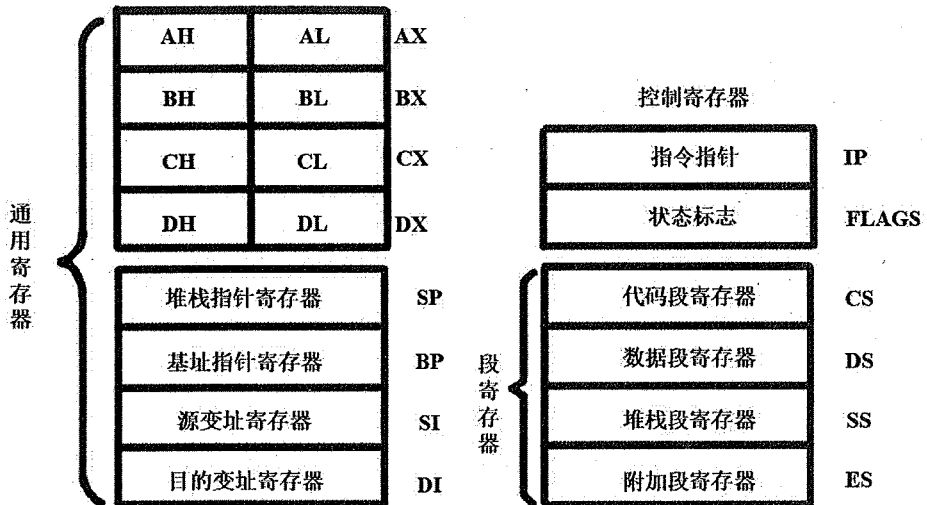
- 1° 存储器的采用分段甚至分页来管理,使得多个程序可以并行运行;
- 2° 分段寻址允许程序在存储器内重定位(浮动),可以把整个程序作为一个整体移到一个新的区域.允许实模式编写的程序在保护模式下

运行.

3° 信息按特征分段存储.

10. 8086/8088 编程结构

寄存器:含 14 个 16 位寄存器,按功能可分为三类:8 个通用寄存器,4 个段寄存器,2 个控制寄存器.



(1) 通用寄存器

AX:累加器.多用于存放中间运算结果.

BX:基址寄存器.用于内存单元的偏移地址;

CX:计数寄存器.用于在循环或串操作指令中存放循环次数或重复次数;

DX:数据寄存器.

SP:堆栈指针寄存器,其内容为栈顶的偏移地址;

BP:基址指针寄存器,通常用于堆栈段,与 SS 搭配使用

SI:源变址寄存器

DI:目标变址寄存器

(2) 段寄存器

存放逻辑段的段基址

CS:代码段寄存器

DS:数据段寄存器

ES:附加段寄存器

SS:堆栈段寄存器

(3) 控制寄存器

① 指令指针寄存器 IP:

存放下一条要执行的指令的偏移地址;

② 标志寄存器 FLAGS;

状态标志:存放运算结果的特征;

控制标志:控制某些特殊操作;

6 个状态标志位(CF,SF,AF,PF,OF,ZF)//重点

3 个控制标志位(IF,TF,DF)

CF —进位标志:算术运算时有进位 CF=1,无进位 CF=0

PF—奇偶标志:运算结果中“1” 的个数为偶数 PF=1

AF—辅助进位标志:第 3 位向第 4 位有进位时 AF=1

ZF—零标志:运算结果为“0”则 ZF=1

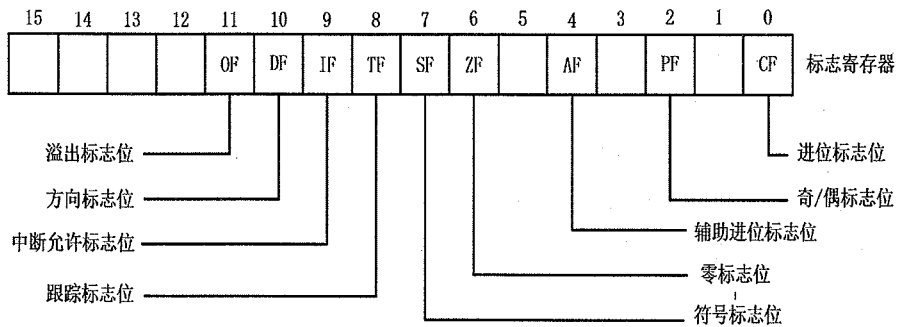
SF—符号标志:运算结果为负数时 SF=1

OF—溢出标志:运算结果超出规定范围 OF=1

IF—中断允许标志:IF=1 中断允许

DF—方向标志,DF=0 地址加 1

TF—跟踪标志,TF=1,为单步工作方式



(4) 8086 对寄存器使用的约定

内存访问类型	默认段寄存器	可指定段寄存器	段内偏移地址来源
堆栈操作	SS	无	SP
源串	DS	ES、SS	SI
目的串	ES	无	DI
BP用作基址寻址	SS	ES、DS	按寻址方式计算得到的有效地址
一般数据存取	DS	ES、SS	按寻址方式计算得到的有效地址

第三章 8086 指令系统

一、本章考点

寻址方式一般必出一道小题,指令部分必须记住并且能熟练运用每种指令.大题部分熟悉所有例题,掌握编程思路与方法.

1. 小题

(1) 寻址方式

- ① 给定 $DS=1000H, SS=2000H, DI=0100H, BP=1000H$. 假定按照实模式操作, 指出下面指令的寻址方式, 确定下面每条指令寻址的存储器地址.

1° $MOV\ AL, [BP+DI]$

2° $MOV\ CX, [DI]$

- ② 给定 $DS=1100H, BX=0200H, LIST=0250H, SI=0500H$. 实模式操作下, 确定 $MOV\ CX, LIST[BX+SI+2]$ 指令寻址的存储器地址.

- ③ 任意写出五种 8088/8086 寻址方式并举例.

【答】

- ① 操作数 AL, CX 寻址方式为寄存器寻址. 操作数 $[DI]$ 为寄存器间接寻址, 操作数 $[DI+BP]$ 为基址变址寻址.

1° 中 $[BP+DI]$ 的物理地址为 $1000+0100+2000*10=21100H$

2° 中 $[DI]$ 的物理地址为 $0100+1000*10=10100H$

- ② 物理地址 $=1100*10+0200+0250+0500+2=11592H$

- ③ 略

(2) 指令

- ① 说明 $PUSH\ BX$ 指令执行时会发生什么操作? 假设 $SP=0100H, SS=0200H$, 确切指出 BH 与 BL 分别存储在哪个存储单元中?

- ② 比较 $MOV\ DI, NUMB$ 指令与 $LEA\ DI, NUMB$ 指令的操作.

- ③ 串操作指令中, 方向标志的作用是什么?

- ④ 说明 $STOSW$ 与 $LODSW$ 的操作.

- ⑤ 串指令用 DI 与 SI 寻址那个存储器段中的数据?

- ⑥ 说明 $IN\ AL, 21H$ 指令实现什么样的功能.

- ⑦ 求 $D9H$ 与 $6EH$ 相减, 并注明受影响的标志位状态.

- ⑧ 设计短程序, 测试 AL 寄存器的最右一位与最左一位, 最右置位跳转至 $RIGHT$, 最左置位跳转至 $LEFT$

【答】

- ① BX 中的数据被压入堆栈中,BH 存储在 011FFH 中,BL 存储在 011FEH 中.
- ② LEA DI,NUMB ;取 NUMB 指代的偏移地址送到 BX 中,与 MOV DI, OFFSET NUMB 含义一致
MOV DI,NUMB;把 NUMB 指代的立即数赋给 DI
- ③ 在串操作期间,方向标志 DF 选择地址指针寄存器 DI 和 SI 是自动递增还是递减.
DF=0,自动递增;程序表示为 CLD;
DF=1,自动递减;程序表示为 STD.
- ④ STOSW,隐含操作数 AX,把 AX 的内容传送到 ES:DI 指明的串中,传送一个字.
LODSW,隐含操作数 AX,将 DS:SI 指明的串中元素传送到 AX,传送一个字.
- ⑤ 源串默认为数据段 DS,允许段超越为 SS,ES;偏移地址在 SI 中;
目的串默认在 ES 段中,偏移地址在 DI 中
- ⑥ 从 AL 存储数值的并行口中输入 21H
- ⑦

```
MOV     AL,    0D9H
MOV     BL,    6EH
SUB     AL,    BL
```

结果 AL=6BH,标志位 CF=0,PF=0,AF=1,ZF=0,SF=0,OF=1
- ⑧ TEST AL,1;测试最右一位
JNZ RIGHT;如果置位跳转至 RIGHT
TEST AL,128
JNZ LEFT

2. 大题:短程序编程

- (1) 写出一个指令序列,完成的功能是将存储器中以 TABLE 为首地址的 50 个字节的内容送到以 LIST 为首地址的连续存储单元中.
- (2) 设计一个短指令序列,把 AX 中的最左两位置位,最右 4 位取反,6,8,10 清零.
- (3) 在 String1 单元开始的存储区域中,连续存放着 100 个字符型数据,请编写程序段,将它们传送到 String2 开始的存储区域中.

【答】

- (1) TABLE DB 50 DUP(?)
LIST DB 50 DUP(?)

```
MOV AX,DS
MOV ES,AX
LEA SI,TABLE
LEA DI,LIST
MOV CX,50
REP MOVSB
END
```

- (2) OR AX,0C000H
XOR AX,000FH
AND AX,0FD5FH
- (3) MOV SI, OFFSET STRING1
MOV DI, OFFSET STRING2
MOV CX, 100
REP MOVSB

二、基本知识

1. 基本概念

- (1) 指令:一条指令由操作码和操作数两部分组成,操作码详细地说明指令要执行的操作,操作数是指令执行时需要的数据.
- (2) 操作数:
操作数可以有一个也可以有两个,还可以没有.
只有一个操作数的指令常称为单操作数指令
有两个操作数的指令常称为双操作数指令
形式上无操作数的指令,通常操作数是隐含的
- (3) 操作数类型

操作数有三种类型:立即数,寄存器操作数和存储器操作数.

- ① 立即数又称为常数
- ② 寄存器操作数

通用寄存器中 AX,BX,CX,DX,SI,DI,BP,SP 存放字操作数;

AH,AL,BH,BL,CH,CL,DH,DL 存放字节操作数;

源操作数:源即起始的意思,指被操作的数据;

目的操作数:顾名思义,目的即到达的意思,指命令操作数据最终要存放的地方.

计算后,结果放在目的操作数中(值发生改变),源操作数不变

2. 指令寻址方式

(1) 寻址方式:

- ① 概念:即获得地址的方法,主要指获得段内偏移地址的方法,段基址常采用默认方式获得.
- ② 分类: (1) 获得指令中操作数地址的方法 (2) 获得要执行的下一条指令的地址的方法.

(2) 寻址方式介绍:

① CPU 的寻址方式总结:

获得操作数所在地址的寻址方式:

立即寻址 MOV AX,3102H

直接寻址 MOV AL,[4000H]

寄存器寻址 MOV AL,BL

寄存器间接寻址 MOV AX,ES:[SI]

寄存器相对寻址 MOV AX,DATA[SI]

基址——变址寻址 MOV AX,[BX][SI]

基址——变址——相对寻址 MOV AX,DATA[BX][SI]

隐含寻址 MUL BL;

② 各种寻址方式简介

1° 立即寻址

操作数是立即数,可以是 8 位或 16 位的二进制数,也可以是字符常数.

例:MOV AX,2000H;2000H 是立即数操作数

MOV AH,'A';'A'是字符常数,等于 41H

ADD AL,6;6 是立即数操作数

2° 直接寻址

操作数在内存中,指令中直接给出操作数所在的内存单元的偏移地址.可以是数值形式的地址,也可以是符号表示的地址——符号地址.

例:MOV BX, [BUFF];

或写为 MOV BX, BUFF,定义 BUFF 指代 3200H

MOV BX,[3200H]

段超越:MOV BX,CS:[2000H]

3° 寄存器寻址

操作数在 CPU 内部的寄存器中.

例:MOV [2000H],BL;操作数 BL 为寄存器寻址.

ADD AX, BX;源和目的操作数的都是寄存器寻址.

MOV AL,BL

MOV AX,DX

MOV SI,BP

4° 寄存器间接寻址

概念:操作数在内存中,内存单元的偏移地址存放在寄存器中.

例:MOV AX, [SI];操作数[SI]的寻址方式为寄存器间接寻址;

SI 的内容为内存单元的偏移地址,DS 为段基址,以 DSX10H+SI 为首地址,取出连续两个内存单元的数据传送给 AX.

如果 DS=2000,SI=02H,指令的执行结果为 AX=C396H

20000H	01010011
20001H	01101110
20002H	10010110
20003H	11000011

图3-4 数据存放

间址寄存器:

8086CPU 中能够作为寄存器间接寻址方式使用的寄存器只有 4 个:BX,BP,SI,DI.这 4 个寄存器在作为间接寻址使用时,要用[]申明,这时常称为它们为地址指针或间址寄存器.BP 在作为间址寄存器时,段基址默认为 SS;其它 3 个的默认段基址为 DS.都可以段超越.

5° 寄存器相对寻址

操作数在内存中,内存单元的偏移地址一部分由间接寻址寄存器提供,一部分是指令给定的 8 位或 16 位地址位移量,二者相加形成操作数的有效地址.

首地址=段寄存器*10+通用寄存器地址+DATA

例:MOV AL, [SI+20H]

MOV AX, [BX]+DATA

MOV AL, 20H [SI]

MOV CX, DATA [DI]

MOV DX, DATA+ [BP]

首地址=段寄存器*10+DATA

6° 基址变址寻址

操作数在内存中,基址寄存器和变址寄存器相加作为操作数的偏移地址.

例:MOV AX, [BX][SI]

8086CPU 中寄存器 BX 和 BP 为基址寄存器,SI 和 DI 为变址寄存器.这种寻址方式中,一个基址寄存器加一个变址寄存器构成操作数,操作数的形式只有 4 种:

[BX][SI];[BX][DI];[BP][SI];[BP][DI]

8086 汇编规定以基地址为主,如果基址寄存器为 BP,则操作数的段基址默认由 SS 提供;若 BX 为基址寄存器,则段基址默认由 DS 提供.

7° 基址变址相对寻址

操作数在内存中,操作数的地址由基址寄存器加上变址寄存器再加上地址位移量构成.

例:MOV AX, DATA[BX][SI]

MOV AX, [BP][DI] DATA

假设:(DS)=8000H, (BX)=2000H, (SI)=1000H, DATA=0200H

则操作数 DATA[BX][SI]的物理地址为:

$8000H \times 10H + 2000H + 1000H + 0200H = 83200H$

8° 隐含寻址

操作码本身隐含地指明了操作数或部分操作数的地址.

例:MUL BL; AL*BL=AX

- ③ 物理地址=段寄存器地址*10+段内偏移地址,无说明段寄存器为默认段寄存器

例:MOV BX, [3200H] 设 DS=A000H

则:A000H*10H+3200H=A3200H,功能:将 A3200H 之内容送入 AL,将 A3201H 之内容送入 AH(BX 取两个内存单元的数据,一个地址存放一个内存单元的数据)

段超越:段寄存器非默认,需要加以说明.

例:MOV BX, ES:[3200H]

称为段超越,物理地址首地址为 ESX10H+3200H.

3. 8086 指令系统

(1) 数据传送指令

- ① 分类:数据传送指令细分为通用数据传送指令,端口输入输出指令,地址传送指令和标志寄存器传送指令.

一般数据传送	MOV, PUSH, POP,XCHG, XLAT, CBW, CWD
输入输出指令	IN, OUT
标志传送指令	LAHF, SAHF, PUSHF, POPF
地址传送指令	LEA, LDS, LES

② 通用数据传送指令

1° MOV

a.立即数到通用寄存器的数据传送

MOV AX, 1000H;AX=1000H

MOV SI, 037BH;SI=037BH

b.注意,给段寄存器赋值时不能直接赋值.

MOV AX, 2000H

MOV DS, AX

c.立即数到存储单元的数据传送

MOV 指令中的两个操作数的类型必须至少有一个是确定的,另一个依附这一个.

PTR 是属性运算符,功能为修改操作数的类型.

WORD PTR 的作用是将操作数的类型设置为字类型.

BYTE PTR 将操作数的类型设置为字节类型.

例:MOV BYTE PTR[SI], 4AH

将立即数 4AH 传送到内存单元,内存单元的地址以间接寻址的方式由 SI 提供,传送一个字节.

③CPU 内部寄存器之间的数据传送

例:MOV AL, BL ;BL 传送给 AL,传送一个字节.

MOV AX, BX ;BX 传送给 AX,传送一个字.

MOV DS, AX ;给数据段寄存器赋值.

MOV SI, BP ;BP 传送给 SI,传送一个字.

④寄存器与存储单元之间的数据传送

MOV AL, [2000H]

MOV AX, [SI]

MOV [3200H], CX

MOV ARRAY[DI], DL

MOV DL, [BX][SI]

MOV 指令对操作数的要求:

MOV 指令的两个操作数类型必须相同;

两个操作数不能同时为存储操作数;

不能用立即数直接给段寄存器赋值;

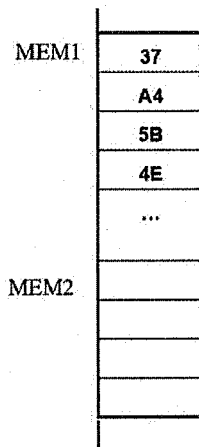
两个操作数不能同时为段寄存器 MOV DS, ES 错误!

MOV 指令不能访问 IP 和 Flags;

MOV 指令不影响标志寄存器的任何标志位 ;

不允许给 CS 赋值.

例 1 把内存中首地址为 MEM1 的 200 个字节送到首地址为 MEM2 的区域;



答:

```
MOV SI,OFFSET MEM1
MOV DI,OFFSET MEM2
MOV CX,200
NEXT: MOV AL,[SI]
      MOV [DI],AL
      INC SI
      INC DI
      DEC CX
      JNZ NEXT
      HLT
```

1) PUSH,POP 指令

a) 堆栈

堆栈是程序在内存中开辟的一个数据区,用以保存寄存器或存储器中暂时不用而又必须保存的数据.堆栈的段基址放在 SS 中,

堆栈指针为 SP;

堆栈是一种线性表,只在栈顶(低地址端)进行输入输出操作.CPU 对堆栈的操作采用先进后出(或后进先出)存取方法.

b) PUSH,POP 指令

CPU 把数据存入堆栈称为压入堆栈 PUSH,从堆栈中取出数据称为弹出堆栈 POP.

c) 堆栈存取规则

每次存取一个字(16 位);

存放数据时,将高字节存入 SP-1 单元中,低字节存入 SP-2 单元中,读取数据时正好相反;

SP 总是指向栈顶;压入操作数时

SP 减 2,弹出操作数 SP 加 2;

操作数不能是立即数;

后进先出(LIFO)的原则;

例:设 AX=1122H,SP=FFFEH,SS=1000H,PUSH AX——
AH 存入 1FFFD H 单元,AL 存入 1FFFC H 单元

③ 输入输出指令

8086CPU 对 I/O 端口的操作使用专用指令:IN ,OUT

只有累加器 AL 或 AX 可以与 I/O 端口进行数据交换;

当端口地址是 8 位的二进制数时,可以在指令中直接使用.

如 a: OUT 25H, AL; IN AL,25H;

OUT 25H, AX; IN AX,25H;

当端口地址为 16 位,不能直接使用,需要预先将其传送到 DX 寄存器中.

如: MOV DX,03F8H;

IN AL,DX; OUT DX,AL;

IN AX,DX; OUT DX,AX;

例 2 从并行口 0378H 输出一个字符'A'

答: MOV DX, 0378H

MOV AL, 'A'

OUT DX, AL

④ 地址传送指令 LEA,LDC,LES

1° 取有效地址指令 LEA:

LEA BX,BUFFER ;取 BUFFER 指代的偏移地址送到 BX 中

说明:源操作数必须是存储器操作数;目的操作数必须是 16 位通用寄存器,通常是地址指针寄存器。

MOV DI, OFFSET BUFF:与 LEA DI, BUFF 作用相同,速度更快.OFFSET 为取值运算符,又称为数值返回运算符,用以求出变量或标号的属性值。

伪指令:

SEG 运算符用以求出变量或标号所在段的段基址。

例:

```
MOV    AX, SEG BUFF    ;BUFF 的段基址送 AX.
MOV    DS, AX
```

OFFSET 用以求出变量或标号的偏移地址。

例:

```
MOV SI, OFFSET BUFF ;BUFF 的偏移地址送 AX
```

- 1) LDS SI, LIST1 ;从 LIST1 存储单元取出 4 个字节,前两个给 SI,后两个给 DS
- 2) LES DI, LIST2 ;从 LIST2 存储单元取出 4 个字节,前两个给 DI,后两个给 ES

⑤ 标志传送指令

PUSHF ;将标志寄存器 FLAGS 压入堆栈;

POPF ;将当前栈顶的内容弹出到标志寄存器 FLAGS

这两条指令常用在过程调用时保护标志位,调用结束时恢复标志位

1° 其他数据传送指令

2° XCHG 指令 (交换指令)

可以交换字节,字,或双字长度,并且可以使用除立即寻址外所有寻址方式。

例: XCHG CX,BP XCHG AL,CL

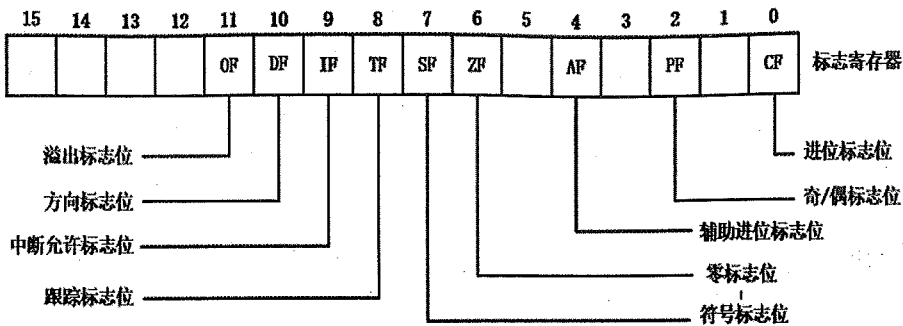
XCHG AL,DATA XCHG AL,[DI]

3° XLAT 指令 (查表指令,换码)

该指令把 AL 寄存器中的内容转换为存储在存储表中的一个数字。操作是以 DS:[BX+AL]为地址,提取存储器中的一个字节再送入 AL。这是唯一一条把 8 位数字加到 16 位数字上的指令。

(2) 算术逻辑运算指令

一般结合 FLAG 标志位情况出题.



算 术 运 算	加法指令	ADD, ADC, INC
	减法指令	SUB, SBB, DEC, NEG, CMP
	乘法指令	MUL, IMUL
	除法指令	DIV, IDIV
	十进制调整指令	DAA, AAA, DAS, AAS, AAM, AAD
逻辑运算和移位指令		AND, OR, NOT, XOR, TEST, SHL, SAL, SHR, ROL, ROR, RCL, RCR

① 加法指令

1° 不带进位的加法运算指令 ADD

ADD AL, BL ;AL+BL 结果存回 AL 中.

2° 带进位的加法运算指令 ADC

例 3 求 03AD9FH 与 25BC6EH 的和,结果存放在 DX:AX 中.

```
MOV AX, 0AD9FH ;AX=AD9FH
MOV BX, 0BC6EH ;BX=BC6EH
ADD AX, BX ;AX=6A0DH,CF=1
MOV DX, 03H ;DX=3
ADC DX, 25H ;DX=29H,
```

结果 DX:AX=296A0DH

3° 加 1 指令 INC

加 1 指令又称增量指令,指令不影响 CF 标志位.

② 减法指令

1° 不带借位 CF 的减法指令 SUB

2° 带借位 CF 的减法指令 SBB

3° 减 1 指令 DEC

DEC 指令又称为减量指令,指令不影响 CF 标志位,对其它 5 个状态标志位产生影响.

4° 操作数求补指令 NEG

功能: $(0 - \text{OPRD})$ 即对 OPRD 包括符号位在内逐位取反后加 1,结果回送到 OPRD.

例 4 **MOV AL,31H**

NEG AL ;AL=CFH,

标志位 CF=1,PF=1,AF=1,ZF=0,SF=1,OF=0

如果操作数非 0,指令的执行使 CF=1, 否则 CF=0.对全部 6 个状态标志位产生影响.

5° 比较指令 CMP

CMP OPRD1, OPRD2

OPRD1 减去 OPRD2,结果并不回送给 OPRD1.指令影响全部 6 个状态标志位.指令执行后两个操作数不变.

③ 乘法指令

1° 无符号数乘法指令 MUL

例:

MUL AH ;AL×AH 结果保存在 AX 中.

MUL BX ;AX×BX 结果保存在 DX:AX 中.

MUL BYTE PTR[SI] ;AL×[SI]结果保存在 AX 中

说明:两个乘数的数据类型要相同,指令影响标志位 CF,OF 位.

字节相乘的乘积在 AX 中,如果标志位 CF=OF=1,表明 AH 不为 0;字相乘的乘积在 DX:AX 中,如果标志位 CF=OF=1,表明乘积的高位 DX 不为 0.

2° 有符号数乘法指令 IMUL

④ 除法指令

除法指令隐含了被除数 AX 或 DX:AX,除数可以是寄存器或存储器操

作数,但不能是立即数.

1° 无符号数除法指令 DIV

例如

DIV BL ;商存到 AL 中,余数存到 AH 中

2° 带符号数的除法 IDIV

3° 符号扩展指令 CBW,CWD

使用除法指令,必须检查被除数的长度,如果不符合要求,要用位扩展指令来转换.

除法运算中,除数不能为零,也不能太小防止结果溢出.

使用符号扩展指令对数据类型进行调整.指令不影响标志位.

a) 指令格式:CBW

功能:字节转换为字.将 AL 中的数的符号位扩展至 16 位,扩展的符号部分存入 AH 中,即由 AL 扩展为 AX,值保持不变.

例:

MOV AL, 3EH ;AL=0011 1110B,下划线部分为符号位

CBW ;AX=0000 0000 0011 1110B

MOV AL, 93H

CBW ;AX=1111 1111 1001 0011B

b) 指令格式:CWD

功能:字转换为双字.(16 位转换为 32 位) 将 AX 中的数的符号位扩展至 32 位,扩展的符号部分存入 DX 中.即由 DX:AX 代替 AX,值保持不变.

⑤ 逻辑运算指令

所有逻辑运算指令对操作数任意位进行运算,清零.逻辑运算指令的执行是位对位的操作,会影响六个状态标志位,其中 CF 与 OF 会被清零(逻辑运算没有进位,也不会移出),其他状态位变化与运算结果有关.

1° 逻辑与指令 AND

指令格式:AND OPRD1, OPRD2 ;OPRD1 目的操作数,OPRD2 源操作数

例:

AND AL, 0FH ;0F=0000 1111,取 AL 的低 4 位,屏蔽高 4 位.

AND AX, BX

AND [SI], AL ;内存单元[SI]与 AL 与,结果存回内存单元

例: AX 与 BX 进行与操作:

MOV AX, 7E6DH

MOV BX, 0D563H

AND AX, BX ;AX=5461H, BX=0D563H

将 AL 中的 ASCII 码转换为 BCD 码:

MOV AL, 35H ;35H 是 5 的 ASCII 码

AND AL, 0FH ; 0F=0000 1111,取 AL 的低 4 位,屏蔽高 4 位

例:想知道 AL 中的第 5 位的值,可以先安排如下一条指令,使 AL 中的其它位都置为 0,而只保留下第 5 位的值:

AND AL, 0010 0000B

用与指令设置标志位 CF=OF=0:

AND AX, AX ;AX 不变,指令执行使标志位 CF=OF=0, AF 值不定,对 SF,PF 和 ZF 产生影响.

2° 逻辑或指令 OR

例:

OR AX, CX

OR [DI], AL

OR AL, 0FH ;AL 的低 4 位被置 1,高 4 位不变.

OR AL, 80H ;AL 的符号位置 1,其它位保持不变

例 5 对 AL 进行偶校验:

偶校验:检验 AL 中是否含有偶数个 1.是,AL 值不变;不是,AL 最高位置一使其变为偶数个 1

OR AL,AL ;PF 标志位变化,检测 AL 中是否偶数个 1,是,PF=1

JPE CONTINUE ;条件转移指令,PF=1,转移至 continue 处继续执行

OR AL,80H ;AL 中最高位置 1

CONTINUE ...

3° 逻辑非指令 NOT

指令格式:AND OPRD

功能:将 OPRD 逐位取反,结果回送 OPRD 中.

例:

```
MOV    AL, 0FH
NOTAL          ;AL=F0H
    NOT  BYTE PTR[SI]
```

4° 逻辑异或指令 XOR

功能:OPRD1 与 OPRD2 按位进行异或操作,结果回送 OPRD1 中.

异或:对应某位相同:结果为 0 对应某位不同:结果为 1

例:

```
XOR     AX, CX
XOR     BYTE PTR[DI], 4AH
XOR     AX, AX      ;AX=0,同时标志位 CF=OF=0,
```

这条指令常用于算术运算指令之前清理运算环境.

5° 测试指令 TEST

指令格式: TEST OPRD1, OPRD2

功能:OPRD1 与 OPRD2 按位进行与操作,但是结果不回送 OPRD1 中,所以指令执行后两个操作数的值保持不变.指令的执行使标志寄存器的标志位 CF=OF=0,AF 值不定,SF,PF 和 ZF 受影响.通常 ZF 位最受关注.

这条指令常用于对 OPRD1 中的特定位进行测试,OPRD2 用于说明测试 OPRD1 中的哪一位.

例:TEST AL, 01H

指令执行后 AL 的值保持不变,但标志位受到影响.如果 ZF=0 说明 AL 的第 0 位为 1,如果 ZF=1 说明 AL 的第 0 位为 0 .

① 移位指令

1° 非循环指令

(1) 逻辑左移指令 SHL

指令格式:SHL OPRD, COUNT

功能:将 OPRD 逐位进行左移,最低位第 0 位向左移到第 1 位,依次移动,最高位移出 OPRD,移到标志寄存器的 CF 中;第 0 位空出,用 0 填补.

说明:OPRD 可以是寄存器或存储器操作数,COUNT 可以为 1 或 CL.

逻辑左移 1 位相当于乘以 2 (二进制的基数),逻辑右移 1 次相当于除以 2,所以移位指令常用于简单的乘除运算.逻辑左移或逻辑右移指令将操作数看作是无符号数.

例:

SHL AL, 1

设指令执行前 AL=3AH ;0011 1010

指令执行后:CF=0, 0111 0100,AL=74H

例:

MOV AL, 35H

SHL AL, 1 ;AL=35H×2=6AH, CF=0

MOV BX, 78CDH

MOV CL, 3

SHL BX, CL ;BX=C668H, CF=1

2° 逻辑右移指令 SHR

指令格式:SHR OPRD, COUNT

功能:将 OPRD 逐位进行右移,最高位向右移到次高位,依次移动,第 0 位移出 OPRD,

移到标志寄存器的 CF 中;最高位空出,用 0 填补.

说明:OPRD 可以是寄存器或存储器操作数,COUNT 可以为 1 或 CL.

3° 算术左移指令 SAL

指令格式:SAL OPRD, COUNT

说明:将 OPRD 逐位进行左移,最低位第 0 位向左移到第 1 位,依次移动,最高位移出 OPRD,移到标志寄存器的 CF 中;第 0 位空出,用 0 填补.算术左移指令将操作数作为带符号数处理.

例:

SAL AL, 1

SAL AX, CL

SAL BYTE PTR[SI], 1

SAL DX, CL

4° 算术右移指令 SAR

指令格式:SAR OPRD, COUNT

功能:将 OPRD 逐位进行右移,最高位向右移到次高位,依次移动,第 0 位移出 OPRD,

移到标志寄存器的 CF 中;最高位保持不变.

说明:OPRD 可以是寄存器或存储器操作数,COUNT 可以为 1 或

CL.

算术右移一次相当于除 2.

5° 循环指令

(1) 不带 CF 的循环移位指令:左移 ROL,右移 ROR

循环左移的指令格式:ROL OPRD, COUNT

功能:将 OPRD 逐位进行左移,第 0 位移到第 1 位,依次移动,最高位移至第 0 位,同时最高位又移到标志寄存器的 CF 中.

循环右移的指令格式:ROR OPRD, COUNT

功能:将 OPRD 逐位进行右移,最高位移到次高位,依次移动,第 0 位移至最高位,同时最高位又移到标志寄存器的 CF 中.

说明:OPRD 可以是寄存器或存储器操作数,COUNT 可以为 1 或

CL.

例:

ROL AL, 1

设指令执行前 AL=3BH ;0011 1011

指令执行后: AL=76H ;CF=0,AL=0111 0110

(2) 带 CF 的循环移位指令:左移 RCL,右移 RCR

循环左移的指令格式:RCL OPRD, COUNT

功能:将 OPRD 逐位进行左移,第 0 位移到第 1 位,依次移动,最高位移至 CF 位,CF 位移到第 0 位.

循环右移的指令格式:RCR OPRD, COUNT

功能:将 OPRD 逐位进行右移,最高位移到次高位,依次移动,第 0 位移至 CF 位,CF 位移到最高位.

说明:OPRD 可以是寄存器或存储器操作数,COUNT 可以为 1 或

CL.

例:

RCL AL, 1

设指令执行前 AL=3BH,CF=1 ;写成 CF+AL 形式,为 1 0011 1011

指令执行后: AL=77H ;CF=0,AL=0111 0111

例 6 将 32 位数 2F6E59ABH 逻辑右移一位,结果存放在 DX:AX 中.

答:

MOV AX, 59ABH

```
MOV DX, 2F6EH
SHR DX, 1 ;DX=17B7H
RCR AX, 1 ;AX=2CD5H
```

(3) 串操作指令

① 概念

1° 简介

串或字符串是指在内存中连续存放的由字节或字组成的数据串,可以是数值型或字符型数据.串操作指令对串中数据进行操作,可以以字节为单位或以字为单位,可操作的最大串长度为 64K.

2° 种类:5 种

MOVS 串传送 CMPS 串比较 SCAS 串扫描 STOS 存入串
LODS 取串

3° 方向标志 DF

在串操作期间,DF 选择地址指针寄存器 DI 和 SI 是自动递增还是递减.

DF=0,自动递增;程序表示为 CLD;

DF=1,自动递减;程序表示为 STD.

4° 串操作指令的共同特点

- 1) 源串默认为数据段 DS,允许段超越为 SS,ES;偏移地址在 SI 中;
- 2) 目的串默认在 ES 段中,偏移地址在 DI 中;
- 3) 使用重复前缀时,串长度放在 CX 寄存器中;
- 4) 地址指针和计数器自动修改.
- 5) 使用串操作指令之前,应预先设置源串指针 DS:SI,目的串指针 ES:DI,重复次数 CX (例如 MOVS 指令,本身只传送一个字节,必须加重复前缀实现循环),方向标志位 DF;

② 串传送指令 MOVS

指令格式: MOVSB,MOVSW,MOVS OPRD1,OPRD2

功能:字符串传送. (不加前缀只重复一次)

MOVSB,隐含操作数,将 DS:SI 指明的一个字节传送到 ES:DI 指明内存单元中.

MOVSW,隐含操作数,将 DS:SI 指明的两个字节传送到 ES:DI 指明的内存单元中.

MOVS OPRD1,OPRD2,这种形式常常用在源串段超越的情况下.

说明:指令不影响状态标志位.

MOVS 指令常与无条件重复前缀 REP 联合使用:

如: REP MOVSB;

 REP MOVSW;

例 7 将数据段中 STRING 1 中 100 个字节传送到附加数据段中的 STRING 2 中:

答:

MOV SI, OFFSET STRING1 ;初始化源串指针

MOV DI, OFFSET STRING2 ;初始化目的指针

MOV CX, 100 ;初始化计数器

CLD ;设置 DF=0,使 SI 和 DI 按增量变化,增量为 1

REP MOVSB ;自动重复传送,直到 CX=0

如果希望一次传送两个字节,则程序改为:

MOV SI, OFFSET STRING1

MOV DI, OFFSET STRING2

MOV CX, 50

CLD

REP MOVSW

如果 STRING 1 在附加数据段中,程序应为:

MOV SI, OFFSET STRING1

MOV DI, OFFSET STRING2

MOV CX, 100

CLD

REP MOVSB PTR ES:[DI], BYTE PTR ES:[SI]

说明:重复前缀 REP 的重复次数由 CX 决定.串指令每执行一次,CX 的内容就减 1;当 CX 为 0,串传送指令执行结束.

③ 串比较指令 CMPS

1° 指令格式: CMPSB CMPSW CMPS OPRD1,OPRD2

2° 功能:数据串比较.

3° CMPSB,隐含操作数,将 DS:SI 指明的串与 ES:DI 指明的串中的对应元素进行比较,以字节为单位.

CMPSW,隐含操作数,将 DS:SI 指明的串与 ES:DI 指明的串中的对应元素进行比较,以字为单位.

MOVS OPRD1,OPRD2,这种形式常常用在源串段超越的情况下.

说明:指令影响状态标志.两个串中的对应元素做减法操作,影响标志寄存器,主要看 ZF 数值.ZF=0,两串相同;否则不同.比较完成后两个串不改变.

4° 常用的重复前缀为:REPE 或 REPNE

REPE CMPSB ;相等时重复比较(REPZ CMPSB)

REPNE CMPSB ;不相等时重复比较(REPNZ CMPSB)

详细说明:

REPE:相等时重复,即 ZF=1,且(CX)≠0 时重复;

REPNE:不相等时重复,即 ZF=0,且(CX)≠0 时重复

REPE CMPSB 指令动作如下:

数据比较;

SI 和 DI 自动增量(或减量);

CX 自动减 1;

如果 ZF=1 且 CX≠0,继续比较下一个数据;否则指令执行结束

例 8 将 STRING 1 与 STRING 2 进行比较;

答:

MOV SI, OFFSET STRING1 ;初始化源串指针

MOV DI, OFFSET STRING2 ;初始化目的指针

MOV CX, 100 ;初始化计数器

CLD ;设置 DF=0,使 SI 和 DI 按增量变化,增量为 1

REPE CMPSB ;对应元素相等时继续比较后面的数据,直到 CX=0

结果 1:两个串比较结束,完全相同,CX=0,指令执行结束,这时标志位 ZF=1.

结果 2:两个串中有不一样的元素,使得指令执行结束,这时 CX≠0,标志位 ZF=0.注意此时两个变址指针 SI,DI 指向下一个待比较的元素.

例 9 从地址 1000:10A0H 开始的区域中存放着 100 个字节的字符串 STRING1,从地址 2000:10B0H 开始的区域中存放着 100 个字

节的字符串 **STRING2**,将 **STRING 1** 与 **STRING 2** 进行比较,如果二者完全相同就将 **AL** 置 0,否则 **AL** 置 -1,并将第一个不相同的元素的地址存入 **BX** 寄存器。

答:

MOV AX, 1000H

MOV DS, AX

MOV AX, 2000H

MOV ES, AX

MOV SI, 10A0H

MOV DI, 10B0H

MOV CX, 100

CLD

REPE CMPSB

JNZ UNEQUAL ;JNZ 是条件转移指令,如果 ZF=0,则转到 UNEQUAL

MOV AL, 0 ;否则把 AL 置 0,转到 EQUAL

JMP EQUAL

UNEQUAL: MOV AL, -1 ;把 AL 置 -1

DEC SI ;指向第一个不相同的元素

MOV BX,SI ;保存第一个不相同的元素的地址

EQUAL: HLT ;HLT 暂停

④ 串扫描指令 SCAS

1° 指令格式: SCASB SCASW

2° 功能:在 ES:DI 指明的串中搜索特定数据.特定数据存在 AL (或 AX) 中.

3° SCASB,隐含操作数 AL,将串中的元素与 AL 进行比较,以字节为单位
SCASW,隐含操作数 AX,将串中的元素与 AX 进行比较,以字为单位.
说明:串中的元素与 AL 或 AX 做减法操作,结果不回送,只影响状态标志.

4° 执行 REP NZ SCASB 指令会有两种结果:

整个串与 AL 比较完,没有找到与 AL 相同的数据,CX=0, ZF=0 指令执行结束.

找到与 AL 一样的元素,ZF=1,指令执行结束,而 CX≠0.注意此时变址指针 DI 指向下一个待比较的元素.

例 10 搜索字符串 **STRING1**, 查是否含有字符“A”, 并确定首次找到字符“A”的位置, 将地址存入 **POINT**; 如果没找到, 将 0 存入 **POINT**.

答:

```
MOV AX, SEG STRING1
MOV ES, AX ;初始化附加段
MOV DI, OFFSET STRING1 ;设置字符串指针
MOV CX, 100 ;设置字符串长度,即循环次数
CLD ;DF=0
MOV AL, 'A' ;要查找的内容存入 AL 中
REPNZ SCASB ;与 AL 不相同继续搜索
JZ FOUND
MOV DI, 0
JMP DONE
FOUND: DEC DI
DONE: MOV POINT, DI
HLT
```

⑤ 存串指令 STOS

指令格式: STOSB

STOSW

功能: 把数据存入串中. 把 AL 或 AX 中的数据传送到 ES:DI 指明的串中.

STOSB, 隐含操作数 AL, 把 AL 的内容传送到 ES:DI 指明的串中, 传送一个字节.

STOSW, 隐含操作数 AX, 把 AX 的内容传送到 ES:DI 指明的串中, 传送一个字.

说明: 指令不影响状态标志. 一般带有重复操作前缀 REP, 相当于用 AL 或 AX 初始化某数据区.

⑥ 取串指令 LODS

指令格式: LODSB LODSW LODS OPRD

功能: 从串中取数据. 将 DS:SI 指明的串中元素传送到 AL 或 AX 中.

LODSB, 隐含操作数 AL, 将 DS:SI 指明的串中元素传送到 AL, 传送一个字节.

LODSW, 隐含操作数 AX, 将 DS:SI 指明的串中元素传送到 AX, 传送一个字.

说明:指令不影响状态标志,一般不带有重复操作前缀。

例 11 从字符串 **STRING1** 中取出 **100** 个字符,加入字符属性修饰符后存入 **B800:0** 的内存区域中。

答:

```
MOV AX, SEG STRING1
MOV DS, AX ;初始化数据段
MOV SI, OFFSET STRING1 ;设置源串指针
MOV AX, 0B800H
MOV ES, AX ;初始化附加段
MOV DI, 0
MOV AX, 9c00H ;拼接字符属性
MOV CX, 100 ;设置字符串长度,即循环次数
CLD ;DF=0
AGAIN: LODSB
STOSW
DEC CX
JNZ AGAIN
```

HLT

(4) 程序控制指令

① 转移指令

1° 无条件转移指令 JMP

1) 相关概念:

- 计算机程序的执行完全按照 CS:IP 的指向执行指令.通常情况下 CS 保持不变,IP 自动增量,程序就按照指令的先后顺序执行.无条件转移指令会修改 CS 和 IP 的值,使程序跳转到另一个位置去执行,改变指令的执行顺序.
- 根据程序的转移范围可分为段内转移和段间转移.
在同一段的范围之内进行转移,只需要修改 IP 的值,称为段内转移.如果 CS 的值也被修改,意味着程序将转移到另外的段去执行,这称为段间转移.段间转移不仅修改段基址 CS 的值,还修改 IP 的值.
- JMP 指令不影响标志位.

2) 段内转移

指令格式: JMP OPRD

说明:根据 OPRD 的类型又分为段内直接转移和段内间接转移.

例如:

段内直接转移:

JMPLABEL

JMP SHORT LABEL

JMP NEAR LABEL

段内间接转移,操作数 16 位

JMP BX

JMP WORD PTR[BX+DI]

说明:

LABEL 为指令标号,对应一条指令,是这条指令的符号地址

JMP LABEL

;程序转移到 LABEL 指明的指令处继续执行.

JMP SHORT LABEL

;程序转移到 LABEL 指明的指令处继续执行.SHORT 为属性说明符,说明转移范围,以当前 IP 为中心,转移范围-128~+127.

JMP NEAR LABEL

;程序转移到 LABEL 指明的指令处继续执行.NEAR 为属性说明符,说明转移范围,以当前 IP 为中心,转移范围-32768~+32767.

JMP BX

;将 BX 的值传送给 IP,程序转移到 CS:IP 处继续执行.操作数可以是所有 16 位通用寄存器.

JMP WORD PTR[BX+DI]

;从[BX+DI]指明的内存区域连续取出两个字节传送给 IP,程序转移到 CS:IP 处继续执行.操作数可以采用各种寻址方式.

例:

...

MOV DI, 0

JMP DONE ;跳转到 DONE 处执行

FOUND: DEC DI

DONE: MOV POINT, DI

MOV AX, 1234H

...

JMP CX ;CX=2000H,程序跳转到段内偏移地址为2

000H 处

...

段间转移

指令格式: `JMP OPRD`

功能:段间转移, $IP \leftarrow OPRD$ 的段内偏移地址, $CS \leftarrow OPRD$ 所在段的段基址.

说明:根据 OPRD 的类型又分为段间直接转移和段间间接转移.指令不影响标志位.

例如:

段间直接转移:`JMP FAR LABEL`

段间间接转移:`JMP DWORD PTR [BX+DI]`

说明:

`JMP FAR LABEL`

;程序转移到 LABEL 指明的指令处继续执行.FAR 说明标号 LABEL 在另外的代码段,与 JMP 指令本身不在同一段.这条指令执行的操作是 $IP \leftarrow LABEL$ 的偏移地址, $CS \leftarrow LABEL$ 所在段的段基址,程序转移到 $CS:IP$ 处继续执行.

`JMP DWORD PTR[BX+DI]`

;从[BX+DI]指明的内存区域连续取出 4 个字节,前两个字节(低地址)传送给 IP,后两个字节送给 CS,程序转移到 $CS:IP$ 处继续执行.操作数属于存储器操作数,可以采用各种存储器的寻址方式.

2° 条件转移指令

1) 相关概念:

2) 条件转移指令先测试条件,若条件成立则执行转移操作;若不成立则不转移并顺序执行下一条指令.所有的条件转移指令转移范围 $-128 \sim +127$,属于段内短转移,都不影响状态标志位.

3) 指令格式: `JCC OPRD`

4) 功能:若条件成立则转移到 OPRD 处执行, $IP \leftarrow IP + \text{位移量}$.

5) 说明:J 是 JUMP 的缩写,CC 表示转移的条件,OPRD 通常是标号.

6) 指令具体如下:

指令名称	汇编格式	转移条件	功能说明
进位转移	<code>JC target</code>	$(CF) = 1$	有进位或借位
无进位转移	<code>JNC target</code>	$(CF) = 0$	无进位或借位

等于或为零转移	JE/JZ target	$(ZF)=1$	相等或结果为 0
不等于或非零转移	JNE/JNZ target	$(ZF)=0$	不相等或结果不为 0
奇偶校验为偶转移	JP/JPE target	$(PF)=1$	有偶数个 1
奇偶校验为奇转移	JP/JPO target	$(PF)=0$	有奇数个 1
结果为负转移	JS target	$(SF)=1$	为负数
结果为正转移	JNS target	$(SF)=0$	为正数
溢出转移	JO target	$(OF)=1$	溢出
不溢出转移	JNO target	$(OF)=0$	不溢出
大于则转移	JA/JNBE target	$(CF)=0$ 或 $(ZF)=0$	无符号数
大于或等于则转移	JAE/JNB target	$(CF)=0$	无符号数
小于则转移	JB/JNAE target	$(CF)=1$	无符号数
小于或等于则转移	JBE/JNA target	$(CF)=1$ 或 $(ZF)=1$	无符号数
大于则转移	JG/JNLE target	$(SF)=(OF)$ 且 $(ZF)=0$	带符号数
大于或等于则转移	JGE/JNL target	$(SF)=(OF)$	带符号数
小于则转移	JL/JNGE target	$(SF)\neq(OF)$ 且 $(ZF)=0$	带符号数
小于或等于则转移	JLE/JNG target	$(SF)\neq(OF)$ 且 $(ZF)=1$	带符号数
CX 内容为 0 转移	JCXZ target	$(CX)=0$	

例 12 测试 AX 为奇数还是偶数,如是奇数则 BX 置成 0FFFFH;如是偶数 BX 置成 0.

答:

```

TEST AX, 01H ;测试 BX 中最低位的逻辑值
JZ EVEN ;ZF=1,AX 为偶数转移至 EVEN 处执行
MOV BX, 0FFFFH ;AX 为奇数,设置奇数标志
JMP CON
EVEN: MOV BX, 0 ;设置偶数标志
CON: ...

```

例 13 AX 与 BX 均为无符号数,测试 AX 与 BX 的大小
答:

```

CMP AX, BX ;比较 AX 与 BX
JZ EQUAL ;AX=BX 则转移到 EQUAL 处
JA LAG ;AX>BX,则转至 LAG 处
JMP CON ;AX<BX,转移至 CON 处
EQUAL: MOV CX, 0 ;置等于标志
LAG: MOV CX, 0FFFFH ;置大于标志
CON: ...

```

② 循环控制指令

循环控制指令共有 3 条,都是利用 CX 作为计数器控制循环,都不影响标志位.

指令格式:LOOP OPRD

LOOPE (或 LOOPZ) OPRD

LOOPNE (或 LOOPNZ) OPRD

功能:如果条件满足就转到 OPRD 指明的指令执行,继续循环;当条件不满足,循环结束,顺序执行下一条指令.

LOOP OPRD

;功能为: $CX \leftarrow CX - 1$ 并测试 CX,如果 $CX \neq 0$ 就转到 OPRD 指明的指令执行,继续循环;当 $CX = 0$ 时,循环结束,顺序执行下一条指令.

例 14 编写程序,求 $1+2+3+4+\dots+100$ 的累加和,结果存于 AX 中.
答:

```

MOV CX, 100 ;初始化计数器 CX
MOV AX, 0 ;累加和单元清 0
ABC:ADD AX, CX ;求累加和
LOOP ABC ;CX≠0 继续循环;CX=0 循环结束,执行下一条指令

```

HLT

③ 过程调用指令

主程序可用调用指令调用子程序,子程序执行结束后自动返回主程序,主程序继续执行.

1° 调用指令 CALL

根据子程序所在的位置,调用指令分为段内调用和段间调用.段内调用只修改 IP 的值,段间调用 CS 和 IP 的值都被修改.

CALL 指令在执行时第一步首先保存断点,以便子程序返回主程序时从断点处继续执行;第二步取出子程序的入口地址赋给 IP 或 CS:IP,转去执行子程序.

指令格式:CALL OPRD

功能:调用 OPRD 指明的子程序.

说明:OPRD 可以是子程序的名字或指令标号,可以是 16 位的寄存器,还可以是两个或 4 个存储单元的内容.

例:

段内调用:

CALL NEAR DELAY

DELAY 是子程序名,NEAR 是属性说明符,说明 DELAY 子程序与这条 CALL 指令在同一个代码段中,NEAR 可以省略.指令执行时首先将当前 IP 的内容压栈,然后 $IP \leftarrow IP + 16$ 位位移量,程序就转移到子程序执行.16 位位移量指的是 CALL 指令的下一条指令与 DELAY 之间的差值.这种 CALL 指令最常见,也称为段内直接调用.

CALL AX

子程序的入口地址由 AX 提供,即将 AX 的内容赋给 IP,其他动作与上面相同.

CALL WORD PTR [BX]

子程序的入口地址由[BX]指明的两个内存单元提供,其他动作与上面同.这两条指令也称为段内间接调用.

段间调用:

CALL FAR MEM

MEM 是子程序名,FAR 是属性说明符,说明 MEM 子程序与这条 CALL 指令不在同一个代码段中,FAR 不能省略.指令执行时首先将当前 CS 和 IP 的内容压栈,然后将 MEM 入口地址的段基址取出

来赋给 CS,将偏移地址取出来赋给 IP,程序就转移到子程序执行.这种 CALL 指令较常见,也称为段间直接调用.例如:

CALL 2000H:0100H ;指令直接给出子程序的段基址和偏移地址.

CALL DWORD PTR [SI]

子程序的入口地址由[SI]指明的 4 个内存单元提供,其中 [SI+1]:[SI+2]的内容赋给,[SI+3]:[SI+4]的内容赋给 CS,其他动作与上面同.这条指令也称为段间间接调用.

2° 返回指令

返回指令放在子程序的末尾,它能返回调用程序.返回指令的操作是弹出断点地址,送给 IP 和 CS.返回指令与 CALL 指令有关联,如果是段内调用,返回指令只弹出 IP 的值;

如果是段间调用,返回指令弹出 IP 和 CS 的值.虽然包含的操作不同,但返回指令的格式却一样.返回指令不影响标志位.

指令格式:RET

功能:返回调用程序.

④ 中断指令

1° 中断相关概念:

中断,是指在程序执行过程中,出现某种紧急事件,CPU 暂停执行现行程序,转去执行事件处理程序,执行完后再返回到被暂停的程序继续执行.

引起中断的事件称为中断源,计算机的中断源可能是某个硬件部件,也可能是软件,中断指令就是软件中断源.

2° 中断执行过程

①FLAGS 压栈

②FLAGS 中的 TF 置 0,IF 置 0 (禁止单步,可屏蔽中断)

③CS 压栈

④IP 压栈

⑤查中断向量表, $n \times 4$ 作为地址,从该地址中取出中断服务程序的入口地址送 CS:IP.

中断指令执行时清除 IF 和 TF,使微处理器禁止可屏蔽中断请求,禁止单步执行中断服务程序.

3° 中断指令

(1) INT 指令

指令格式:INT n

说明:n 为中断类型码,取值范围 0~255.INT 指令为 2 个字节,第一个字节是操作码,第二个字节是中断类型码.

(2) 中断返回指令

指令格式:IRET

功能:中断返回

说明:所有中断服务程序中的最后一条指令都是 IRET,它的具体操作如下:

弹出 IP

弹出 CS

弹出 FLAGS

(3) INT3 指令

指令格式:INT3

说明:该指令用于在软件调试中设置断点,方便调试程序.INT3 指令是单字节指令,其他软中断指令都是两字节指令.

(4) INTO

说明:INTO 为溢出中断,相当于 INT 4 指令.INTO 指令执行时检查 OF 位,若 OF=1,则进入中断类型码为 4 的中断服务程序;若 OF=0,则指令执行结束.

(5) 处理器控制指令

处理器控制指令对 CPU 实施控制,使 CPU 暂停,使 CPU 等待等等,还包括对 FLAGS 中的一些标志位进行设置的指令.指令如下:

HLT: 停止软件的执行.使 CPU 进行暂停状态,直到有复位 (RESET) 信号或 DMA 操作或着中断请求时,退出暂停状态.

WAIT:8086/8088 检测/TEST 引脚,若/TEST 引脚为逻辑 1 电平,WAIT 指令结束,顺序执行下一条指令.若/TEST 引脚为逻辑 0 电平,CPU 进入等待状态,直至/TEST 引脚为高电平.

ESC:转义 (ESC) 指令,在多处理器系统中 CPU 把控制权交给协处理器.并使协处理器能取得操作码和操作数进行操作.

LOCK:LOCK 可以作为任何 CPU 指令的前缀,锁存指令,它是单字节指令.它使总线锁存信号 LOCK 引脚为逻辑 0,通常用来禁止外部总线上的主控制器或其他部件,直到指令执行结束,如 LOCK: MOV AL,

[SI].

指令格式	操作说明
CLC	清进位标志位, $CF = 0$
STC	置进位标志位, $CF = 1$
CMC	进位标志位取反, CF
CLD	清方向标志位, $DF = 0$, 串操作从低地址开始到高地址
STD	置方向标志位, $DF = 1$, 串操作从高地址开始到低地址
CLI	清中断标志位, $IF = 0$, 关中断
STI	置中断标志位, $IF = 1$, 开中断
HLT	处理器暂停, CPU 不做任何操作
WAIT	处理器等待, 等待 TEST 引线转为低电平
ESC	处理器把控制权交给协处理器
LOCK	总线封锁, 它可以作为任何 CPU 指令的前缀
NOP	空操作, 消耗 3 个时钟周期, 常用于延时程序

第四章 汇编语言程序设计

一、本章考点

1. 小题部分

重点掌握 DOS 功能调用,汇编语言程序开发过程.

例:

- (1) 简述汇编语言程序开发过程.
- (2) BCD 数怎么转换为 ASCII 码?
- (3) 如果 AH 内容是 4CH,则 INT 21H 实现什么功能?
- (4) 从键盘输入单个字符可以使用 DOS 功能调用,请写出调用的指令段.

答:

- (1) 建立汇编语言源程序

生成目标程序

生成可执行文件

连接程序 LINK.EXE 把 MASM 产生的目标文件 (OBJ),与其他目标文件及系统提供的一些库文件连接在一起,生成以 EXE 为扩展名的可执行文件.

调试与运行程序

- (2) 加上 3030H
- (3) 返回操作系统
- (4) MOV AH,1
INT 21H

2. 大题部分:考察编程.掌握本章 12 道例题.

二、基础知识

1. 汇编语言源程序

用汇编语言编写的程序称为汇编语言源程序.

- (1) 汇编语言源程序结构

汇编语言源程序结构是指语句的格式和程序的组成部分.

汇编语言源程序结构如下:

数据段名 SEGMENT

.....

数据段名 ENDS

附加数据段名 SEGMENT

.....

附加数据段名 ENDS

堆栈段名 SEGMENT

.....

堆栈段名 ENDS

代码段名 SEGMENT

.....

代码段名 ENDS

END XXX

每段以 SEGMENT 开始,以 ENDS 结束,整个程序以 END 语句结束.

例 1 将 STRING 1 中 100 个字节传送到 STRING 2 中

答:

DATA SEGMENT ;定义数据段

STRING1 DB 100 DUP (55H)

DATA ENDS ;数据段结束

EDATA SEGMENT ;定义附加段

STRING2 DB 100 DUP (?)

EDATA ENDS ;附加段结束

STACK SEGMENT ;定义堆栈段

DW 256 DUP (?)

STACK ENDS ;堆栈段结束

CODE SEGMENT ;定义代码段

ASSUME CS:CODE, DS:DATA, ES:EDATA, SS:STACK

START: MOV AX, DATA

MOV DS, AX ;初始化 DS

MOV AX, EDATA

MOV ES, AX ;初始化 ES

MOV AX, STACK

MOV SS, AX

MOV SI, OFFSET STRING1 ;初始化源串指针

```

MOV    DI, OFFSET STRING2    ;初始化目的指针
MOV    CX, 100                ;初始化计数器
CLD                                ;设置 DF=0,使 SI 和 DI 按增量变化,增量为 1
REP    MOVSB
MOV    AH, 4CH
INT    21H
CODE   ENDS                    ;代码段结束
END     START

```

(2) 汇编语言的语句格式

汇编语言源程序中一行只能写一个语句.每个语句可以有 4 部分:标号,操作码助记符,操作数助记符,注释.

指示性语句:又称为伪操作语句,它不是 8088/8086CPU 的指令.源程序汇编后指示性语句不生成目标代码,所以常被称为伪指令.

例:DATA SEGMENT ;定义数据段

STRING1 DB 100 DUP (55H)

DATA ENDS ;数据段结束

指令性语句:

指令性语句是可执行语句,是 8088/8086CPU 的指令.源程序汇编后指令性语句生成目标代码.

(3) 汇编语言源程序的处理过程

汇编程序 (Assembler) 就是处理汇编语言源程序的系统程序,处理的过程称为汇编.

源程序经过汇编生成目标程序,目标程序经过连接程序连接,就得到可执行文件.

(4) 汇编语言中的操作数

① 常量:常量也称常数,有数值常量和字符常量两种.

数值常量可以是二进制数,十进制数和十六进制数.十六进制数若是以字母 (A~F) 开始,需在前面加一个数字 0,用以说明这是数值常量,不是字符串.

② 变量:变量是指存储单元中的一组数据.每个变量可以有一个名字 (变量名),也可以没有.一个变量名可以表示一个数据或一组类型相同的数据.

变量名是变量的符号地址.例如: STR DB 'STRING'

类型属性:变量的数据类型,有:BYTE,WORD,DWORD,DQ(8 个字节),DT(10 个字节)。

③ 标号与名字

标号与名字都是符号地址,都可以作为指令的操作数使用。

④ 表达式:表达式由常量,变量和标号通过运算符结合而成.表达式中的运算在汇编时完成,运算结果可以是操作数也可以是操作数地址。

常用运算符:算术运算符: +, -, *, /, MOD ;逻辑运算符:AND,OR,XOR,NOT ;关系运算符:EQ(相等),NE(不等),LT(小于),GT(大于),LE(小于或等于),GE(大于或等于) ;取值运算符:OFFSET,SEG ;修改属性运算符:PTR,修改操作数的类型,操作仅限于本条指令。

2. 伪指令

汇编语言中的指示性语句也称为伪指令.伪指令由汇编程序处理,不生成目标代码,不参与程序的执行。

(1) 段定义伪指令

段定义伪指令 SEGMENT 和 ENDS

格式:段名 SEGMENT [定位类型] [组合类型] [类别]

...

段名 ENDS

例如:

```
DATA      SEGMENT
BUFFER    DB    100 DUP(?)
COUNT    DW    100
DATA      ENDS
```

(2) ASSUME 伪指令

格式:ASSUME 段寄存器名:段名[,段寄存器名:段名],...

ASSUME 伪指令说明段名和段基址寄存器之间的关系,但它不能给段寄存器赋值,段寄存器的值需要在代码段中由指令性语句赋值。

例如:

```
ASSUME    CS:CSEG, DS:DSEG, SS:SSEG, ES:EDSEG
```

说明 CSEG 段是代码段,DSEG 段是数据段,SSEG 段是堆栈段,EDSEG 段是附加数据段。

例 2 测试内存 TAB 单元内的数为奇数还是偶数,如是奇数则 BX 置成 0FFFFH;如是偶数 BX 置成 0。

答:

```

DSEG SEGMENT      ;默认定位类型 PARA,默认组合类型 NONE
TAB      DB ?
DSEG     ENDS
SSEG SEGMENT 'STACK'
        DW 256 DUP (0)
SSEG     ENDS
CSEG     SEGMENT      ;默认定位类型 PARA,默认组合类型 NON
E
ASSUME   CS:CSEG, DS:DSEG, SS:SSEG
START:   MOV     AX, DSEG
        MOV     DS, AX                ;给 DS 段寄存器赋值
        MOV     AX, SSEG
        MOV     SS, AX                ;给 SS 段寄存器赋值
        MOV     AL, TAB
        TEST    AL, 01H      ;测试 BX 中最低位的逻辑值
        JZ      EVEN1        ;ZF=1, AL 为偶数转移至 EVEN 处执行
        MOV     BX, 0FFFFH    ;AL 为奇数,设置奇数标志
        JMP     CON
EVEN1:   MOV     BX, 0          ;设置偶数标志
CON:     MOV     AH, 4CH
        INT     21H
CSEG     ENDS                ;代码段结束
        END     START

```

(3) 数据定义伪指令

数据定义伪指令也称为变量定义伪指令,或存储单元分配伪指令.它用来定义变量,确定变量的类型,给变量赋初值,为变量分配存储空间等.

格式: [变量名] 伪操作助记符 [操作数 1][,操作数 2]...

```

STR      DB 'STRING'
NUM      DW 0AAH,23H
LAB0     DQ 01A4578H

```

数据变量有 5 种定义命令,如下:

DB:定义变量为字节类型,其后的每个操作数都占一个字节.

DW:定义变量为字类型,其后的每个操作数都占两个字节.

DD:定义变量为双字类型,其后的每个操作数都占二个字,即四个字节.

DQ:定义变量为四个字类型,其后的每个操作数都占四个字,即八个字节.

DT:定义变量为十个字节,其后的每个操作数都占十个字节.

例:NUM DD 0AAH,3EF6AC23H

;定义 NUM 为双字类型变量,2 个双字类型的操作数顺序存储在以 NUM 为首地址的连续内存单元中,每个操作数占 4 个内存单元.

- 数据定义伪指令中的操作数可以是数值型常量,字符串常量,也可以是常量表达式,还可以是问号?,问号表示预留相应数量的存储单元,但不存入数据.

例:

DATA1 DW 16*9, 55*3

DATA2 DB ?,?

变量 DATA2 有 2 个字节类型的操作数,
为每个操作数预留 2 个存储单元,不进行初始化.

- 如果操作数很多而且相同,可以使用重复数据操作符 DUP 定义变量.

例:

DATA3 DB 6 DUP (AAH)

DATA4 DB 3 DUP (?,55H,?)

变量 DATA3 有 6 个操作数,初始化为 AAH;
变量 DATA4 有 3 组操作数,每组为 3 个,
共 9 个字节类型的操作数.

(4) 符号定义伪指令

① EQU 伪指令

格式:符号名 EQU 表达式

不能呢对一个符号重复定义.

例:

CONS EQU 10 ;常数赋给符号 CONS

运用:用法:

MOV AL, ALPHA

CMP AL, CONS

符号提前定义.

② = 伪指令

格式: 符号名=表达式

说明:功能与 EQU 一样,给符号赋值,唯一的区别是可以对一个符号名重复定义.一般等号伪指令定义数值常量.

例如:

NUM=8

NUM=NUM+6

这两条伪指令汇编之后,NUM=14.

(5) 过程定义伪指令

过程定义伪指令也称为子程序定义伪指令.在程序中使用调用命令 CALL 调用它.过程定义伪指令的格式:

过程名 PROC [属性]

...

过程名 ENDP

说明:过程的属性可以是 NEAR 或 FAR,过程与调用命令在同一个代码段,过程的属性可以设置为 NEAR 类型;过程与调用命令不在同一个代码段,过程的属性应该设置为 FAR 类型.NEAR 为缺省属性.

(6) 程序结束伪指令

格式:END [标号]

END 为伪操作符,标号为程序开始执行的指令的符号地址.如果程序包含多个模块,只有主程序模块的结束伪指令 END 后可以加标号,其他程序模块的 END 后不能指定标号.

(7) 其他常见伪指令

① 程序开始伪指令 NAME

格式:NAME 模块名

功能:定义本程序模块的名字,告诉汇编程序 MASM:源程序从这开始

② 标题定义伪指令 TITLE

格式:TITLE 标题字符串

功能:打印源程序清单时,标题字符串作为每一页的标题.

③ ORG 伪指令

格式:ORG 表达式

功能:指定后面的指令或数据从表达式指出的地址(偏移地址)开始存放.

3. DOS 功能调用

(1) DOS 操作系统为用户提供的系统功能调用分类:

BIOS 功能调用,也叫低级调用,调用它们可以驱动磁盘,控制显示器输出,驱动打印机和管理时钟;

DOS 功能调用,也叫高级调用,调用它们可以管理内存,管理设备,管理文件和目录.

(2) 应用:

DOS 操作系统中 21H 号中断被称为 DOS 系统功能调用,它的内部提供了八十多个功能子程序,可以实现字符输入,字符显示和打印,磁盘读写,文件建立打开关闭,文件读写等功能.为了调用方便,系统对这些功能子程序进行编号,称为功能号.调用的步骤如下:

- ① 把要调用的功能号送 AH 寄存器
- ② 根据调用要求设置入口参数
- ③ INT 21H

(3) 具体操作简介

① 输入单个字符

从键盘输入单个字符可以使用 1,7,8 号功能.1 号功能接收键盘输入的字符并保存在 AL 中,同时显示在屏幕上.7,8 号功能接收键盘输入的字符保存在 AL 中但不显示.它们都不需要入口参数.

例如:

```
MOV  AH, 1
INT  21H
```

这两条指令执行后,光标在屏幕上闪动,等待键盘按键.一旦有键按下,其 ASCII 码存入 AL 中,字符显示在屏幕上.

② 输入字符串

0A 号系统调用:从键盘输入字符串存入指定的内存区域.

入口参数为 DS:DX

使用 0A 号功能之前首先要定义一个数据区,要求数据区的第一个字节含有允许输入的最大字符个数(包括回车符),第二个字节用于存放实际输入的字符个数,从第三个字节开始作为字符串存储空间.如果计划最多输入 10 个字符,数据区的定义方法如下:

```
BUFF DB 10,0,10 DUP(?)
```

调用 0A 号功能的方法如下:

```
MOV    DX, OFFSET BUFF
MOV    AH, 0AH
INT     21H
```

BUFF 数据区允许输入的最大字符个数为 10 个,若实际输入的字符个数(包括回车符)超过 10 个,则后面的字符由于没有存储空间而被丢弃,且喇叭会发出嘟嘟声报警,直到键入回车符.一般在定义数据区时,会比计划输入的字符数多一些.

例 2 从键盘上输入字符串 'WELCOME'

答:

```
DATA    SEGMENT
    BUFF    DB 10,0,10 DUP(?)    ;定义数据区
DATA    ENDS
CODE    SEGMENT
    ASSUME  CS: CODE, DS: DATA
START:  MOV    AX, DATA
        MOV    DS, AX
        MOV    DX, OFFSET BUFF
MOV     AH, 0AH    ;功能号送 AH
        INT     21H    ;功能调用
MOV     AH, 4CH
        INT     21H
CODE    ENDS
        END     START
```

③ 显示单个字符

2 号功能:显示单个字符.

入口参数:DL,将待显示字符的 ASCII 码送 DL 寄存器.

例如在屏幕上显示大写字母 B,可以用下面的 3 条指令实现:

```
MOV    DL, 'B'    ;待显示字符的 ASCII 码送 DL
MOV    AH, 2      ;功能号送 AH
INT    21H        ;功能调用
```

④ 显示字符串

9 号功能:将字符串显示在屏幕上.

入口参数:DS:DX ,字符串必须以'\$'结尾.

例 3 在屏幕上显示字符串.

答:

```

MOV    DX, OFFSET STRING ;设置入口参数
MOV    AH, 09H           ;功能号送 AH
INT     21H
DATA    SEGMENT
STRING  DB  'WELCOME TO JILIN UNIVERSITY', '$'
DATA    ENDS
CODE    SEGMENT
        ASSUME CS:CODE, DS:DATA
START:  MOV    AX , DATA
        MOV    DS, AX
        MOV    DX, OFFSET STRING ;设置入口参数
        MOV    AH, 09H           ;功能号送 AH
        INT     21H             ;功能调用
        MOV    AH, 4CH
        INT     21H
CODE     ENDS
        END START

```

⑤ 返回操作系统

4CH 号功能:使程序正常结束并返回操作系统,

调用方法如下:

```

MOV    AH, 4CH
INT     21H

```

4. 汇编语言程序设计

(1) 程序设计步骤

- ① 分析问题确定算法
- ② 画出程序流程图
- ③ 编写程序
- ④ 上机调试程序

(2) 顺序程序设计

例 4 编写计算 $S=A*B-C$ 的程序, A,B,C 是无符号字节变量, S 是字变量.

答:

```

DATA SEGMENT           ;定义数据段 "DATA"
    A DB 38             ;定义数据,字节类型变量,不超过 255
    B DB 54
    C DB 16
    S DW ?              ;? 给 S 变量留出一个字的存储空间
DATA ENDS               ;数据段设定完毕
CODE SEGMENT            ;定义代码段 CODE
    ASSUME CS:CODE, DS:DATA ;说明 CODE 是代码段,DAT
A 是数据段
START: MOV AX, DATA
      MOV DS, AX
      MOV AL, A
      MOV BL, B
      MUL BL             ;A*B,结果存在 AX 中
      MOV BL, C
      MOV BH, 0
      SUB AX, BX         ;AX-C,结果在 AX 中
      MOV S, AX          ;保存计算结果
      MOV AH, 4CH
      INT 21H            ;返回操作系统
CODE ENDS               ;代码段设定完毕
END START

```

例 5 在内存中从 TABLE 单元开始的连续 16 个单元中,存放着 0~15 的平方值(平方表),查表求任意数 $X(0 \leq X \leq 15)$ 的平方值,将结果保存在 RESULT 中.

TABLE →		
	1H	DS:0H
	4H	DS:1H
	9H	DS:2H
	10H	DS:3H
	19H	DS:4H
	24H	DS:5H
	31H	DS:6H
	40H	DS:7H
	51H	DS:8H
	64H	DS:9H
	79H	DS:AH
	90H	DS:BH
	A9H	DS:CH
	C4H	DS:DH
	E1H	DS:EH

图4-7变量的内存分配

答:

```
DATA        SEGMENT
```

```
TABLE  DB  0,1,4,9,16,25,36, 49,64,81
```

```
          DB 100,121,144,169,196,225
```

```
          X  DB 11
```

```
RESULT  DB  ?      ;定义结果存放单元
```

```
DATA        ENDS
```

```
STACK     SEGMENT      'STACK'
```

```
          DW 100H DUP(?) ;定义堆栈空间:256 字节空间
```

```
STACK     ENDS          ;定义堆栈段
```

```
CODE      SEGMENT
```

```
          ASSUME CS:CODE, DS:DATA, SS:STACK
```

```
START:  MOV AX, DATA    ;初始化数据段
```

```
          MOV DS,AX
```

```
          LEA BX, TABLE ;设置平方表的基地址
```

```
          MOV AH,0
```

```
          MOV AL, X      ;取待查数
```

```
          ADD BX, AX     ;计算在表中具体地址
```

```
          MOV AL, [BX]
```

```
          MOV RESULT, AL ;X 的平方数存入 RESULT
```

```
          MOV AH, 4CH    ;返回操作系统
```

```

    INT 21H
CODE   ENDS
    END   START

```

例5类似题 在内存中从TABLE单元开始的连续16个单元中,存放着0~15的平方值(平方表),查表求任意数X($0 \leq X \leq 15$)的平方值,将结果保存在RESULT中.要求程序中使用XLAT指令实现查表转换.

答:

```

DATA      SEGMENT
TABLE  DB 0,1,4,9,16,25,36, 49,64,81
        DB 100,121,144,169,196,225
        X DB 11
RESULT DB ?      ;定义结果存放单元
DATA      ENDS

STACK     SEGMENT     'STACK'
    DW 100 DUP(?)    ;定义堆栈空间:256 字节空间
STACK     ENDS        ;定义堆栈段

```

```

CODE      SEGMENT
    ASSUME CS:CODE, DS:DATA, SS:STACK
START:    MOV  AX, DATA    ;初始化数据段
          MOV  DS,AX
          LEA  BX, TABLE  ;设置平方表的基地址
          MOV  AL, X        ;取待查数
          XLAT              ;AL中存有要转换的数据
          MOV  RESULT, AL   ;X的平方数存入RESULT
          MOV  AH, 4CH       ;返回操作系统
          INT  21H
CODE      ENDS
    END   START

```

(3) 分支程序设计

分支程序结构分为简单分支结构和多分支结构两种形式

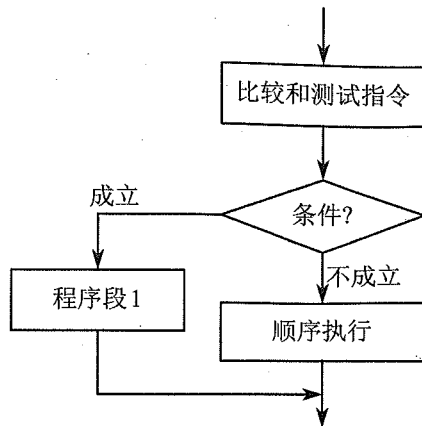


图 简单分支程序结构

例 6 数据段中 DATA 单元和 DATA+1 单元各存有一个无符号数, 比较两数的大小, 大的存入 DATA 单元, 小的存入 DATA+1 单元.

答:

```

MOV  AL, DATA
CMP   AL, DATA+1      ;比较,AL,DATA 不变,标志位
                        变化
JNC   CHANGE           ;若 CF=0,即 DATA≥DATA+1,转移到 C
HANGE
MOV   BL, DATA+1      ;条件不成立顺序执行
MOV   DATA, BL        ;交换
MOV   DATA+1,AL
CHANGE:...
  
```

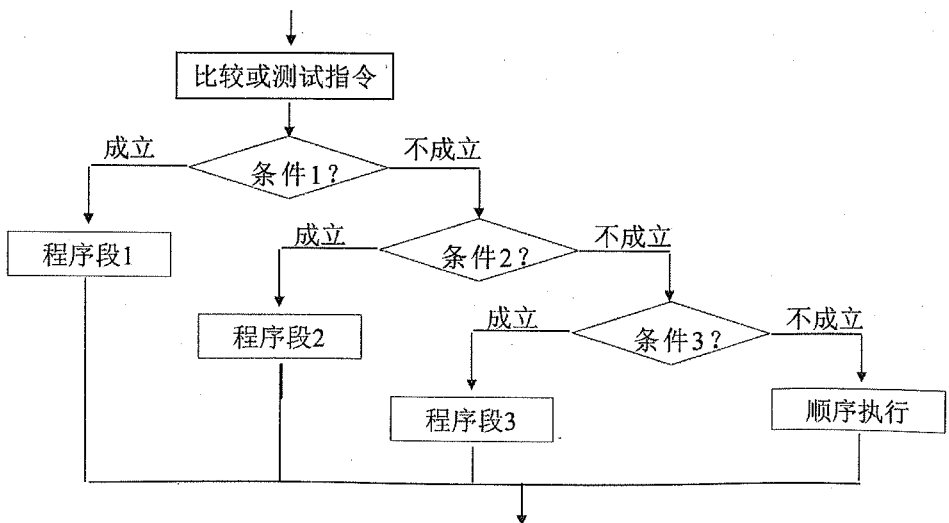


图4.9 多分支程序结构

例 7 在提示信息 ‘PLEASE INPUT CHARACTER:’后从键盘输入字符,如果输入的是 ESC 键,则结束程序;如果输入的是小写字母则显示;如果是大写字母,则转换为小写字母显示.

答:

分析:数据段包含提示信息,其余程序中用键盘随机输入.

程序:

DATA SEGMENT

MESSAGE DB 'Please input character:'0DH,0AH, '\$'

;提示字符串加单引号自动转换为 ASCII 码;

;0DH:回车键控制符,0AH:换行控制符,\$是显示字符

DATA ENDS

STACK SEGMENT 'STACK'

DW100H DUP(?)

STACK ENDS

CODESEGMENT

ASSUME CS:CODE, DS: DATA, SS: STACK

START: MOV AX, DATA

MOV DS, AX ;给 DS 赋初值

MAS:

MOV DX, OFFSET MESSAGE

MOV AH, 9

INT21H ;DOS 功能调用,显示字符串

AGAIN:

MOV AH, 1

INT 21H ;按键,按下键的 ASCII 码存储在 AL 中

CMP AL,1BH ;1B:ESC

JZ EXIT ;是 ESC, 转移到 EXIT

CMP AL,41H

JC MAS

CMP AL, 5BH ;大写字母 41H~5AH

JC LOW0 ;是大写字母,转移到 LOW0

CMP AL,61H

JC MAS

```

    CMP    AL, 7BH
    JC     LOW1      ;是小写字母,转移到 LOW1
    JMP     MAS
LOW0:  ADD     AL, 20H
;加上 20H 转换为小写字母的 ASCII,继续进行 LOW1
LOW1:  MOV     DL, AL
    MOV     AH, 2
    INT     21H      ;调用功能,显示 DL 中单个字符
    JMP     AGAIN
EXIT: MOV     AH, 4CH
    INT     21H
    CODE    ENDS
    END     START

```

(4) 循环程序设计

循环程序结构由循环初始化,循环体和循环控制三部分组成.循环程序结构有两种:先执行后判断和先判断后执行.

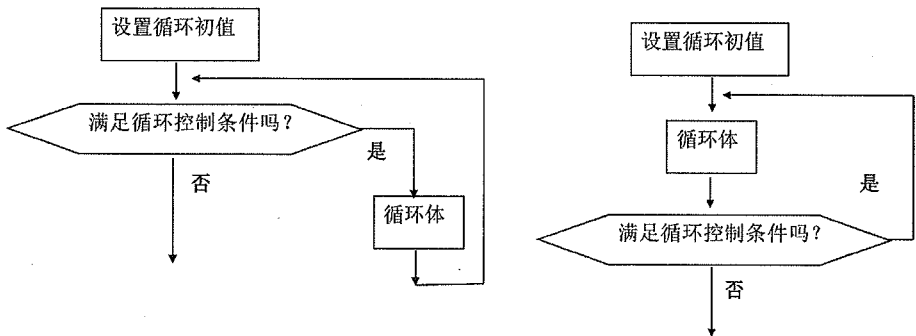


图4-10 循环控制程序结构

例 8 在数据段中从 **BUFF** 单元开始存放 **100** 个字节类型的无符号数,编写程序找出其中最大的数并存入 **MAX** 单元中.

答:

方法:第一个数与第二个数比较,第一个数大不变,第一个数小交换;在将结果得到的数与第三个数比较,以此类推.

```
DATA SEGMENT
```

```
BUFF DB 100H DUP(?) ;定义 100 个数据
```

```
MAX DB ?
```

```
DATA ENDS
```

```
CODE SEGMENT
```

```

ASSUME CS:CODE, DS:DATA
START: MOV AX, DATA
        MOV DS, AX
        MOV CX, 99          ;设置循环次数
        LEA SI, BUFF        ;数据首地址送 SI
        MOV AL, [SI]        ;取第一个数
        INC SI
CON:    CMP AL, [SI]         ;与第二个数比较大小
        JNC NEXT            ;若 AL≥[SI],则跳转 NEXT
        ;处
        MOV AL, [SI]        ;AL< [SI], 替换 AL
NEXT:   INC SI               ;修改地址指针
        LOOP CON             ;测试循环条件 CX=0?

        MOV MAX, AL
        MOV AH, 4CH

        INT 21H
CODE    ENDS
        END START

```

例 9 在数据段中从 BUFF 单元开始存放 100 个字节类型的无符号数, 将它们按从大到小的顺序排序。

答:

解析:排序:

排序有多种算法,这里使用起泡法.从第一个数开始依次对相邻的两个数进行比较,100 个数需要比较 99 次,所以程序需要设计一个 99 次的循环.在这个循环里每次比较时如果前边的数小于后面的数,这两个数交换位置.这 99 次的循环结束后,最小的数已经交换到了最后,还剩 99 个数要用同样的比较方法找到最小的数并放到最后,这只要再设计一个 98 次的循环就可以了.以此类推这个过程需要 99 轮。

用起泡算法排序需要设计两重循环,内循环完成数的比较和交换,初始内循环计数值为 N-1 次,之后每次进入内循环计数值减 1;外循环需要 N-1 次,从外循环进入内循环时注意地址指针初始化和内循环次数的设置。

下面的程序在内循环中设置了交换标志,从外循环进入内循环时检查

交换标志,如果标志不为 0 说明前一个内循环里至少有两数据的顺序不合要求,需要再执行一次内循环;如果标志为 0,说明所有数据的排序结束.

```

LEN    EQU    100
DATA   SEGMENT
    BUFF          DB 100 dup(?)      ;定义 100 个数据
    CHANGE DB 0      ;设置交换标志
DATAENDS
CODE   SEGMENT
    ASSUME CS:CODE, DS:DATA
START: MOV  AX, DATA
        MOV  DS, AX
    LEA  BX, BUFF      ;BX 作数据的地址指针
    LEA  DI, CHANGE;DI 作交换标志
    MOV  DX, LEN-1     ;DX 保存循环次数
SORT:MOV  SI, BX      ;内循环初始化,设置地址指针
    MOV  CX, DX        ;设置计数值,等于参加比较的数据数量
        MOV  BYTE PTR[DI], 0      ;初始化交换标志
GOON:  MOV  AL, [SI]      ;内循环开始
        INCSI
        CMP  AL, [SI]          ;前一个数和后一个数比较
        JNC  NEXT              ;前大后小, 转
NEXT  不交换
        MOV  BYTE PTR[DI], 1      ;前小后大, 置交换标志
    MOV  AH, [SI]
        MOV  [SI], AL      ;交换
        MOV  [SI-1], AH
NEXT:  LOOP GOON      ;内循环结束
    DEC  DX
    JZ   NEXT1      ;外循环计数值为 0,程序结束
    CMP  BYTE PTR[DI], 0      ;如果内循环中没有交换,程序
    结束
        JNZ  SORT          ;开始新一轮内循环
NEXT1: MOV  AH, 4CH

```

INT21H

CODE ENDS

END START

(5) 过程设计

过程又称为子程序.CALL 指令连接到过程,RET 指令从过程中返回.

过程有两种属性:NEAR,FAR

由整个软件使用的过程应该写为远过程 FAR (global,全局),由指定任务使用的过程应该写为近过程 NEAR.近过程必须和调用程序在同一个段,远过程可以随意.

① 过程定义

过程定义就是子程序定义,由伪指令完成.

② 过程调用和返回

过程调用通过 CALL 指令实现, CALL 指令执行时将当前 IP 或 CS 和 IP 压入栈堆中,然后将子程序的首地址赋给 IP 或 CS 和 IP,CPU 开始执行子程序.

RET 指令执行时弹出栈中的数据,修改 IP 或 CS 和 IP 的内容,从而实现返回调用程序的目的.

③ 保护与恢复现场

保护和恢复现场常使用 PUSH 和 POP 指令.

④ 参数传送

主程序在调用子程序时,要为子程序预置数据,在子程序返回时给出数据处理的结果,这称为数据传送或变量传送.方法主要有以下几种:

1° 寄存器传送.

2° 地址表传送

3° 堆栈传

⑤ 子程序嵌套与递归子程序

子程序可以调用其它子程序,称为子程序嵌套或过程嵌套.子程序也可以调用自己,称为递归调用.

例 10 计算 $S = 1^2 + 2^2 + \dots + N^2$

答:

;第一部分:调用平方和过程求 20 个数的平方和;

;CX 是入口参数,调用之前应该预置 CX=N;

;DX 为出口参数,N 个数的平方和存在 DX 中.

```

DATA SEGMENT 'DATA'
CON EQU 20
S DW ?
DATA ENDS
CODE SEGMENT 'CODE'
ASSUME CS:CODE, DS:DATA
START: MOV AX, DATA
MOV DS, AX
MOV CX, CON ;设置过程的入口参数 CX=20
CALL GO ;调用过程
MOV S, DX ;保存出口参数
MOV AX, 4C00H
INT 21H
;第二部分:平方和子程序
GO PROC NEAR ;近过程定义,起名 GO,近过程与主程序在一个段里
MOV DX, 0
MOV BL, 1 ;BL 表示自然数
MOV AL, BL
CC: MUL BL ;AL*BL 结果存在 AX 中
ADD DX, AX ;当 N≤50 时,不会产生进位
INC BL ;BL+1
MOV AL, BL ;AL=BL
LOOP CC ;CX 为计数器,CX-1=0 进行下一步
RET ;过程返回
GO ENDP ;过程定义结束 GO ENDP ;过程定义结束
CODE ENDS
END START

```

例 11 求 $n!$ (设 $n < 10$) .当 $n = 0$ 时 $n! = 1$; 当 $n > 0$ 时 $n! = n(n-1)$

答:

```

DATA SEGMENT
    N DW 9 ;自然数
    FNUM DW ?,? ;存结果
DATA ENDS

```

```

CODE SEGMENT
    ASSUME CS:CODE, DS:DATA
MAIN PROC FAR
START: MOV  AX, DATA
        MOV  DS, AX
        PUSH CX
        MOV  AX, N
        MOV  DX, 0      ; DX 清 0, 准备存放结果的高 16
        CALL DG          ; 调用子程序
        LEA  SI, FNUM
        MOV  [SI], AX     ; 存结果的低 16 位
        MOV  [SI+2], DX; 存结果的高 16 位
        POP  CX
        RET              ; 程序结束
MAIN      ENDP
DG PROC NEAR          ; 子程序定义
    PUSH  AX ; 压入 9, 利用递归依次压入 987654321
    SUB   AX, 1
    JNZCON
    POP   AX
    JMP   TURN
CON: CALL DG
    POP   CX ; 利用递归返回依次弹出 123456789
    MUL   CX ; n!
    TURN: RET
DG ENDP
    CODE ENDS
    END    START

```

例 12 编写一个多字节减法子程序

答:

;子程序名:CALSUB

;子程序功能:多字节二进制数减法

;入口参数: CX 为数的长度 (按字计算), DS:DI 为第一个数的首地址,
DS:SI 为第二个数的首地址; DS:BX 为结果的首地址.

;出口参数:无.

```

CALSUB PROC NEAR
    PUSH AX ;保护主程序现场 AX, FLAGS
    PUSHF
    CLC ;清 0 进位标志 CF
    CAL1: MOV AL, [DI] ;取被减数
    SBB AL, [SI] ;减法
    MOV [BX], AL ;存结果
    INC SI ;调整指针
    INC DI
    INC BX
    LOOP CAL1 ;处理高位字
    POPF ;恢复主程序现场 FLAGS, AX
    POP AX
    RET
CALSUB ENDP

```

例 13 编程实现:统计寄存器 **BX** 中二进制位“1”的个数,结果存在 **A** 中.

答:

```

DATA SEGMENT
ABC DW ?
DATA ENDS
STACK SEGMENT 'STACK'
DW 100 DUP(?)
STACK ENDS
CODE SEGMENT
ASSUME CS:CODE,DS:DATA,SS:STACK
START:
MOV AX,DATA
MOV DS,AX
MOV BX,ABC
MOV AL,0
MOV CX,1
L1: SHR BX,1
JC L2
JMP L

```

```
L2:  INC AL
L3:  LOOP L1
EXIT: MOV AH,4CH
      INT 21H
CODE ENDS
      END START
```

(6) 数制转换问题

① 从二进制转换为 ASCII 码

有两种方法完成从二进制数到 ASCII 码得转换:

- 1° 如果数据小于 100,则用 AAM 指令实现;
- 2° 通过一系列的十进制除法(除以 10)实现.

方法 1:

AAM 指令将 AX 中的数值转换为两位非压缩 BCD 码,存入 AX 中
各指令用法:

AAA (加法后调整为 ASCII)

AAD (除法后调整为 ASCII)

AAM (乘法后调整为 ASCII)

AAS (减法后调整为 ASCII)

注意,这些指令操作完后,调整二进制为非压缩 BCD 码,不是 ASCII 码,需加 3030H 转换为 ASCII 码

方法 2:

- 1° 除以 10,然后将余数存入堆栈作为 BCD 数的有效位
- 2° 重复第 1 步,直到商为 0
- 3° 取出每一位余数,并加上 30H 将其转换成 ASCII 码

② 从 ASCII 码转换为二进制

ASCII 转换为二进制通常从键盘输入开始,如果输入单键,则从该数中减去 30H 实现转换;如果键入的多于一个键,则转换仍然从键值中减去 30H,并将结果先乘 10,再加上刚才所得数字.

算法如下:1)将二进制结果单元清零 2)从键盘键入的字符中减去 30H,把它转换为 BCD 码;3)结果乘以 10,再加上新的 BCD 数字;4)重复第二步和第三步,直到键入的数字不是 ASCII 编码的数字为止.

例 14 下面程序的功能是:把 BUFF 中的二进制数转换为 ASCII 码,依次存入 RESULT 开始的存储单元中并显示在屏幕上.例如,将 2AB9 H 转换为 ASCII 码并存入 RESULT 中,RESULT 开始的 4 个单元的

内容依次为 32H,41H,42H,39H.请将程序补充完整.

DATA SEGMENT

BUFF DW 2AB9H

RESULT DB 4 DUP(0)

DATA ENDS

CODE SEGMENT

ASSUME CS:CODE, DS:DATA

START: MOV AX, DATA

MOV DS, AX

MOV AX, BUFF ;取 BUFF 中的数据

MOV CX, 4

LEA BX, RESULT

MOV SI, 3 ;SI 指向 RESULT+3

A1: MOV DX, AX ;备份数据

;提取 AX 的低 4 位

;与 0AH 比较

;大于等于 0AH 则转移到 A2

;在 0~9 之间就加 30H

;跳转到 A3

;A2,在 A~F 之间就加 37H

;A3,保存 ASCII 值

;调整指针 SI

;取出备份的数据

SHR AX, 4 ;右移 4 位,准备转换下一位 ASCII 数据

LOOP A1 ;4 位 ASCII 都转换完了吗? 没有则返回 A1 继续处理

MOV CX, 4 ;显示 4 位 ASCII 数据

MOV SI, 0

A4: MOV DL, [BX+SI]

INC SI

MOV AH, 2

INT 21H

LOOP A4

MOV AH, 4CH ;返回 DOS

INT 21H

CODE ENDS

END START

答:

AND AX, 0FH ;AND AL,0FH

CMP AL, 0AH

JNC A2 ; JA A2

ADD AL, 30H

JMP A3

A2: ADD AL, 37H

A3: MOV [BX+SI] , AL

DEC SI

MOV AX, DX

5. 汇编语言程序开发过程

(1) 建立汇编语言源程序

(2) 生成目标程序

(3) 生成可执行文件

连接程序 LINK.EXE 把 MASM 产生的目标文件 (OBJ), 与其他目标文件及系统提供的一些库文件连接在一起, 生成以 EXE 为扩展名的可执行文件.

(4) 调试与运行程序

MYFILE EXE 文件可以直接执行, 但程序中的逻辑错误需要经过调试才能剔除. 调试阶段常用的调试工具为 DEBUG.

第五章 存储器

一、本章考点

1. 小题:本章出小题概率较小,重点掌握大题部分.

例:

- (1) 存储器件上/OE、/WE 引脚及/CS 或/CE 引脚的用途是什么?
- (2) DRAM 是哪种器件的缩写?
- (3) 2716EPROM 可容纳多少字节数?
- (4) 为什么存储器地址译码电路很重要?

答:

- (1) 片选标识为/CS,片允许 (/CE) ,是存储器件的芯片选择线,用来选中这个器件.

输入输出控制线:/OE、/WE

/OE 低电平,存储器件输出数据;/WE 低电平,存储器件输入数据.同为低电平无效态,同为高电平高阻态.

- (2) 动态随机存储器
- (3) 2K
- (4) 通常,存储器件与微处理器的地址码不一样,例如,

8088 CPU: A0—A19

2716 EPROM: A0—A10

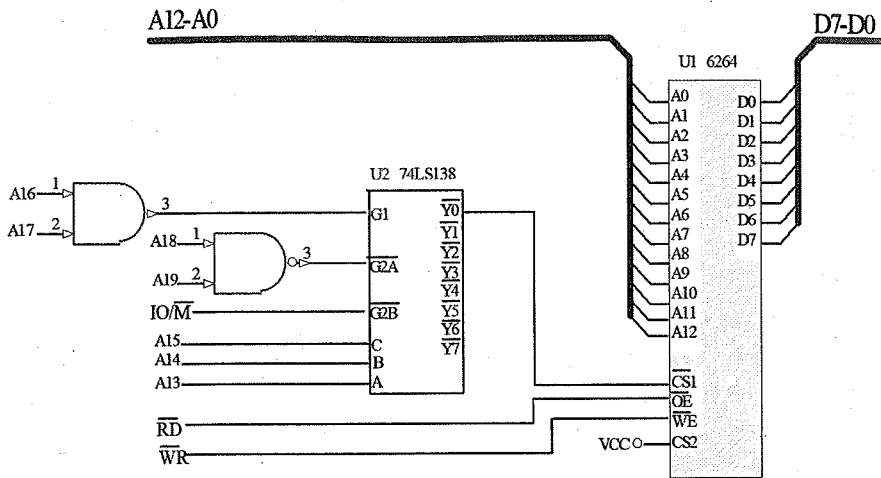
当存储器件与微处理器相连,有必要对微处理器发送来的地址进行译码.

2. 大题

一般必考一道大题,存储器应用

题型:译码电路地址范围分析 (一般来说,考试时存储芯片接静态 RAM 6264/6116/62256 或只读存储器 ROM 2716/2764,常用译码器 74LS138 进行译码)

例1:试写出接 Y0~Y7 地址范围



结果:

Y0:F0000~F1FFFFH

Y1:F2000~F3FFFFH

Y2:F4000~F5FFFFH

Y3:F6000~F7FFFFH

Y4:F8000~F9FFFFH

Y5:FA000~FBFFFFH

Y6:FC000~FDFFFFH

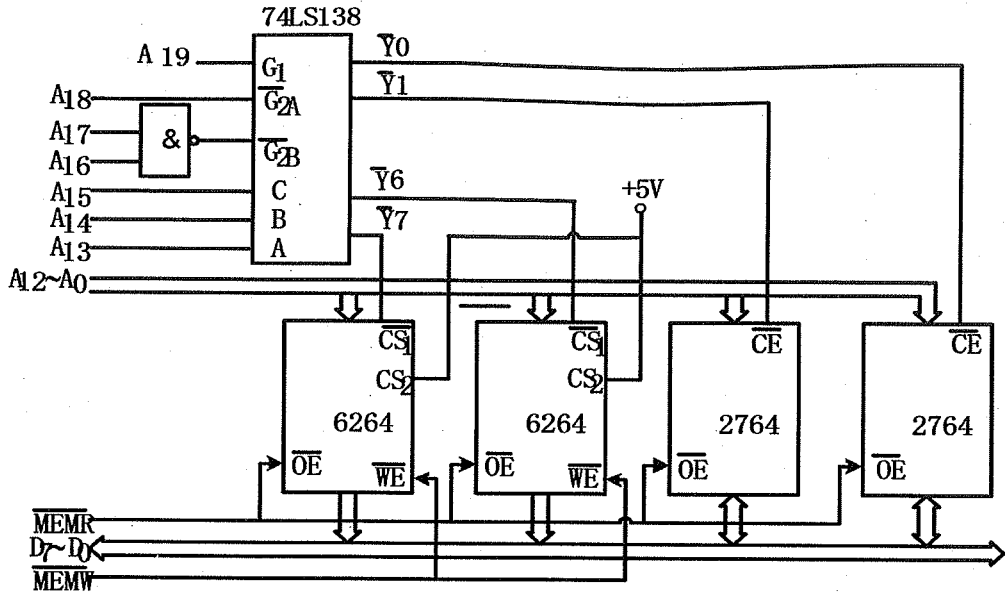
Y7:FE000~FFFFFFH

分析:

只分析接/Y0 一种情况.

IO/(M)=0,所以接存储器;A19,A18 接与非门接/G2A,所以 A19=1,A18=1;A17,A16 接与门接 G1,所以 A17=1.A16=1;如果选中 Y0,/Y0=/CS1=0,则 A13~A15=000; A12~A0=0 0000 0000 0000~1 1111 1111 1111,所以接 Y0 存储器地址范围为 1111 0000 0000 0000 0000~1111 0001 1111 1111 1111,及 F0000~F1FFFFH.

例 2:若用存储器芯片 SRAM6264 (8Kx8) 和 EPROM2764 (8Kx8) 构成 32Kx8bit 位的存储器系统 (如图 1 所示),请根据图示回答各个存储芯片的地址范围是多少?



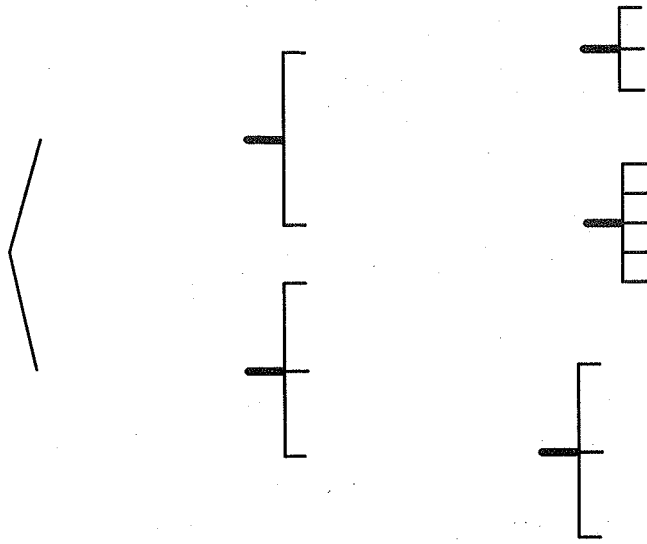
答案:

B0000H~B1FFFFH,
B2000H~B3FFFFH,
BC000H~BDFFFFH,
BE000H~BFFFFFFH

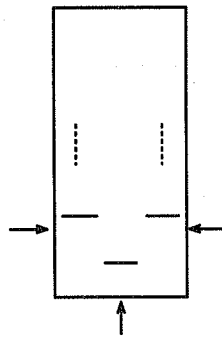
二、基础知识点

1. 存储器概述

(1) 存储器:内存储器与外存储器



(2) 存储器件



① 地址线: $A_0 \sim A_n$

所有的存储器件都有地址输入引脚(地址线),用来接收 CPU 发过来的存储单元地址信息.地址线的根数与芯片内含有的存储单元个数有一个对应的关系.

例:

含有 10 根地址线的芯片,地址线标记为 $A_0 \sim A_9$,存储单元应该有 $2^{10} = 1024$ 个存储单元;

一个存储容量为 4KB 的存储芯片应该有 12 根地址线 $A_0 \sim A_{11}$,一个含有 1MB 的存储芯片应该有 20 根地址线 $A_0 \sim A_{19}$.

存储单元的地址范围:

如果存储芯片的地址线为 $A_0 \sim A_9$,它的存储单元的地址范围为:

00 0000 0000~11 1111 1111B,即 000H~3FFH.

地址线为 $A_0 \sim A_{10}$ 根的芯片,地址范围为:

000 0000 0000~111 1111 1111B,即 000H~7FFH.

通常用 3FFH 表示 1KB 的存储区域,用 7FFH 表示 2KB 的存储区域,用 FFFH 表示 4KB 的存储区域,用 FFFFH 表示 64KB 的存储区域,用 FFFFFH 表示 1MB 的存储区域.

② 数据线: $O_0 \sim O_n$

数据线负责数据的输出,或者是数据的输入/输出.数据线的根数说明芯片内一个存储单元所能存放的二进制数的位数,芯片含有 8 根数据线 $D_0 \sim D_7$,说明它的一个存储单元能存放 8 位二进制数.

位宽:数据线的根数常称为芯片的位宽,8 位宽的存储芯片常常称为字节宽存储器件.多数存储芯片是 8 位宽,也有 16 位宽的芯片,还有 4 位宽,1 位宽的存储芯片.

③ 芯片选择线:/CS

所有存储器件都有芯片选择线,而且至少一根,用来选中这个器件.片选标识为/Cs,片允许 (/CE),选择输入(/S).有效状态 0,无效状态 1.

如果存储芯片含有多根选择线,则只有在所有选择线都处于有效状态时芯片有效.

④ 输入输出控制线:/OE,/WE

/OE 低电平,存储器件输出数据;/WE 低电平,存储器件输入数据.同为低电平无效态,同为高电平高阻态.

ROM 存储器件因为只能读出数据不能写入数据,所以通常只有一个输出使能(/OE)引脚,有些芯片将其标为/G (输出选通).

有些 RAM 存储器件受到芯片引脚数量的限制,控制数据输入输出的引脚只有一个,常常标记为 R/ (/W).当 R/ (/W) 引脚为高电平时芯片输出数据,执行读操作;当它为低电平时芯片接收数据,执行写操作.

2. 静态 RAM 器件

静态随机存储器以双稳态触发器为基本存储电路,存储速度快,保存数据不需要刷新,但集成度低,功耗大.

一般功能结构:存储体+外围功能辅助电路.

存储器有许多的存储元组成,存储元以矩阵形式排列,采用行列单独译码的译码方式.I/O 缓冲器从系统数据总线接收数据或把存储单元存储的数据输出到数据总线上.

例:

6116:2K*8 每个芯片有2K个存储单元,每个存储单元可以存储8位信息. 存储单元个数为 $2 \times 2^{10} = 2^{11}$ 个,所以地址线有 $A_0 \sim A_{10}$ 共11根.

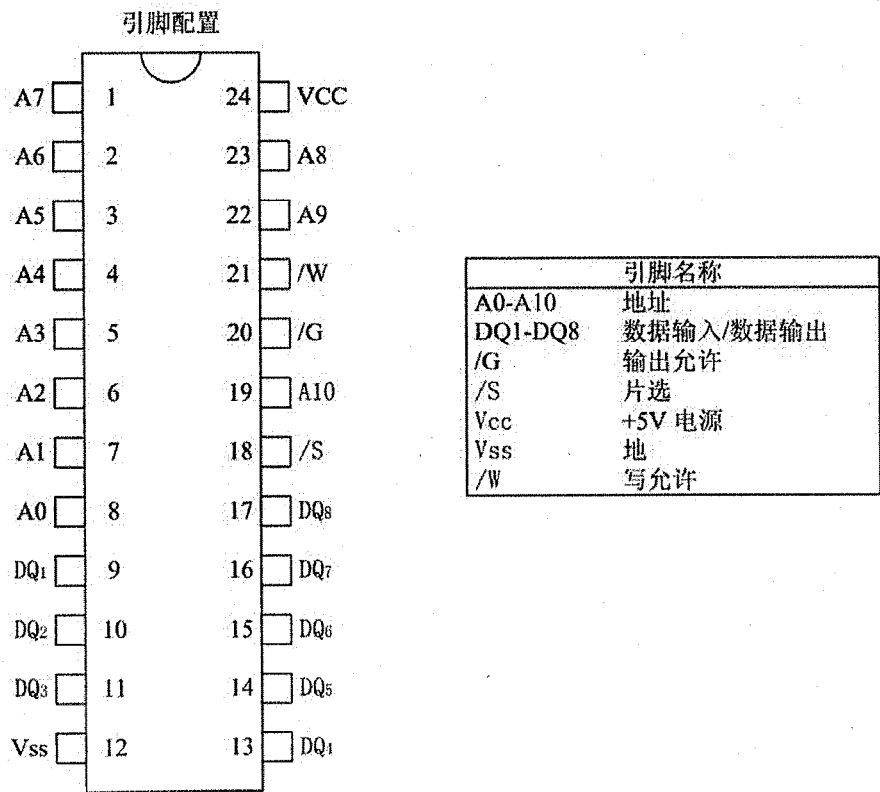
62256:32K*8 同理,每个芯片有2K个存储单元,每个存储单元可以存储8位信息.存储单元个数为 $32 \times 2^{10} = 2^{15}$ 个,所以地址线有 $A_0 \sim A_{14}$ 共15根.

工作过程:以 6116 为例

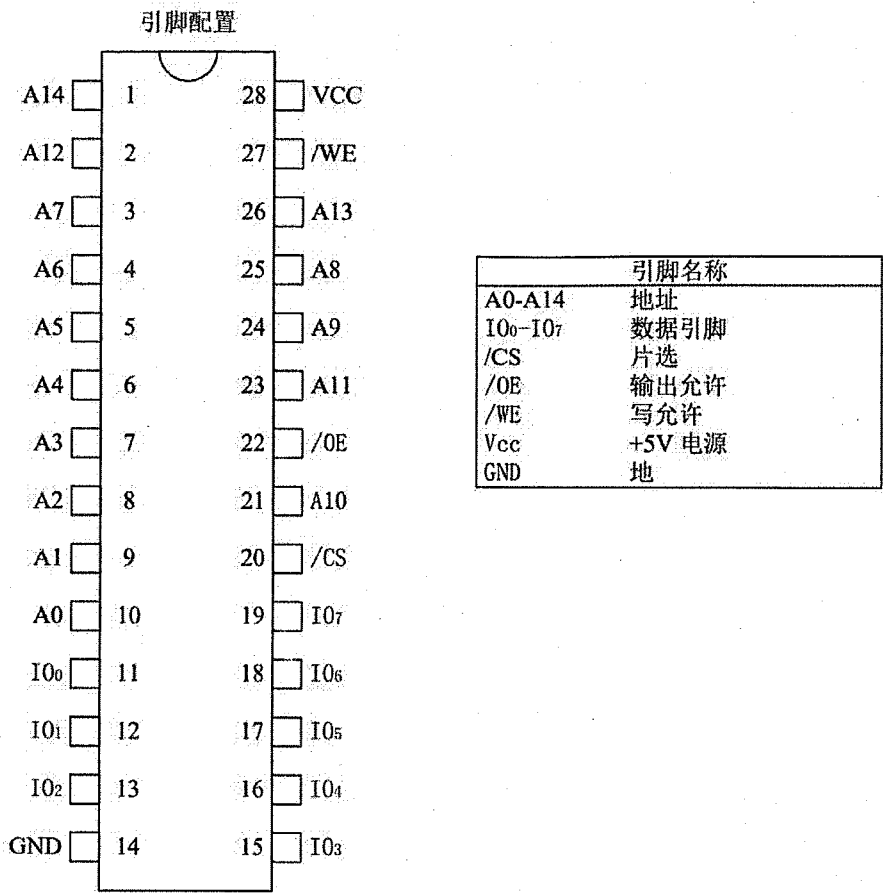
写数据:

CPU 把 20 位存储单元物理地址送到地址线上, $A_0 \sim A_{10}$ 送到芯片地址线 $A_0 \sim A_{10}$ 上, $A_{11} \sim A_{19}$ 译码后输送给片选/S 令其为 0;接着 CPU 送出写信号/ $\overline{WR}=0$,并将数据发送至数据线 $D_0 \sim D_7$ 上, $\overline{WR}$ 使 $\overline{W}$ 有效,从而信息写入芯片指定的存储单元.

读数据与写数据过程类似,只是 CPU 送出 $\overline{RD}=0$, $\overline{RD}$ 使 $\overline{G}$ 有效.



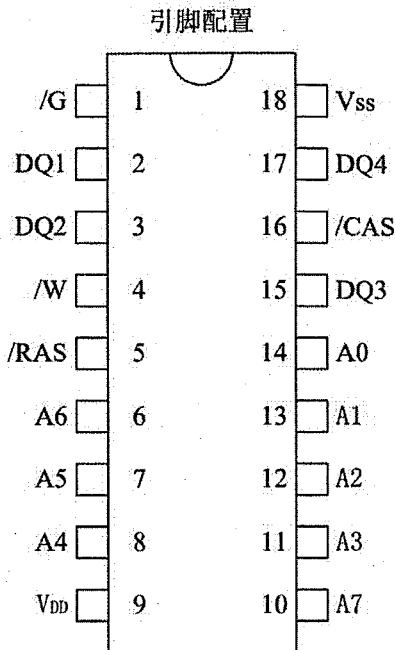
6116, 2K * 8 static RAM (SRAM)引脚图



62256, 32K * 8 static RAM (SRAM)引脚图

3. 动态随机存储器 (DRAM)

它在一个集成电容上仅将数据保留 2ms 或 4ms 的时间,2ms 或 4ms 之后 D RAM 中的内容必须全部刷新.在 DRAM 中,通过 2ms 或 4ms 的间隔时间内读 256 次,存储器的整个内容被刷新.



TMS4464, 64K * 4 动态 RAM (DRAM) 引脚图

TMS4464

64k*4: 存储 256K 位数据, 8 个地址引脚:

首先, A0—A7 被置于地址线上, 由 /RAS 选通进入一个内部的行锁存器作为行地址;

然后, A8—A15 被置于同样的 8 条地址线上, 由 /CAS 选通进入一个内部的列锁存器作为列地址;

保持在这些内部锁存器中的 16 位地址寻址 4 位存储单元中的内容。

4. 只读存储器 ROM

ROM 是非易失性存储器, 分为 EPROM (可擦除可编程只读存储器), EEPROM (电可擦除可编程 ROM)。

例:

2716(2K*8 EPROM)

2764(8K*8 EPROM)

工作过程 (以 2764 为例):

读出方式: 读出 2764 内容, 与 RAM 类似

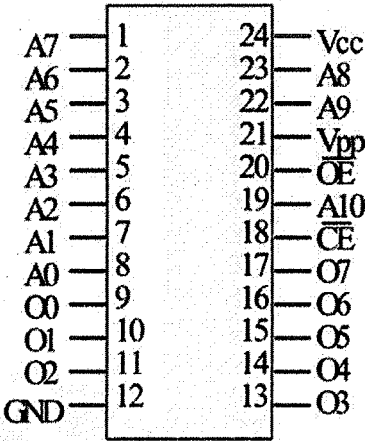
编程写入:

标准编程: $V_{CC}=5V$, V_{PP} 高电平, 地址线上给出需要编程的存储单元地址, /C

E=0,/OE=1,数据线上写入数据,PEM 接 50ms 高脉冲.每 50ms 写入一个字节.

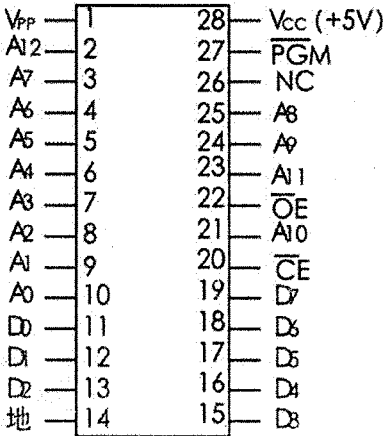
快速编程:100 微秒写入一个字节.

2716



引脚功能	
A0~A10	地址线
O0~O7	数据线
$\overline{CE}$	片选
$\overline{OE}$	输出允许
Vpp	编程电压输入
Vcc	+5V电源
GND	地

2764



引脚功能	
A0~A12	地址线
D0~D7	数据线
$\overline{CE}$	片选
$\overline{OE}$	输出允许
Vpp	编程电压输入
PGM	编程脉冲输入
Vcc	+5V电源
GND	地

5. 高速缓冲存储器 Cache

为了缓解 CPU 和主存之间存取速度的矛盾,在 CPU 和主存之间插入一小块 SRAM,称为 Cache.CPU 在读取指令或数据时总是先在 cache 中寻找,若找到则读入,若找不到再到主存中寻找.

Cache 的工作原理:

如果 CPU 在读取指令或数据时在 cache 中没有找到(未命中),而是到主存中寻找.系统就将当前的指令及相关联的后继指令集从内存读到 Cache,

使 CPU 执行下一条指令时,从 Cache 中读取(命中)。

把一段时间内一定范围里的信息成批地从主存读到 cache 中,使 CPU 到 cache 中读取后继的指令和数据,减少 CPU 访问主存的次数,从而缓解 CPU 和主存之间存取速度的矛盾。

6. 地址译码

通常,存储器件与微处理器的地址码不一样,例如,

8088 CPU: A0—A19

2716 EPROM: A0—A10

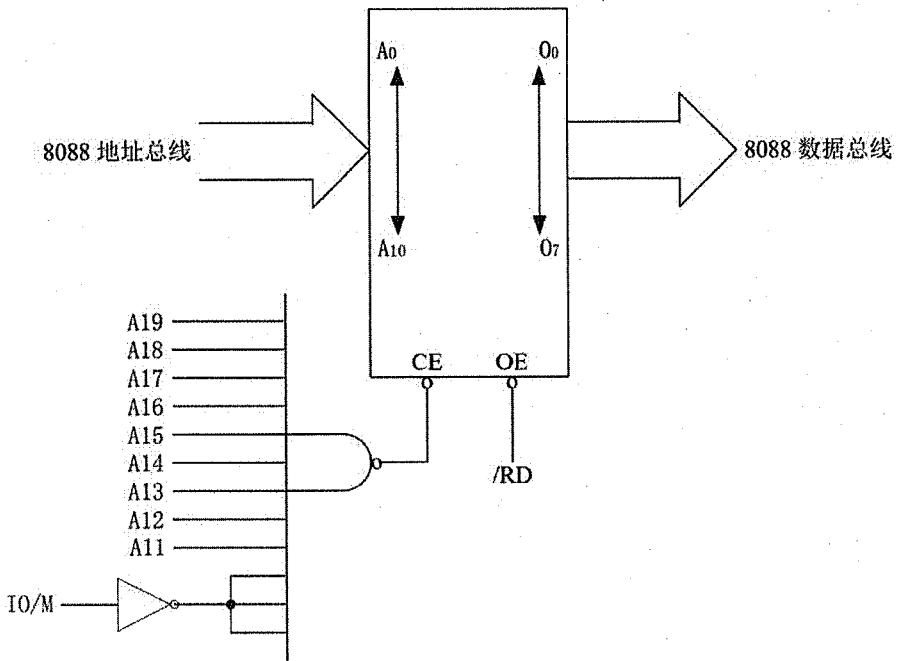
当存储器件与微处理器相连,有必要对微处理器发送来的地址进行译码。

译码方式:

全地址译码:全部地址线与芯片连接;

部分地址译码:系统部分地址线与芯片连接。

(1) 简单的与非门译码电路



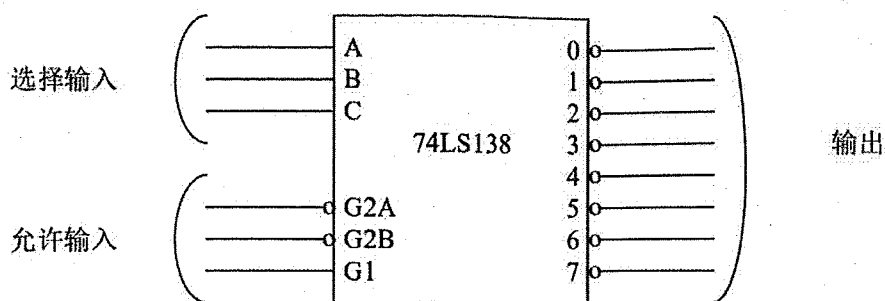
图中 CE 改为 $\overline{\text{CE}}$, OE 改为 $\overline{\text{OE}}$, IO/ $\overline{\text{M}}$

译码的地址范围是:

从 1111 1111 1000 0000 0000 = FF800H 到 1111 1111 1111 1111 1111 = FFFFFFFH

分析: $\overline{\text{CE}}=0$ 选中,所以 IO/ $\overline{\text{M}}=1$, A19~A11=1111 1111 1, A10~A0 用来选中存储器件。

(2) 3—8 译码器 (74LS138)



74LS138 3-到-8 译码器

输入						输出							
允许			选择										
/G2A	/G2B	G1	C	B	A	/0	/1	/2	/3	/4	/5	/6	/7
1	X	X	X	X	X	1	1	1	1	1	1	1	1
X	1	X	X	X	X	1	1	1	1	1	1	1	1
X	X	0	X	X	X	1	1	1	1	1	1	1	1
0	0	1	0	0	0	0	1	1	1	1	1	1	1
0	0	1	0	0	1	1	0	1	1	1	1	1	1
0	0	1	0	1	0	1	1	0	1	1	1	1	1
0	0	1	0	1	1	1	1	1	0	1	1	1	1
0	0	1	1	0	0	1	1	1	1	0	1	1	1
0	0	1	1	0	1	1	1	1	1	1	0	1	1
0	0	1	1	1	0	1	1	1	1	1	1	0	1
0	0	1	1	1	1	1	1	1	1	1	1	1	0

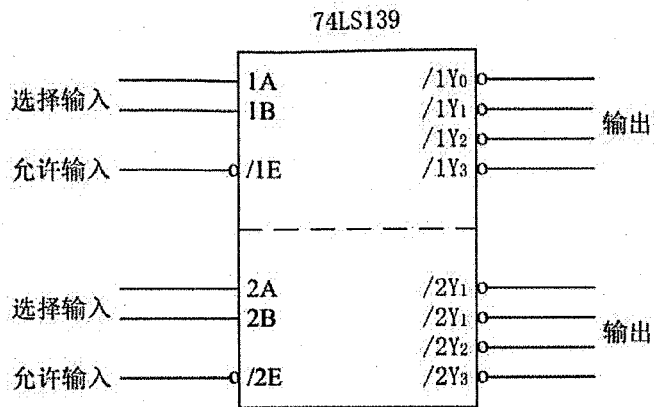
74LS138, 3-到-8 译码器功能表

在任何时候 8 个输出中只有一个会变成低电平

为使译码器的任一输出变为低电平,3 个允许输入(/G2A, /G2B 和 G1)均必须有效,/G2A 和 /G2B 输入必须都为低电平(逻辑 0),G1 必须为高电平(逻辑 1)

一旦 74LS138 被允许,地址输入(C,B 和 A)就选择某一个输出引脚变低

(3) 2-4 双线译码器



/E	A	B	/Y ₀	/Y ₁	/Y ₂	/Y ₃
0	0	0	0	1	1	1
0	0	1	1	0	1	1
0	1	0	1	1	0	1
0	1	1	1	1	1	0
1	X	X	1	1	1	1

74LS139, 双 2-4 线译码器的引脚图和真值表

第六章 输入输出与中断技术

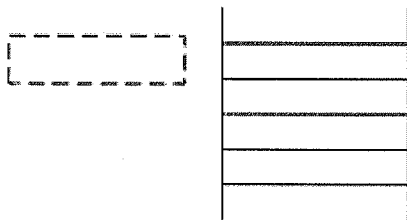
一、本章考点

1. 小题:

一般在 I/O 系统,数据传送方式,中断技术中考察两道.

例:

- (1) 比较存储器映像 I/O 系统与独立编址 I/O 系统.
- (2) 解释术语握手应用于计算机 I/O 系统的情形.
- (3) 输入端口与输出端口分别必须具备什么能力?
- (4) 8088 CPU 数据传送控制方式有哪几种?
- (5) 简述 DMA 方式工作过程.
- (6) 请从中断请求输入引脚,触发信号,是否受 IF 标志位控制的角度简述 8088/8086CPU 对于存储器奇偶错,电源掉电等紧急事件如何处理? 对一般的外部中断请求如何处理?
- (7) 列出微处理器上的中断引脚.
- (8) 8088/8086CPU 的 I/O 系统采用独立编址方式,它的优缺点是什么?
- (9) 中断源的分类
- (10) 中断服务程序的一般结构是什么?
- (11) 中断指令 INT 34H 调用用的中断服务程序的入口地址存放在存储器的哪几个单元?
- (12) 中断类型码为 30H 的中断服务程序,存放在 0300H:2900H 开始的存储区域中,请将该中断类型在中断向量表中的存放地址和内容,填入下图中.



答:

(1) 与存储器统一编址

这种方式又称为存储器映射编址方式.它将 I/O 端口作为内存单元对待,由 CPU 统一分配地址.通常在 CPU 的地址空间中划出一部分作为输入输出系统的端口地址范围,不再作为内存地址使用.

独立编址

CPU 给 I/O 端口分配一个独立的地址空间,提供专用的控制信号.I/O 端口地址空间与内存地址空间隔离.

- (2) 握手:握手或查询使用几条控制线使两个独立设备同步的方式.例如,计算机通过输入来自打印机的 BUSY 信号询问打印机是否“忙”,不忙则计算机输出数据给打印机,并用一个数据选通信号通知打印机数据有效.这种在计算机与打印机之间的通信就是一种握手或查询.
- (3) 输入端口必须具有通断控制能力.若外设本身具有数据保持能力,通常可以仅用一个三态门缓冲器作为输入接口,三态门具有“通断”控制能力.输出端口应具备数据锁存能力.
- (4) 无条件传送方式,程序查询传送方式,中断传送方式,直接存储器存取(DMA)方式.
- (5)
 - ① 外设向 DMA 控制器发出“DMA 传送请求”信号(DRQ);
 - ② DMA 控制器收到请求后,向 CPU 发出“总线请求”信号 HOLD;
 - ③ CPU 将数据总线,地址总线和相应的控制信号线均置为高阻态,向 DMA 控制器发出“总线响应”信号(HLDA).
 - ④ DMA 控制器收到 HLDA 信号后,就开始控制总线,并向外设发出 DMA 响应信号 DACK;
 - ⑤ DMA 控制器送出地址信号和相应的控制信号,实现外设与内存或内存与内存之间的直接数据传送;
 - ⑥ 规定的数据传送完后,DMA 控制器就撤销发往 CPU 的 HOLD 信号.CPU 检测到 HOLD 失效后,紧接着撤销 HLDA 信号,并在下一时钟周期重新开始控制总线.
- (6) 紧急事件采用可屏蔽中断,采用电平触发方式,高电平触发.CPU 收到中断请求信号后,检测中断允许标志位 IF,若 IF=1,CPU 响应 INTR 请求;若 IF=0 时,CPU 屏蔽 INTR 请求.被屏蔽的中断请求信号可一直保持高电平,直到 CPU 接收.
一般事件采用非屏蔽中断,非屏蔽中断不能被软件禁止,即不受 IF 标志位控制.NMI 采用上升沿触发,中断类型码为 2,由 CPU 内部译码产生.
- (7) 8086/8088CPU 含有 2 个中断请求信号输入引脚 INTR 和 NMI,还有 1 个中断应答引脚 (/INTA).
- (8) 优点:不占用内存空间,而且 I/O 端口地址线根数少,译码电路简单.

缺点:需要专用的控制信号和专用的 I/O 指令.

(9) 外部中断与内部中断

(10)

- ① 保护现场.在中断服务程序的起始部分安排若干条入栈指令,将各寄存器的内容压入堆栈保存.
- ② 开中断.在中断服务程序执行期间允许级别更高的中断请求中断现行的中断服务程序,实现中断嵌套.
- ③ 中断服务.完成中断源的具体要求.
- ④ 恢复现场.中断服务程序结束前,必须恢复主程序的中断现场.通常是将保存在堆栈中的现场信息弹出到原来的寄存器中.
- ⑤ 中断返回.返回到原程序的断点处,继续执行原程序

(11) 000D0 H~000D3 H

根据基础知识点,中断向量在中断向量表中的存放首地址称为向量地址,其值为:中断类型码 $\times 4$.

调用中断过程:

向量地址 $4n$,把 $4n+1:4n$ 两个单元的内容取出并装入 IP 寄存器;再把 $(4n+3:4n+2)$ 两个单元的内容取出并装入 CS 寄存器,CPU 就获得了 n 的中断服务程序的入口地址,进而转去执行中断服务程序.

入口地址为 $34H \times 4 = D0H$,所以 000D0 H~000D3 H

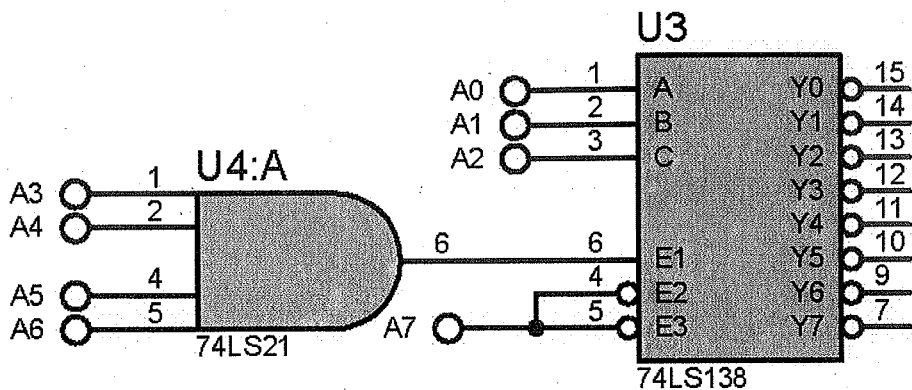
(12)

000C0H	
	00
	29
	00
	03

2. 大题:只考查 I/O 译码电路.

例:利用 74LS138 译码器与门电路,设计一个 8 位 I/O 端口的译码电路,产生 78H~7FH 的地址输出.

答:



E1,E2,E3 分别为 G1,G2A,G2B

二、基础知识

1. I/O 接口

概述:通常把处理器和主存储器之外的部分称为输入输出系统,包括输入输出设备,输入输出接口和输入输出软件.

(1) I/O 接口功能及分类:

● I/O 接口:

高速的微处理器与低速的外设能够协调的工作,需要有一个具有数据缓冲和锁存能力,数据格式转换能力,提供外设状态和定时控制能力的连接电路,我们把它叫做输入输出接口.

I/O 接口需要具备的功能:

- ① 地址译码
- ② 数据传送
- ③ 提供握手联络信号
- ④ 数据格式转换
- ⑤ 信号电平转换
- ⑥ 错误检测功能

握手:握手或查询使用几条控制线使两个独立设备同步的方式.例如,计算机通过输入来自打印机的 BUSY 信号询问打印机是否“忙”,不忙则计算机输出数据给打印机,并用一个数据选通信号通知打印机数据有效.这种在计算机与打印机之间的通信就是一种握手或查询.

● I/O 接口的分类:

按数据传送方式,I/O 接口分为并行接口和串行接口两类.

- ① 并行接口,一次传送一个字节或多个字节的所有位.
- ② 串行接口,一次传送一位,数据逐位传送.接口内必须有串/并转换部件.

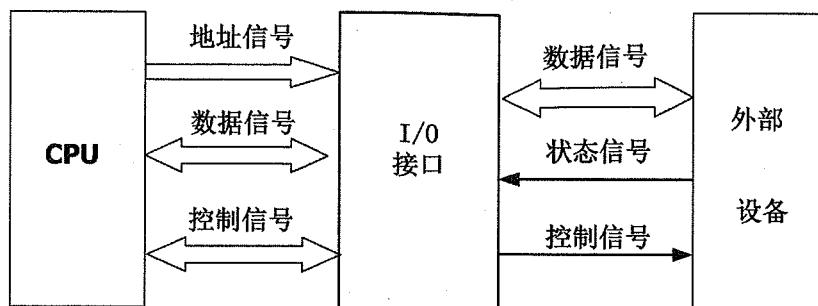


图6-1 CPU与外部设备之间的接口

(2) I/O 端口

① 分类:

1° 根据端口传输的信息,端口可分为数据端口,状态端口和控制端口,用以传输数据信息,状态信息和控制信息.状态端口为信息输入端口.I/O 接口在开始工作前,CPU 需要设定它的工作方式,这些信息存放在控制端口.

2° 根据信息传输方向,端口又可以分为输入端口,输出端口或是输入/输出的双向端口.

只用来输出数据的数据端口称为数据输出端口,只用来输入数据的数据端口称为数据输入端口.

输入端口必须具有通断控制能力.若外设本身具有数据保持能力,通常可以仅用一个三态门缓冲器作为输入接口,三态门具有“通断”控制能力.输出端口应具备数据锁存能力.

② I/O 端口的编址方式

编址:CPU 通过地址识别端口,端口地址也称为端口号.为 I/O 端口分配地址称为编址.

编址方式:两种,与存储器统一编址,独立编址方式

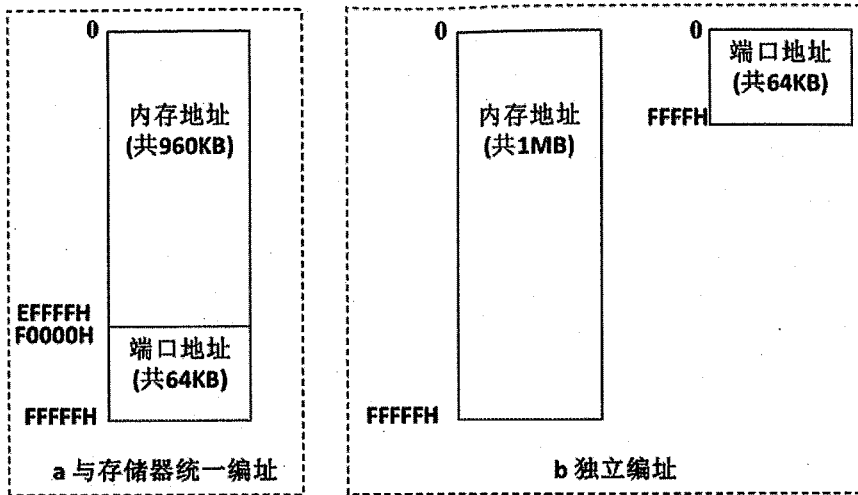


图6-2 两种不同编址方式的地址空间

1° 与存储器统一编址

这种方式又称为存储器映射编址方式.它将 I/O 端口作为内存单元对待,由 CPU 统一分配地址.通常在 CPU 的地址空间中划出一部分作为输入输出系统的端口地址范围,不再作为内存地址使用.

优点:访问 I/O 端口和访问内存单元一样,所有访问内存的指令都可以访问 I/O 端口,不用设置专门的 I/O 指令;也不需要专用的 I/O 端口控制信号,简化了系统总线.

缺点:占用一部分 CPU 地址空间.

2° 独立编址

CPU 给 I/O 端口分配一个独立的地址空间,提供专用的控制信号.I/O 端口地址空间与内存地址空间隔离.

优点:不占用内存空间,而且 I/O 端口地址线根数少,译码电路简单.

缺点:需要专用的控制信号和专用的 I/O 指令.

注:

8088 系统中,内存空间 00000H~FFFFFH (1MB),使用 A0~A19 全部 20 根地址线寻址.I/O 地址空间 0000H~FFFFH (64KB),使用 A0~A15 共 16 根地址线寻址.

在最大模式下,CPU 读写内存使用(/MEMR)和(/MEMW)控制信号,读写 I/O 端口使用(/IOR)和(/IOW).

在最小模式下,内存空间和 I/O 空间由 IO/ (M) 区分,IO/ (M) 为逻辑 0,CPU 读写内存;IO/ (M) 为逻辑 1,CPU 读写 I/O 接口.读写控制信号为 (/RD)和(/WR).

数据输入输出使用专用指令 IN 和 OUT.

端口位宽:I/O 端口一次能够传送数据的位数,称为端口位宽.

端口位宽有 8 位,16 位和 32 位几种.16 位的端口实际上是连续编址的 2 个 8 位端口,32 位端口实际上是连续编址的 4 个 8 位端口.

3° I/O 端口地址译码

- I/O 端口的地址译码与内存地址译码原理一样,可以用基本逻辑门电路搭建,也可以使用专用的译码器译码.
- 与存储器统一编址的 I/O 端口地址译码较为复杂.
独立编址的 I/O 端口,地址译码电路简单.
- 8088 系统中 I/O 端口地址译码是对 A0~A15 共 16 根地址线译码,控制信号为(/IOR)和(/IOW)或者 IO/ (/M) ,(/WR),(/RD).

在嵌入式系统中或者小型系统中,如果 I/O 设备端口不超过 256 个,为了简化地址译码电路,则可以只对 A0~A7 地址线译码.

(3) 基本输入输出接口

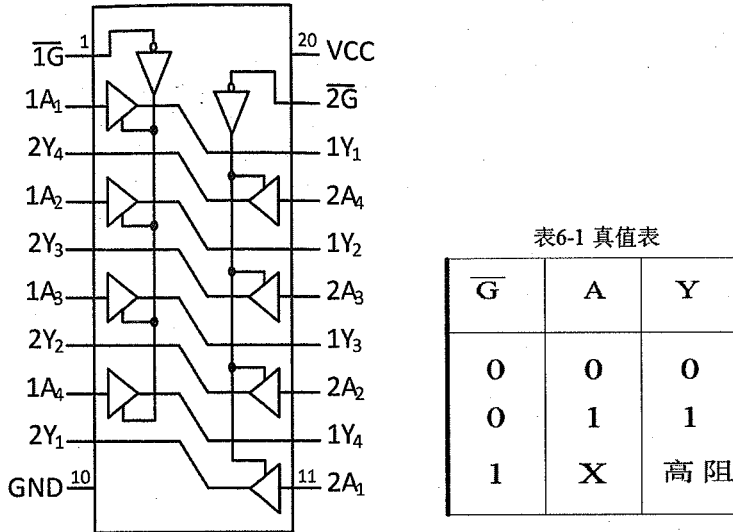
① 基本输入接口

基本输入接口是一组三态缓冲器.输入端口必须具有“通断”控制能力.

若外设本身具有数据保持能力,通常可以仅用一个三态门缓冲器作为输入接口.

芯片 74LS241,74LS244,74LS245,74LS240 (反相) 等 8 位总线缓冲器/驱动器/接收器,都是常用的总线接口器件.三态缓冲器是接口电路的基础电路,每个接口电路都必须有此部分.

74LS244



两个控制端:1G 和 2G,低电平有效,各控制 4 个三态门

A 端为数据输入端,Y 端为数据输出端.

当控制端为低电平时,数据从 A 端输入,Y 端输出;当控制端为高电平时,三态门呈高阻状态,A 端与 Y 端断开.

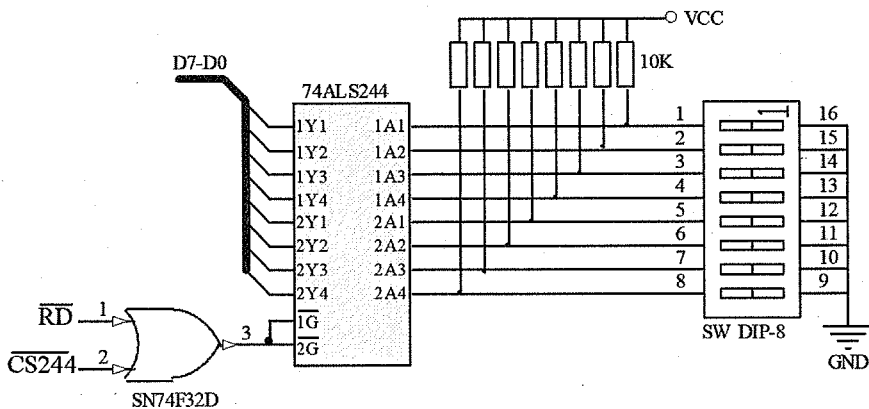


图6-6 输入开关状态的基本输入接口

② 基本输出接口

输出接口应具备数据锁存能力.锁存器或触发器都具有数据保持能力,经常设置在接口内部.

如 74ALS273 (8D 触发器),74ALS373 (8D 锁存器/触发器),74ALS374 (8D 锁存器/触发器),74ALS574 (8D 锁存器/触发器).

8 个数据输入端 D7~D0,对应应有 8 个数据输出端 Q7~Q0.MR 引脚为主清除端,低电平有效.CP 为触发端,上升沿触发.当 CP 从低电平到高

电平跳变时,D7~D0 端的数据输出到 Q7~Q0 并保持,直到 CP 端下一个上升沿信号出现。

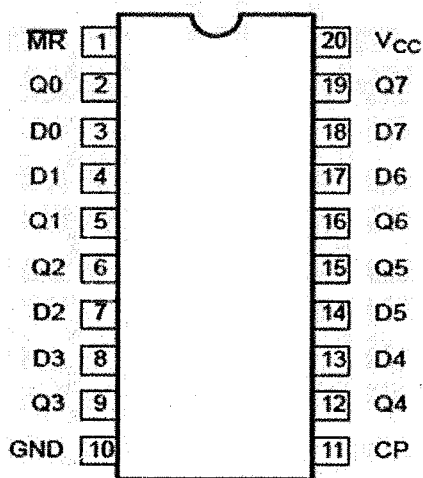


图6-10 74ALS273

表6-3 74ALS273真值表

$\overline{\text{MR}}$	CP	Dn	Qn
0	X	X	0
1	$\uparrow$	1	1
1	$\uparrow$	0	0

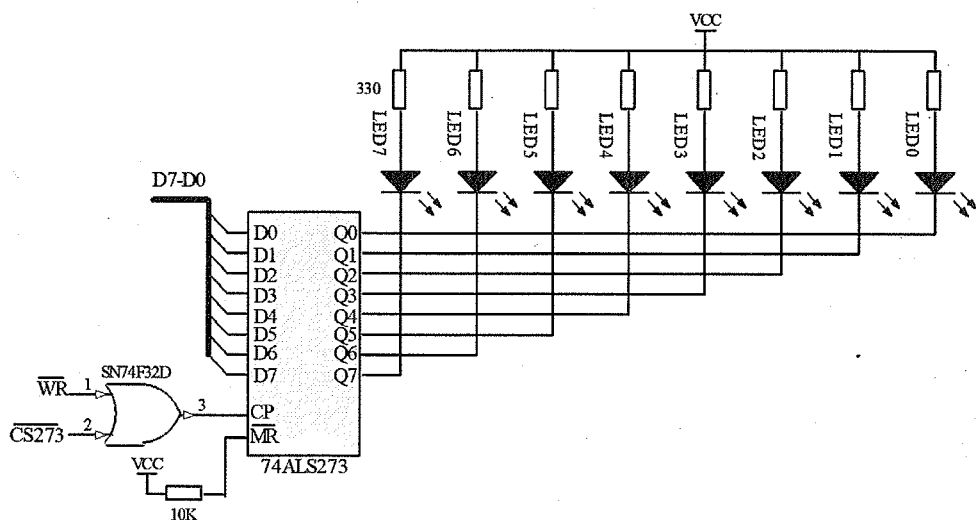


图6-9 驱动8只LED灯的基本输出接口

2. 数据传送控制方式

CPU 数据传送控制方式:无条件传送方式,程序查询传送方式,中断传送方式,直接存储器存取 (DMA) 方式。

无条件传送方式,程序查询方式和中断方式在数据传送过程中,CPU 从内存读出数据,再输出到外部设备,因此,这三种方式被统称为程序控制下的输入输出方式

(1) 无条件传送方式

无条件传送方式又称为同步方式,适合简单外设的数据输入输出,外设

随时可以接收数据,随时可以提供数据,永远与 CPU 同步,CPU 可直接与其进行数据交换.

(2) 程序查询传送方式

CPU 与外设接收或发送信息速度比外设快,CPU 与外设交换信息时,首先要查询设备目前的工作状态,当设备准备好,CPU 就与其交换信息,否则就等待,这种工作方式称为程序查询方式或握手.

例:打印机 (每秒 200 字符)

CPU 通过数据端口 DataP 传送数据 (D7~D0),并产生一个 /STB 信号作为数据选通信号发给打印机.打印机设置 BUSY 信号为逻辑 1 表示打印机“忙”,CPU 通过状态端口 StateP 读取状态信息 BUSY.

;打印 BL 中的字符

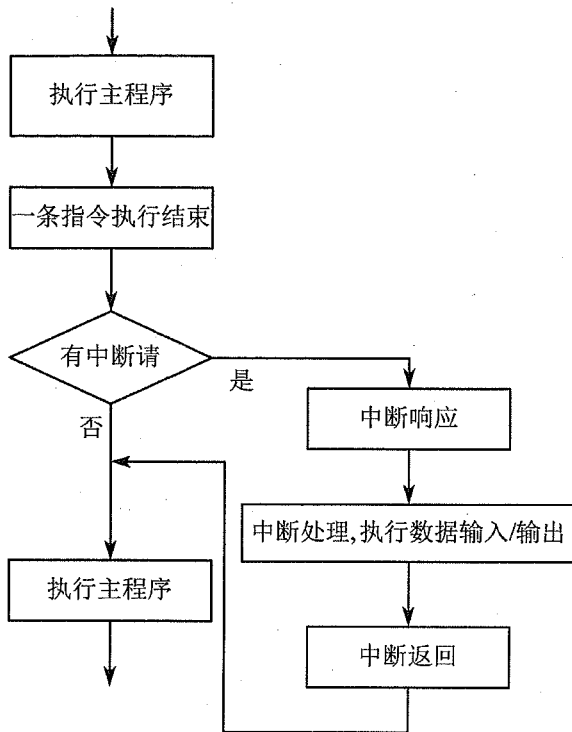
PRINT PROC NEAR

```
WAIT1:  IN  AL, StateP      ;输入状态字
        TEST AL, BUSY_BIT  ;测试状态位
        JNZ  WAIT1         ;不为 0 继续测试
        MOV  AL, BL
        OUT  DataP, AL     ;输出数据到打印机
        RET
```

PRINT ENDP

(3) 中断传送方式

中断是一种异步机构.



(4) DMA 方式

直接存储器存取方式简称为 DMA 方式。

是在内存储器 and I/O 设备之间建立数据通路, 不经过 CPU 的干预, 实现内存与外设之间的快速数据传送。

① 功能

DMAC 接收外设请求信号, 并向 CPU 发出总线请求信号。

DMA 过程中可实现: 向地址总线发送地址信号, 能够生成读写控制信号, 控制数据传送的字节数。

DMA 过程结束时, 能向 CPU 发出 DMA 结束信号, 并将总线的控制权交还给 CPU。

1° 工作流程:

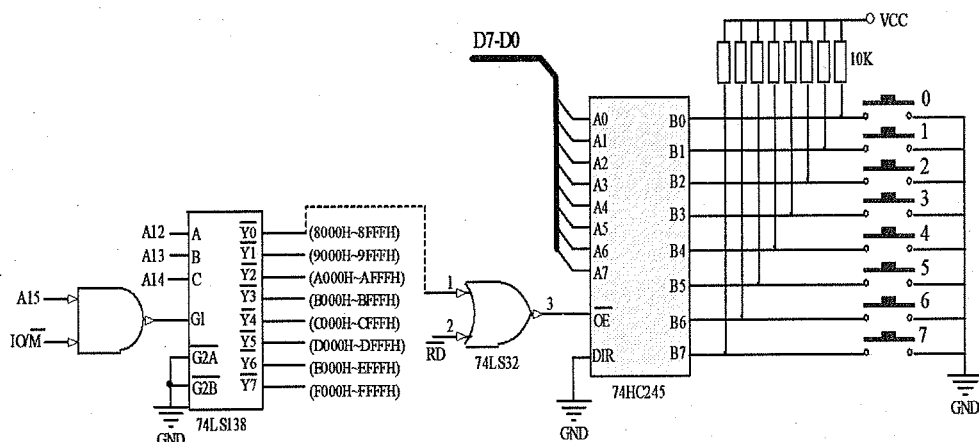
- ① 外设向 DMA 控制器发出“DMA 传送请求”信号 (DRQ);
- ② DMA 控制器收到请求后, 向 CPU 发出“总线请求”信号 HOLD;
- ③ CPU 将数据总线, 地址总线和相应的控制信号线均置为高阻态, 向 DMA 控制器发出“总线响应”信号 (HLDA)。
- ④ DMA 控制器收到 HLDA 信号后, 就开始控制总线, 并向外设发出 DMA 响应信号 DACK;

- ⑤ DMA 控制器送出地址信号和相应的控制信号,实现外设与内存或内存与内存之间的直接数据传送;
- ⑥ 规定的数据传送完后,DMA 控制器就撤销发往 CPU 的 HOLD 信号.CPU 检测到 HOLD 失效后,紧接着撤销 HLDA 信号,并在下一时钟周期重新开始控制总线.

3. 键盘,显示接口

(1) 键盘:识别键盘上哪个键被压下的过程称为键盘扫描.

例 1 线性键盘



PORT245 EQU 8000H

;8 个按键的线性键盘,按键识别过程.

;AH 中含有键值,如果 AH 为 8 则为出错

KEY PROC NEAR

AGAIN: MOV DX, PORT245

K1: IN AL, DX

CMP AL,0FFH

JZ K1 ;无按键,继续等待

CALL D20ms ;延时 20ms 去抖动

MOV DX,PORT245 ;重新检测

IN AL, DX

CMP AL, 0FFH

JZ K1 ;说明刚才检测到的按键是源于干扰,重新检测

MOV AH,0 ;识别按键

K2: SHR AL,1

JNC K3 ;是 0 转 K3,键被按下

字型	共阳极代码	共阴极代码	字型	共阳极代码	共阴极代码
0	C0H	3FH	9	90H	6FH
1	F9H	06H	A	88H	77H
2	A4H	5BH	B	83H	7CH
3	B0H	4FH	C	C6H	39H
4	99H	66H	D	A1H	5EH
5	92H	6DH	E	86H	79H
6	82H	7DH	F	8EH	71H
7	F8H	07H	灭	FFH	00H
8	80H	7FH			

(3) 16*16 LED 点阵显示接口（一般不考察）

4. 中断

(1) 概念

中断,是指在程序执行过程中,出现某种紧急事件,CPU 暂停执行现行程序,转去执行处理该事件的程序——中断服务程序,执行完后再返回到被暂停的程序继续执行,这一过程称为中断。

(2) 中断源

中断源可以笼统地分为两类:CPU 内产生的,称为内部中断;其他的称为外部中断。

① 内部中断:

由 CPU 本身产生的中断,由控制器产生的中断,由程序员安排的中断指令引起的中断。

② 外部中断:

根据中断事件的紧迫程度将中断源划分为可屏蔽中断和不可屏蔽中断。

可屏蔽中断是指可以延时处理的事件。

不可屏蔽中断是指事件异常紧急,必须马上处理,例如掉电,内存奇偶校验错引起的中断。

(3) 中断处理的基本过程

中断处理的基本过程包括中断请求,中断判优,中断响应,中断服务和中断返回等五个阶段。

① 中断请求:外部中断

- 发生在 CPU 内部的中断,不需要中断请求,CPU 内部的中断控制逻

辑直接接收处理.

- 外部中断请求由中断源提出.外部中断源利用 CPU 的中断输入引脚输入中断请求信号.一般 CPU 设有两个中断请求输入引脚:可屏蔽中断请求输入引中断请求触发器脚和不可屏蔽中断请求输入引脚.
- 中断请求触发器:每个中断源都有一个中断请求触发器,锁存自己的中断请求信号,并保持到 CPU 响应这个中断请求之后才将其清除.
- 中断允许触发器:在 CPU 内部有一个中断允许触发器,当其为“1”时,允许 CPU 响应中断,称为开中断.

② 中断判优

- 概念:当多个中断源同时向 CPU 提出中断请求时,CPU 必须找出中断优先级最高的中断源,这一过程称为中断判优.
- 中断判优可以采用硬件方法,也可采用软件方法.
- 硬件方法常见方式:菊花链判优电路和中断控制器判优.

③ 中断响应

中断响应时,CPU 向中断源发出中断响应信号,同时:

- 1° 保护硬件现场;
- 2° 关中断;
- 3° 保护断点;
- 4° 获得中断服务程序的入口地址.

④ 中断服务

中断服务程序的一般结构为:

- 1° 保护现场.在中断服务程序的起始部分安排若干条入栈指令,将各寄存器的内容压入堆栈保存.
- 2° 开中断.在中断服务程序执行期间允许级别更高的中断请求中断现行的中断服务程序,实现中断嵌套.
- 3° 中断服务.完成中断源的具体要求.
- 4° 恢复现场.中断服务程序结束前,必须恢复主程序的中断现场.通常是将保存在堆栈中的现场信息弹出到原来的寄存器中.
- 5° 中断返回.返回到原程序的断点处,继续执行原程序

⑤ 中断返回

返回到原程序的断点处,恢复硬件现场,继续执行原程序.中断返回操作是中断响应操作的逆过程.

(4) 8086/8088 中断系统

8086/8088CPU 含有 2 个中断请求信号输入引脚 INTR 和 NMI,还有 1 个中断应答引脚 (/INTA).与中断相关的指令有:INT,INTO,INT3 和 IRET,还有 2 个与中断相关的标志位 IF,TF.

① 中断向量和中断向量表

8086/8088CPU 的中断系统可以处理 256 种中断.

1° 中断向量:

- 1) 概念:每种中断都有对应的中断服务程序.中断服务程序的入口地址称为中断向量.
- 2) 内容:每个中断向量都包括两部分:段基址和偏移地址.因此,存放 1 个中断向量需要 4 个内存单元,256 种中断向量共需要 1K 个内存单元.

2° 中断向量表:

- 1) 概念:256 种中断向量存储在内存中构成一张表,称为中断向量表.
- 2) 内容:中断向量表位于内存起始地址 00000~003FFH 的存储区内.从地址 00000H 开始,每 4 个单元存放一个中断向量,其中低地址的两个单元存放中断向量的偏移地址,高地址的两个单元存放中断向量的段基址.256 种中断向量按中断向量码从 0 到 255 的顺序依次存入中断向量表中.

3° 向量地址:

中断向量在中断向量表中的存放首地址称为向量地址,其值为:中断类型码 $\times 4$.

4° 调用中断过程:

向量地址 $4n$,把 $4n+1:4n$ 两个单元的内容取出并装入 IP 寄存器;再把 $(4n+3:4n+2)$ 两个单元的内容取出并装入 CS 寄存器,CPU 就获得了 n 的中断服务程序的入口地址,进而转去执行中断服务程序.

② 中断类型

1° 微处理器专用中断

全部 256 种中断类型中 0~31.

8086/8088CPU 专用中断类型有 5 个:

- 1) 类型 0:除法错中断,除法运算结果溢出或是除以 0 时发生,屏幕出现错误提示.
- 2) 类型 1:单步中断,也称为陷阱中断.如果 TF 标志位为 1,CPU 每执

行一条指令后,就停下来等待,并提示 CPU 内部各寄存器的值和一些附带信息.

- 3) 类型 2:非屏蔽中断,该中断不能被软件禁止.
- 4) 类型 3: 断点中断,由一个单字节指令 INT3 引起的中断,指令代码为 0CCH.
- 5) 类型 4:溢出中断,INTO 指令的专用向量.如果溢出标志位 OF 为 1,则 INTO 指令中断正在执行的程序.

2° 用户中断

从类型 32 开始直到 255 作为用户中断类型,用户中断向量由用户填写到中断向量表中,中断服务程序也由用户编写.

③ 硬件中断

8086/8088CPU 芯片有两个硬件中断请求信号输入引脚:NMI 和 INTR,用来接收外部中断源产生的中断请求.NMI 引脚接收非屏蔽中断请求,INTR 引脚接收可屏蔽中断请求

1° 非屏蔽中断

非屏蔽中断不能被软件禁止,即不受 IF 标志位控制.NMI 采用上升沿触发,中断类型码为 2,由 CPU 内部译码产生.

2° 可屏蔽中断

采用电平触发方式,高电平有效.

CPU 收到中断请求信号后,检测中断允许标志位 IF,若 IF=1,CPU 响应 INTR 请求;若 IF=0 时,CPU 屏蔽 INTR 请求.被屏蔽的中断请求信号可一直保持高电平,直到 CPU 接收.

可屏蔽中断的优先级低于不可屏蔽中断.

④ 中断处理流程

- CPU 在每条指令的最后一个时钟周期按照下列顺序检测有无中断请求:

- 1° 指令执行时是否有异常情况发生,如除法错;
- 2° 有没有单步中断请求 (TF=1);
- 3° 有没有 NMI 非屏蔽中断请求;
- 4° 有没有协处理器段超限;
- 5° 有可屏蔽中断请求信号吗?
- 6° 是中断指令吗?

- 如果有一个或多个中断条件出现,CPU 响应中断.如果检测到内

部中断或非屏蔽中断,CPU 从内部获得中断类型码;如果检测到可屏蔽中断请求,CPU 进一步测试 IF 标志位,如果 IF=1,CPU 就进入中断响应总线周期,从中断控制器获取中断类型码.

- 获得中断类型码之后,各种中断的处理过程相同.CPU 将中断类型码放入暂存器保存,以下动作顺序发生:

标志寄存器的内容入栈;

清除中断标志 IF 和 TF;

CS 的内容入栈;

IP 的内容入栈;

根据中断类型码,在中断向量表中取出中断向量装入 IP 和 CS;

执行中断服务程序;

中断返回.

第七章 可编程接口芯片

一、本章考点

1. 小题

本章一般出小题几率较小,若考察,一般从工作方式与控制字方向出题.

例:

- (1) 82C55 有哪三种工作方式?
- (2) 82C55 的选通输入操作中用什么来置位 IBF 引脚?
- (3) 82C55 的选通输入操作中是如何允许中断请求引脚的?
- (4) 82C55 的选通输出操作中,/ACK 信号的用途是什么?
- (5) 8254 的每个计数器可以用多少种不同方式工作?
- (6) 解释简单四线圈步进电机的工作.

答:

- (1) 方式 0,基本输入输出方式

适合于三个端口;作为输出口时它具有锁存功能,作为输入时不能锁存 A 口,C 口的高 4 位,B 口以及 C 口的低四位可分别定义为输入或输出,各端口相互独立,故共有 16 种组合.

方式 1:选通输入输出方式.A 口,B 口作为数据的输入口或输出口,但数据的输入输出要在控制信号的控制下,控制信号由 C 口的某些位来提供;

方式 1 适用于程序查询和中断控制方式,它不能工作在无条件传送方式下;

方式 2,双向传输方式,只有 A 口可以工作在这种方式下.既能发送数据,又能接收数据;它还要利用 C 口的 5 条线作为控制信号线; A 口的输入输出均具有数据锁存能力,数据的输入输出均可以引起中断.

- (2) IBF 是/STB 的应答信号,输入选通信号/STB 有效时,IBF=1

(3)

- ① CPU 通过执行 OUT 指令写“方式选择控制字”到 8255A,设定端口 A 为“方式 1 输入”,然后置位 PC4=1,于是 INTEA=1,端口 A 处于中断允许状态.
- ② 当外设的选通信号(/STB)有效时,外设的数据装入端口 A 的输入数据

缓冲器, $IBF = 1$.

③ 这时 $INTE=1, IBF=1, (/STB) = 1$, 所以 $INTR$ 由 0 变 1, 端口 A 向 CPU 发出中断请求信号 $INTR$.

(4) $/ACK$: 外设收到数据的应答信号.

(5) 6 种

(6) 采用单极性直流电源供电, 只要对步进电机的各相绕组 (线圈) 按合适的时序通电, 就能使电机转动. 步进电机在 360° 内旋转时是不连续的. 每当电机绕组接受一个电脉冲, 转子就转过一个相应的步距角. 步进电机的转速应由慢到快逐步加速. 电机绕组的通电顺序控制电机的转向.

2. 大题: 一般必考两道大题

(1) 一道大题 82C55(8255A) 应用

基础知识点 1.5 处例题 3, 例题 5 (一般考察)

不同方式下 8255A 编程; 步进电机或键盘编程, 一般要求编程一小部分, 读懂并理解例子程序. (几率较小)

(2) 一道大题 8253 应用

基础知识点 2.4 例 9, 例 10

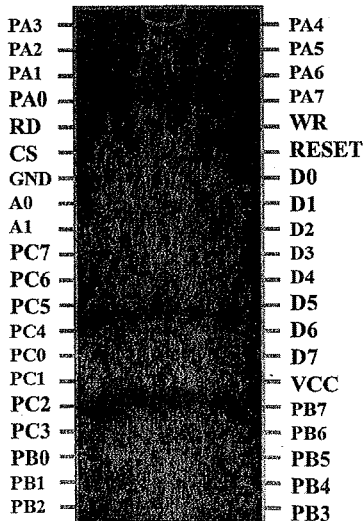
题型与例题类似, 主要掌握编程思路.

二、基础知识点

1. 可编程外围设备接口: 8255A

(1) 功能结构

① 引脚及功能



40 引脚,单一+5V 电源,双列直插式;

D7~ D0:数据信号线;

CS:片选信号;

A0A1 口地址选择信号;

WR:写信号;

RD:读信号线;

Reset 复位信号高电平有效; 8255 复位后,各端口都被预置为输入状态.

PA0 ~ PA7 :端口 A 的 8 条输入输出线

PB0 ~ PB7 :端口 B 的 8 条输入或输出线

PC0 ~ PC7 :端口 C 的 8 条输入或输出线,也可以用作控制信号的输出或状态信号的输入线;

② 端口地址

8255A 包含 A 口,B 口,C 口三个独立的数据端口,及一个控制端口.各端口地址如下:

A1A0=00 A 口

A1A0=01 B 口

A1A0=10 C 口

A1A0=11 控制端口

数据端口特点:

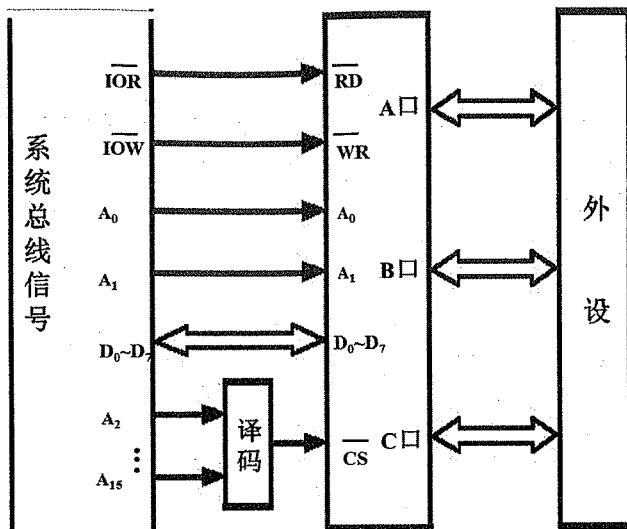
A 口,B 口,C 口可以作为三个独立的数据输入输出通道(端口)单独使用;也可以将 A 口,B 口作为数据输入输出, C 口输出控制信号或输入状态信号;

A 口——具有数据输入输出功能,且能锁存;

B 口——具有数据输入输出功能,且能锁存;

C 口——具有数据输入输出功能,输出时可以锁存;

C 口的主要特点是可以按位进行操作;可以分为 2 个 4 位的端口;既可以传送数据也可以用来输出控制信号或输入状态信号;



8255 与系统的连接示意图

(2) 工作方式

三种工作方式:方式 0,方式 1,方式 2

① 方式 0:

基本输入输出方式

适合于三个端口;作为输出口时它具有锁存功能,作为输入时不能锁存。A 口、C 口的高 4 位、B 口以及 C 口的低 4 位可分别定义为输入或输出,各端口相互独立,故共有 16 种组合。C 口可以按位进行置位和复位。

最适用于无条件传送方式;

也能用于查询工作方式,这时常将 C 口的高 4 位定义为输入口,输入外设的状态信号,而将 C 口的低 4 位定义为输出口,输出控制信息;这时可用 A、B 口传送数据;方式 0 不能工作中断方式;

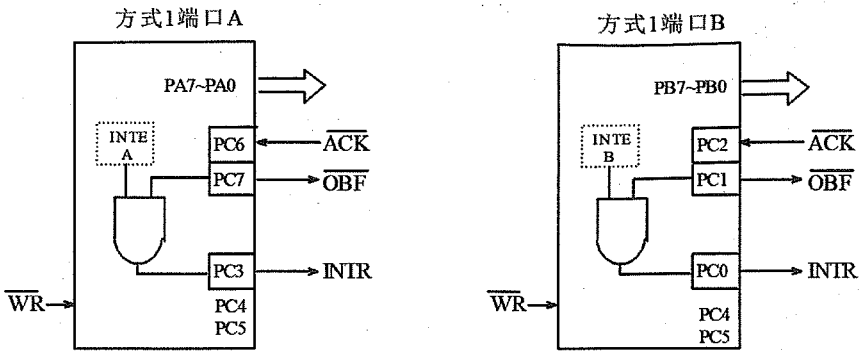
② 方式 1:

选通输入输出方式:A 口、B 口作为数据的输入口或输出口,但数据的输入输出要在控制信号的控制下,控制信号由 C 口的某些位来提供;

方式 1 适用于程序查询和中断控制方式,它不能工作在无条件传送方式下;

1° A、B 口输出:

C 口的 6 条线作为选通控制信号:PC3、PC6、PC7 配合 A 口,PC0、PC1、PC2 配合 B 口;



/OBF:输出缓冲器满信号; /ACK:外设收到数据的应答信号;

INTE:中断允许状态; INTR:中断请求信号;

工作过程:

查询方式

查询端口 C 的 BIT7 (PC7),如果是 0 则等待;如果不是 0,则 CPU 发送数据.

```
WAIT:  IN  AL,PORTC
        TEST AL,BIT7
        JZ  WAIT
        MOV AL,[SI]
        OUT PORTA,AL
```

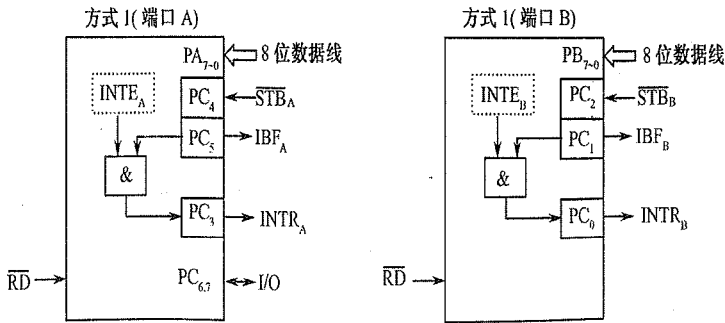
中断方式

置位 PC6,允许中断. (位操作命令)

等待中断.

在中断服务程序中发送数据.

2° A,B 口输入:



STB:输入选通信号;外设用它将数据锁存于输入数据缓冲器中;

IBF:数据输入缓冲器满,/STB 的应答信号;(/STB 有效,IBF 有效;当 INTE=1,IBF=1,/STB=1 时,INTR 由 0 变 1,端口 A 向 COU 发出中断请求信号 INTR,外设停止发送数据,CPU 开始进入中断程序)

INTR:中断请求信号 INTR;

INTE:中断允许信号;

以端口 A 为例,在允许中断情况下,方式 1 输入数据的工作过程为:

① CPU 通过执行 OUT 指令写“方式选择控制字”到 8255A,设定端口 A 为“方式 1 输入”,然后置位 PC4=1,于是 INTEA=1,端口 A 处于中断允许状态.

② 当外设的选通信号(/STB)有效时,外设的数据装入端口 A 的输入数据缓冲器,IBF =1.

③ 这时 INTE=1,IBF=1,(/STB) =1,所以 INTR 由 0 变 1,端口 A 向 CPU 发出中断请求信号 INTR.

④ CPU 响应中断,进入中断服务程序,通过执行 IN 指令从 A 口输入数据.(/RD)的下降沿使 INTR=0,清除中断请求,(/RD)的上升沿使 IBF =0,此时端口 A 又可以接收外设送来的新数据了.

方式 2:

双向传输方式,只有 A 口可以工作在这种方式下.既能发送数据,又能接收数据;它还要利用 C 口的 5 条线作为控制信号线; A 口的输入输出均具有数据锁存能力,数据的输入输出均可以引起中断;

方式 2 适用于程序查询和中断控制方式.它不能工作在无条件传送方式下.

这时 B 口可以工作在方式 0 或方式 1,C 口剩余的 3 条线可以作为输入输出线或方式 1 下的控制线.

(3) 8255 控制字

① 方式选择控制字 (82C55 初始化)

8255 初始化时用方式选择控制字进行初始化.

方式选择初始化示例:

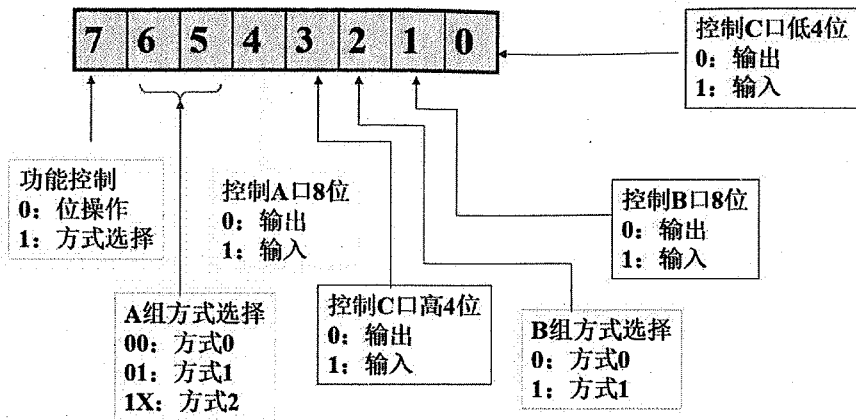
A 口,B 口工作在方式 0 下,A 口输出,B 口输入,则:10000010

A 口工作在方式 0 ,B 口工作在方式 1 下,A 口输出,B 口输入.

10000110

A 口工作在方式 1 ,B 口工作在方式 0 下,A 口输入,B 口输出.

10110000



例 1: 设 8255 的端口地址为 FBC0~FBC3H, A 口设置方式 0 输入, B 口方式 0 输出, C 口高 4 位方式 0 输出, C 口低 4 位方式 0 输入.

答: 控制字为: 10010001B

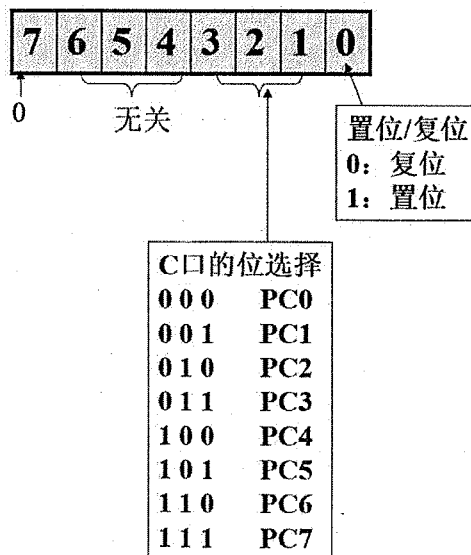
8255 初始化程序为:

```
MOV DX, 0FBC3H
```

```
MOV AL, 91H
```

```
OUT DX, AL
```

② 按位操作控制字 (控制 C 端口)



设置 PC0 为置位, 则:

00000001

设置 PC4 为复位,则:

00001000

设置 PC3 为置位,则:

00000111

设控制端口的地址为:

FFF7H,则:

MOV DX,0FFF7H

MOV AL,08H

OUT DX,AL

MOV AL,07H

OUT DX,AL

(4) 8255A 与微处理器连接

8255A 的初始化:在 8255 控制端口写入方式选择控制字

例 2 要求设置 8255A 的 A 组和 B 组都为方式 0,端口 C 的高 4 位作输出,低 4 位作输入.

程序如下:

MOV DX, PORT-C ;8255A 的控制口地址送 DX

MOV AL, 10000001B ;按要求的方式选择控制字内容送 AL

OUT DX, AL ;写入控制口,完成初始化

(5) 8255A 应用实例

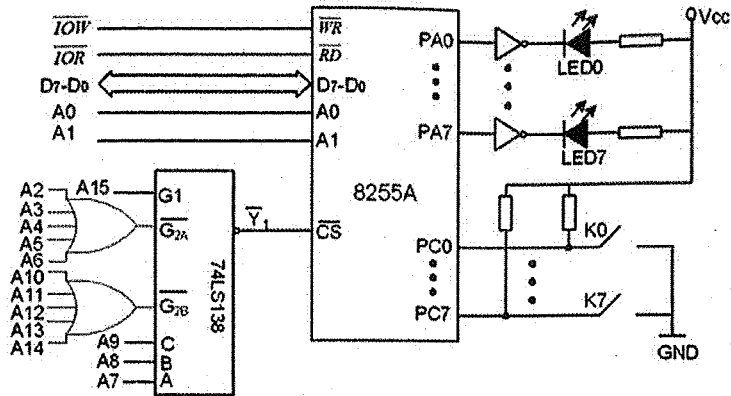
类 1:方式 0 运用

例 3:

如图所示,8255 的 A 口,C 口均工作在方式 0 下.以 8255 的 PA 口为输出口,控制 8 个单色 LED 灯;PC 口为输入口,连接 8 个开关 K0~K7,根据开关状态说明:

① 8255 的端口地址及方式控制字;

② 编程控制:检测开关的状态,如果 K0~K7 全闭合,PA0~PA7 控制的灯亮,否则控制的灯灭 .



解题思路:

- ① 端口地址:读图得出,图中 A19~A0 对应值为端口地址.根据第五章 3—8 译码器内容,得出 A19~A2=1000 0000 1000 00,由 8255 端口基础知识:

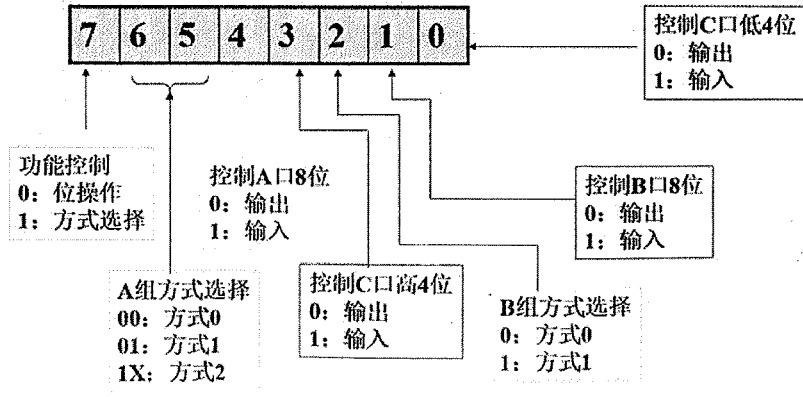
8255A 包含 A 口,B 口,C 口三个独立的数据端口,及一个控制端口.各端口地址如下:

- A1A0=00 A 口
- A1A0=01 B 口
- A1A0=10 C 口
- A1A0=11 控制端口

可得出端口地址:

A 口:8080H; B 口:8081H; C 口:8082H; 控制端口:8083H

A,C 口均工作在方式 0 下,B 口也工作在方式 0 下.A 口输出,B 口输入输出均可,C 口输入.可以写出方式选择控制字:10001001=89H 或 1000 1011=8B H



- ② 编程思路:8255A 初始化;输入端口输入按键状态;根据输入的按键状态输出端口输出相应值.

分析题干,只有 K0~K7 全闭合时接到 A 口的 7 个灯全部点亮.7 个灯点亮时,由于二极管正向端接高电平,负向端连非门连 A 口,所以全部点亮时,C 口:00H,A 口输出:FFH;其他状态,C 口:01H~FFH,A 口输出:00H.

答案:

- ① 端口地址:A 口:8080H; B 口:8081H; C 口:8082H; 控制端口:8083H
方式控制字:10001001=89 H 或 10001011=8B H
- ② 程序如下:

```
CODE SEGMENT
ASSUME CS:CODE
START PROC NEAR
```

```
    MOV DX,8083H    ;赋给 DX 控制端口地址
    MOV AL,8BH      ;AL 中写入方式选择控制字
    OUT DX,AL        ;将方式选择控制字写入控制端口中
; 8255A 初始化过程
```

L0:

```
    MOV DX,8082H    ;输入口 C 口地址
    IN AL,DX         ;C 口把开关状态送入 DX 中
    CMP AL,0
    JZ HIGH          ;C 口所有开关闭合跳转到灯亮程序
    MOV DX,8080H     ;输出口 A 口地址
    MOV AL,0
    OUT DX,AL        ;灯灭程序
    JMP L1            ;跳到程序末尾
```

HIGH:

```
    MOV DX,8080H
    MOV AL,0FFH
    OUT DX,AL        ;灯亮程序
```

L1: JMP L0 ; 回到初始化下一步继续检测开关状态输入情况,实现循环

```

START ENDP
CODE ENDS
END START

```

类二:方式 1 选通输入输出运用

方式 1 选通输入:

键盘编码器编码器,当一个按键被按下时,键盘编码器去除按键的抖动,输出按键的 ASCII 编码的键代码,同时提供一个选通信号(/DAV) 键盘模块的数据线与端口 A 的引脚连接,(/DAV)与端口 A 的(/STB)连接.当键盘模块上有键按下时,(/DAV)被激活并维持 1.0us.8255A 利用(/STB)脉冲信号从端口 A 的引脚上捕获数据.PC5 为输入缓冲器满信号,是 IBF 有效,外设停止发送数据,CPU 开始接受信号标志.可以使用查询方式,也可以使用中断方式读取键盘编码.

例 4:

;通过 8255A 接收键盘模块的键值.

;设置 8255A 的 A 组工作在方式 1,端口 A 做输入.控制字为:10110010
B

;查询方式读取键盘编码,并从 AL 中返回其 ASCII 码键值

```

BIT5      EQU 20H    ;0010 0000,选中 PC5
PORTA     EQU 50H
PORTC     EQU 52H    ;定义端口地址
READ      PROC NEAR
    MOV AL, 0B2H
    OUT 53H, AL      ;初始化 8255
L1:    IN AL, PORTC
    TEST AL, BIT5    ;测试 PC5,缓冲器满开始接收数据
    JZ L1
    IN AL, PORTA     ;AL 中返回键值
    RET

```

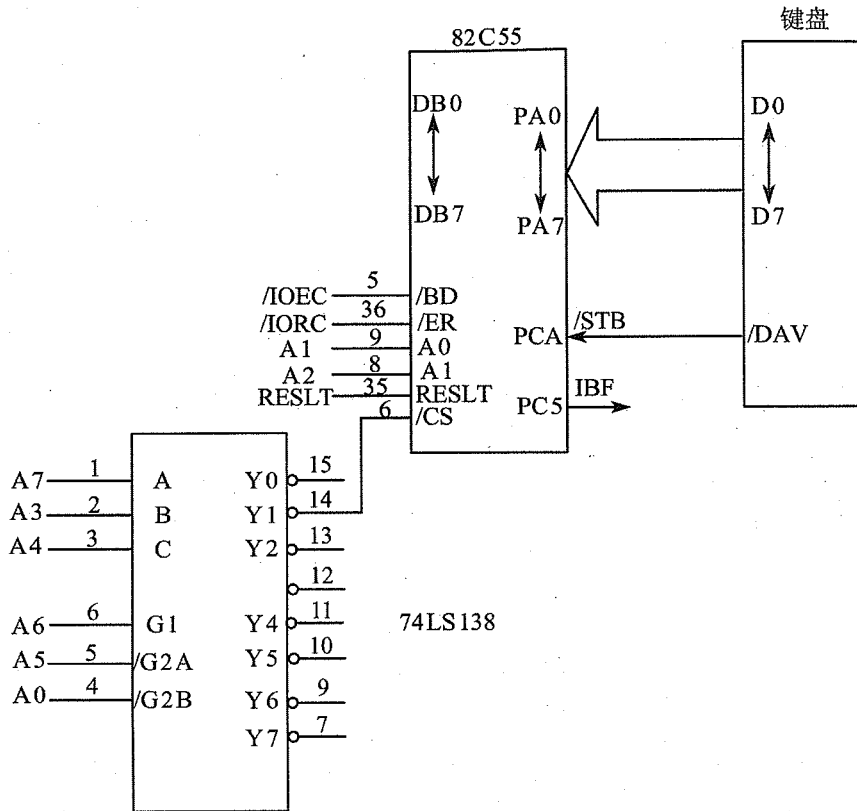
```

READ      ENDP

```

例 5:微处理器通过 74LS138 译码器给 8255 芯片分配端口地址.8255 以方式 1 选通输入方式与键盘相连,每次当在键盘上按下一个键时,数据有效信号/DAV 被激活,与之相连的 8255 端口 B 的/STB 选通控制输入端将键盘数据送入端口 B.设计一个子程序,从端口 A 读取键值,并

将返回的 ASCII 码存于 BH 中。



答案:

分析:端口地址:A7~A0=11000XX0H=CX H

A:C0H B:C2H C:C4H 控制端口:C6H

控制字:1011 1111 H

编程思路:

方式1选通输入,先对8255A初始化,再编程.数据输入后检测PC5口, PC5=1,缓冲器满把AL中数据传送至BH中。

程序:

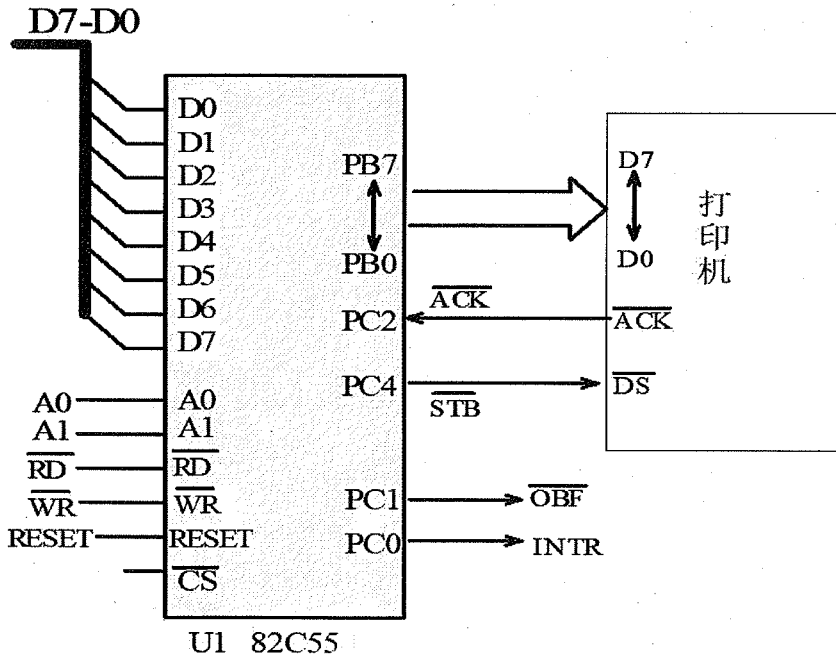
```
KEYS PROC NEAR
    MOV AL,0BFH
    OUT 0C6H,AL ;8255A 初始化
LOOP1:IN AL,0C4H
    TEST AL,20H ;检测 PC5;
    JZ LOOP1 ;缓冲器满进行下一步
    IN AL,0C2H
    MOV BH,AL ;将 AL 中数据传送至 BH 中
```

KEYS ENDP

方式 1 选通输出:连接打印机

打印机的 D7~D0 数据线与端口 B 的 8 根 I/O 线连接;打印机的(/ACK)信号与 8255A 的(/ACK)连接;(/DS)信号是打印机的数据选通信号,8255A 没有数据输出选通信号,因此(/DS)信号接到 PC4 (或 PC5) 上,通过置位复位操作产生数据选通信号。

8255A 的 B 组工作在方式 1,端口 B 作输出,PC2 为(/ACK)信号,输入,低电平有效.PC1 为(/OBF)信号,输出.PC0 为中断请求信号。



编制程序如下:

;控制字:10010001B

MAIN PROC NEAR

START:MOV AX, SEG BUFF

MOV DS, AX

MOV SI, OFFSET BUFF

MOV CX, 2048

MOV DX, 0FBC3H

MOV AL, 91H ;8255A 初始化

OUT DX, AL ; C 口高位方式 0 输出,低位方式 0 输入

MOV AL, 09H;

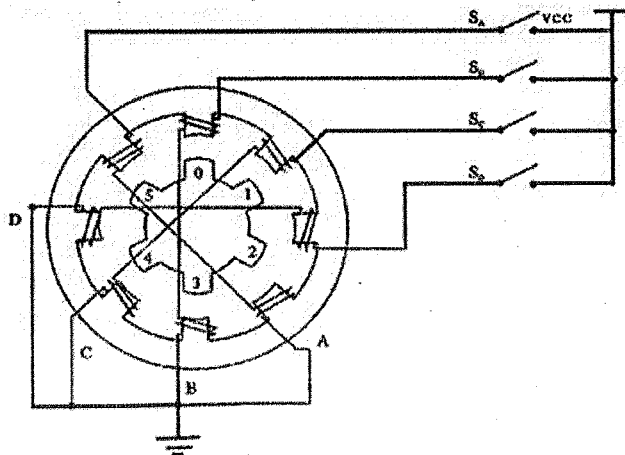
```

OUT DX, AL          ;使 PC4 置位,即使选通无效
    WAIT: MOV DX, 0FBC2H
        IN AL, DX          ;输入端口 B 的状态
TEST AL, 02H        ;检测 PC1 是否为 0
JZ WAIT            ;为 0 则等待
MOV DX, 0FBC1H
    MOV AL,[SI]
OUT DX, AL          ;从 B 口输出数据
MOV DX, 0FBC3H
MOV AL, 08H
OUT DX, AL          ;STROBE 信号为低电平
CALL DELAY          ;延时
MOV AL, 09H
OUT DX, AL          ; STROBE 信号为高电平
INC SI              ;修改指针,指向下一个字符
LOOP WAIT
DONE: MOV AH, 4CH
    INT 21H
    MAIN ENDP

```

类三:步进电机

是一个四相步进电机工作原理示意图,采用单极性直流电源供电,只要对步进电机的各相绕组(线圈)按合适的时序通电,就能使电机转动。步进电机在 360° 内旋转时是不连续的,每当电机绕组接受一个电脉冲,转子就转过一个相应的步距角。



角位移的大小与脉冲数成正比,电机转速与脉冲频率成正比.步进电机的转速应由慢到快逐步加速.电机绕组的通电顺序控制电机的转向,当通电时序为AB—BC—CD—DA时为正转,通电时序为DA—CD—BC—AB时为反转.

电机驱动方式可以采用:

单四拍 (A→B→C→D→A)

双四拍 (AB→BC→CD→DA→AB)

单,双八拍 (A→AB→B→BC→C→CD→D→DA→A)

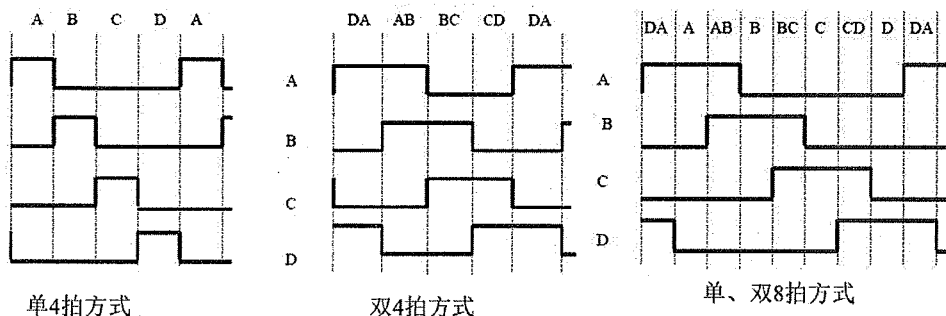
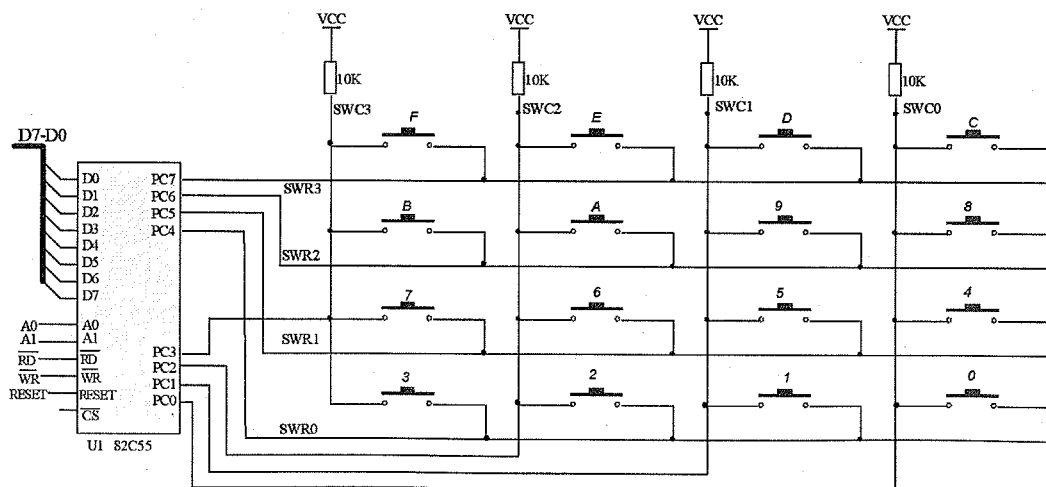


图7-14 步进电机工作方式

; 控制步进电机工作的汇编语言程序:见本章末尾附录

类四:键盘



第1步可能读入的数据:

0FH ;无键按下

0EH ;0列有键按下

0DH ;1列有键按下

0BH ;2列有键按下

07H ;3列有键按下

3	2	1	0
F 77	E 7B	D 7D	C 7E
B B7	A BB	9 BD	8 BE
7 D7	6 DB	5 DD	4 DE
3 E7	2 EB	1 ED	0 EE

行扫描法:

具体步骤:

- 1° PC7~PC4 输出全 0,从 PC3~PC0 读入,若读入的数据不是 0FH,表明有键按下;
- 2° 延时 20ms 去抖动,重复第一步动作,若还有键闭合,则认为确实有键闭合;否则返回第一步;
- 3° 查找被确认的键;从第 0 行开始逐行输出 0,然后读入列值,检查有无为 0 的位;若无,转下一行;
- 4° 将输出的数据与第 1 步输入的数据组成一个字节,查表(键的位置码表),确定键值.

反转法:

- 1° 将 PC7~PC4 设定为输出,PC3~PC0 设定为输入.然后 PC7~PC4 输出全 0,从 PC3~PC0 读入列值,若读入的数据有一位为 0,表明该列有键按下,保存列值;
- 2° 延时 20ms 去抖动,重复第一步动作,若还有键闭合,则认为确实有键闭合;否则返回第一步;
- 3° 将 PC7~PC4 设定为输入,PC3~PC0 设定为输出.把保存的列值从 PC3~PC0 输出,从 PC7~PC4 读入行值,读入的数据必有一位为 0,保存行值.将行值和列值组合在一起,用查表的方法得到按键的键值.

两种方法程序示例见本章末尾附录.

2. 定时器/计数器 8253

定时:

软件定时:采用软件编程来实现定时和计数要求,通用灵活,但占用 CPU 时间.

硬件定时:采用逻辑电路来外接定时部件和电阻,电容来实现.通用性差,若改变对时间和计数的要求需调整硬件.

8253/8254 属于硬件定时.

8254 是 8253 的改进型,它的操作方式以及引脚与 8253 完全相同.改进主要在两个方面:

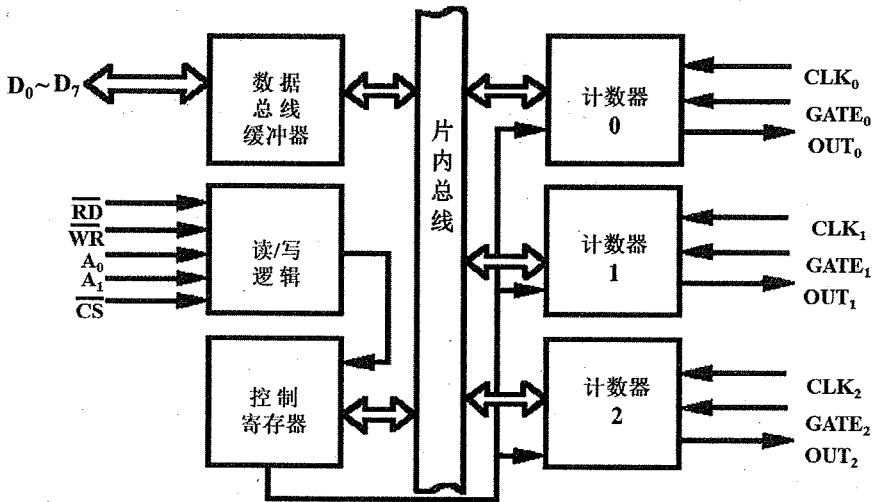
8254 计数时钟频率最高为 10MHz

读回命令字

8254 的每个计数器都有一个状态字也可由读回命令锁存.

(1) 8253 的功能结构

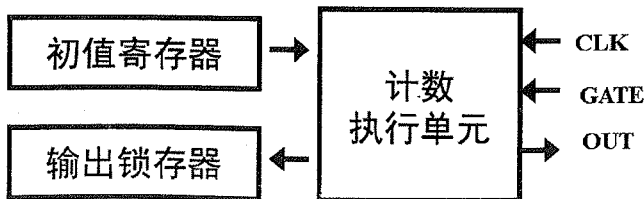
① 功能结构



可编程定时器8253的内部结构图

具有三个功能完全相同的独立的 16 位减法计数器;每个计数器都包括一个 16 位的初值寄存器,一个计数执行单元和一个输出锁存器.

② 工作原理



计数器初始化时,计数初值被写入初值寄存器.当计数执行单元开始工

作时,CLK 端来的第一个脉冲信号下降沿使计数初值从初值寄存器进入到计数执行单元.之后,计数执行单元开始对 CLK 端的输入脉冲进行减 1 计数,减到 0 时,OUT 端输出特定脉冲信号,整个过程可以重复进行.

计数初值也叫时间常数.

③ 引线及功能

D7	1	24	V _{cc}
D6	2	23	WR
D5	3	22	RD
D4	4	21	CS
D3	5	20	A1
D2	6	19	A0
D1	7	18	CLK2
D0	8	17	OUT2
CLKD	9	16	GATE2
OUT0	10	15	CLK1
GATE0	11	14	OUT1
GND	12	13	GATE1

D0~D7 :8 位双向数据线;

CS:芯片选择;

RD,WR:读写控制信号;

A1,A0:地址线;

A1A0=00 计数器 0

A1A0=01 计数器 1

A1A0=10 计数器 2

A1A0=11 控制寄存器

CLK:计数器的时钟信号输入端,最高计数频率 2MHz;

GATE:门控信号;控制计数器的启动和停止;

OUT:计数器输出信号

④ 启动方法

软启动:由程序指令启动,GATE=1.

硬件启动:由外部信号启动.写入初值后并不启动计数,门控信号 GATE 由低变高电平后,在 CLK 的下降沿开始计数.

(2) 8253 的控制字

8253 的有 6 种计数方式,可采用两种数制计数,16 位计数初值可以采用 3 种方法写入,这些都由控制字控制.所以,在使用 8253 之前,必须向 8253 控制寄存器写入控制字.

在计数器工作过程中,当 8253 收到控制字 D5D4=00 时,计数器的当前值就被锁存到 16 位的输出锁存器中,此时计数器照常计数,但输出锁存器的值不再变化,待 CPU 将输出锁存器中的计数值读走后,输出锁存器的内容又开始随着计数器变化.CPU 读取计数值的方法是:先读写低 8 位,再读写高 8 位.

二进制计数的计数范围 0000~65536,0 为最大计数初值,代表 65536;

十进制计数的计数范围 0000~10000,0 为最大计数初值,代表 10000.

十进制计数的计数初值以 BCD 码的形式写入计数器.

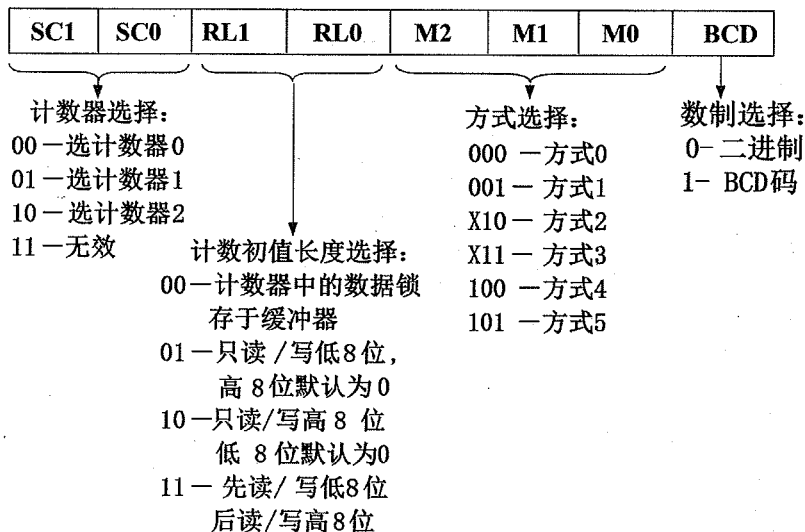


图7-3 8253 控制字

计数器初始化:写入控制字与时间常数.注意,在计数器端口写入初值时,要先写入低 8 位,再写入高 8 位.

例 6:设 8253 的端口地址为:40H~43H,设置计数器 0 工作在方式 0,二进制计数,计数初值为 1000.设置计数器 1 工作在方式 2,BCD 计数,计数初值 800.

答:

计数器 0 的控制字为:00110000B

计数器 1 的控制字为:01110101B

;初始化计数器 0

```
MOV AL, 30H      ; 计数器 0,方式 0,二进制计数
OUT 43H, AL
```

```

MOV  AX, 1000    ; 写入计数初值.
OUT   40H, AL     ; 写入低八位
MOV   AL, AH
OUT   40H, AL     ; 写入高八位

```

;注意,在计数器端口写入初值时,要先写入低 8 位,再写入高 8 位
;初始化计数器 1

```

MOV   AL, 75H     ; 计数器,方式 2,BCD 计数
OUT   43H, AL
MOV   AX, 800H    ; 写入计数初值 (BCD 数).
OUT   41H, AL     ; 写入低八位
MOV   AL, AH
OUT   41H, AL     ; 写入高八位

```

(3) 8253 的工作方式

8253 计数器有 6 种工作方式.计数器启动计数,可以通过 CPU 执行输出指令启动,称为软启动;也可以由 GATE 端的外部信号的上升沿触发,称为硬启动.方式 0,2,3,4 采用软启动,1,2,3,5 采用硬启动.方式 2,3 两种启动方式都可用.

共同特点:

写入控制字后,OUT 端进入初始状态.方式 0 为低,其他方式为高电平.计数初值写入计数器之后,暂存在初值寄存器中,要经过一个时钟脉冲后,计数初值才进入计数执行单元,开始计数.

在时钟脉冲的下降沿,计数器减 1.

分频系数/时间常数 N:计数初值

在 OUT 端连续输出一个宽度为 TCLK 的负脉冲信号,其脉冲周期为 $T_0 = N * TCLK$ (高电平: $(N-1) TCLK$,低电平: $TCLK$);脉冲频率: $F_0 = F_c / N$;分频系数 $N = F_c / F_0$ (主要运用于方式 2,3)

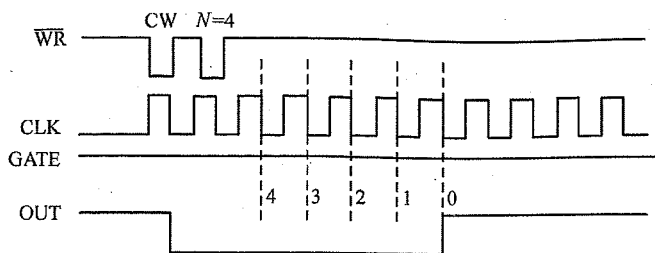
① 方式 0:计数结束中断

软件启动,不自动重复计数方式;计数结束后,OUT 端输出为高电平,可以作为中断请求信号使用;

只计数一个周期;重新写入计数初值后开始新的计数周期.

在计数过程中 GATE 变为低电平,则暂停计数;

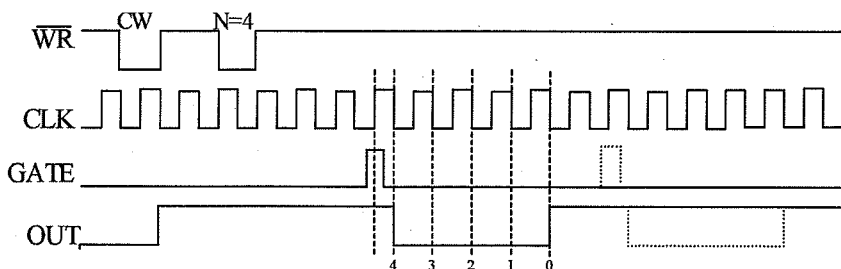
在计数过程中,可以随时修改计数初值;



② 方式 1:可重复触发的单稳态触发器

硬件启动,不自动重复的工作方式;

当写入控制字后,OUT 端输出高电平.写入计数初值后计数器并不开始工作;当 GATE 端由低到高触发后,在下一个 CLK 脉冲的下降沿开始计数,OUT 端立刻由高到低,计数结束后 OUT 端输出高电平;



GATE 的上升沿触发计数器,计数过程中 GATE 即使变低也不影响计数;

可重复触发;

OUT 端的负脉冲宽度=计数初值 $N \times T_{CLK}$

例 7 如果 $CLK=2MHz$,时间常数 $N=1000$,则

OUT 端的负脉冲宽度为

答: $f_c = 2MHz$

$T_c = 1/f_c = 0.5 \times 10^{-6}$,

OUT 端负脉冲宽度= $N \times T_c = 1000 \times 0.5 \times 10^{-6} = 500us$

③ 方式 2:频率发生器

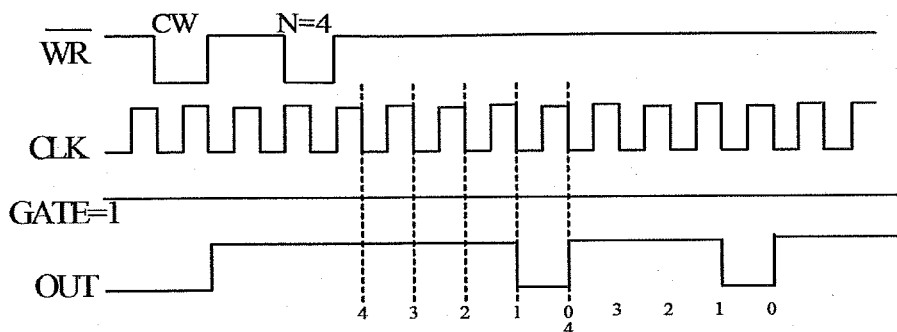
也叫分频器;可以软件启动,也可以硬件启动,计数器自动重复工作.

在写入控制字和计数初值期间 GATE 一直为高电平,则在下一个 CLK 开始计数,既软启动;若写入计数初值时 GATE 为低电平,则要等到 GATE 由低到高时才启动,即硬件启动;

一旦计数器启动,计数器可以自动重复工作.

写入控制字后,OUT 为高电平.如果 GATE 为高,写入初值后开始减 1

计数.当计数值减到 1 时,OUT 输出变为低电平,再经过一个 CLK 之后计数值减为 0,OUT 变为高电平,接着自动重新装入初值,计数器重复工作.



例 8 设 8253 的端口地址为:40H~43H,用计数器 2 作频率发生器,二进制计数,输入脉冲信号为 2M,计数初值为 800,输出脉冲信号的频率是多少? 写出 8253 初始化程序.

答:

控制字为:10110100B=0B4H

输出脉冲信号的频率为: $F_{out}=F_{CLK}/N=2\times 10^6/800=2500\text{Hz}$

初始化程序:

```
MOV AL, 0B4H
```

```
OUT 43H, AL ;控制字送入控制寄存器
```

```
MOV AX, 800
```

```
OUT 42H, AL ;计数初值低 8 位送入计数器 2
```

```
MOV AL, AH
```

```
OUT 42H, AL ;计数初值高 8 位送入计数器 2,计数器启动.
```

④ 方式 3

可以软启动,也可以硬件启动,计数器自动重复工作;

写入控制字后,OUT 为高电平.如果 GATE 为高,写入初值后开始减 1 计数.当计数值减到 $N/2$ 时,OUT 输出变为低电平,直到计数值减为 0;接着自动重新装入初值,OUT 变为高电平,计数器自动重复工作.

OUT 输出对称方波 (N 为偶数) 宽度: $N/2$

若计数初值为奇数, OUT 输出非对称方波:

GATE 由高变低,停止计数.

例 9:设 8253 的端口地址为:FCF8H~FCFFH,利用计数器 1 作方波发

生器,给定 CLK1 为 2MHz,要求产生频率为 1KHz 的方波,BCD 码计数.编程初始化 8253.

答:计数器 1 工作在方式 3:01110111B

分频系数: $N = FCLK / FOUT = 2 \times 10^6 / 1000 = 2000$

程序设计如下:

```
MOV DX, 0FCFBH
```

```
MOV AL, 01110111B
```

```
OUT DX, AL
```

```
MOV DX, 0FCF9H
```

```
MOV AX, 2000H ;BCD 码计数初值
```

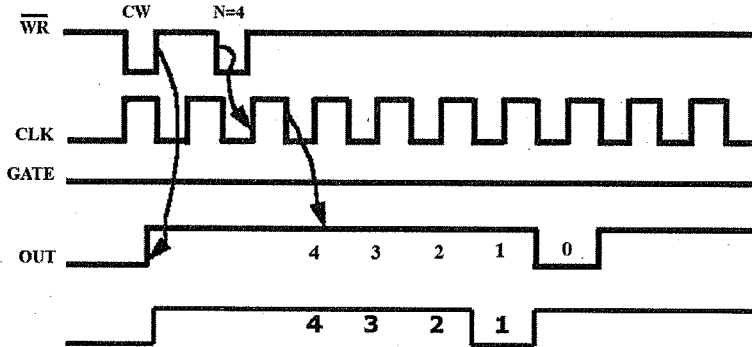
```
OUT DX, AL ;写入计数初值低字节
```

```
MOV AL, AH
```

```
OUT DX, AL ;写入计数初值高字节,计数器启动
```

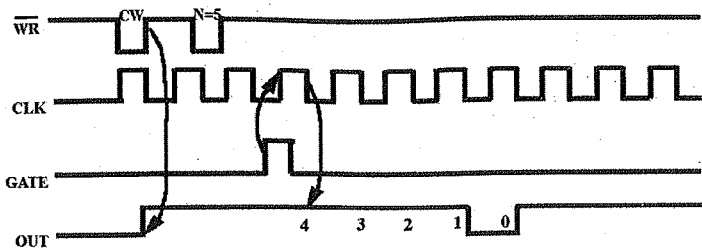
⑤ 方式 4:软件触发选通

软件启动,不自动重复计数方式;写入控制字后,OUT 为高电平,计数器装入初值后即启动计数,计数结束时 OUT 输出宽度为 TCLK 的负脉冲, GATE 为低,停止计数.



⑥ 方式 5:硬件触发选通

硬件启动,不自动重复的工作方式. GATE 的上升沿触发计数器;计数结束时,在 OUT 端输出一个脉冲宽度为 TCLK 的负脉冲信号: GATE 的上升沿,将重新计数.



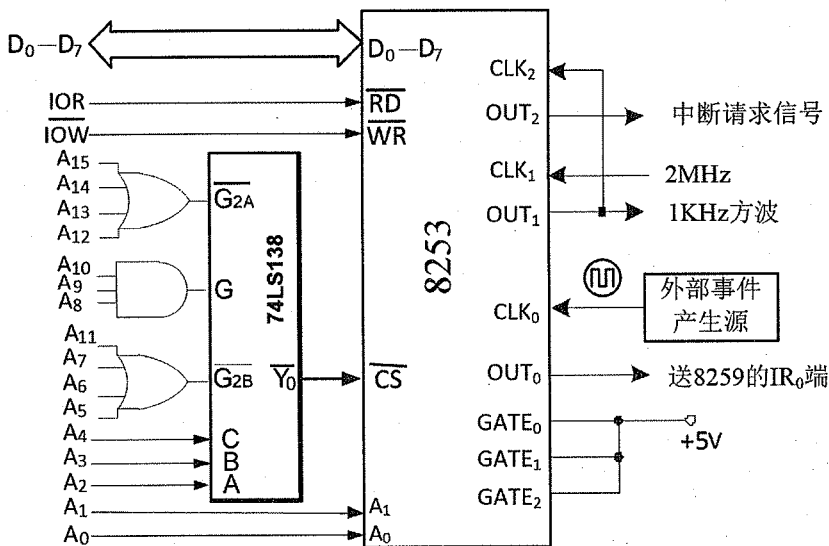
(4) 8253 实例

例 9 8253 与计算机系统的连接如图所示,三个计数器使用情况如下:

CNT0:对外部事件计数,计满 100 次向 CPU 发出中断请求。

CNT1:产生 1kHz 方波。

CNT2:标准定时,定时时间为 1s。



分析:

计数器 0 应设置为方式 0,完成计数功能。

计数器 1 应设置为方式 3,CLK1 输入的时钟脉冲频率为 2MHz,输出频率为 1kHz 的方波,周期为 1ms;

计数器 1 的计数初值为 2000。

计数器 2 设置为方式 0,完成定时功能,定时 1 秒,利用 OUT2 向 CPU 发出一次中断请求,其输入时钟频率为 1kHz,计数初值应为 1000。

8253 的端口地址为 01FC~01FFH

程序:

```
START: MOV DX, 01FFH
```

```

MOV AL, 10H
OUT DX, AL           ;计数器 0 工作在方式 0
MOV DX, 01FCH
MOV AL, 100
OUT DX, AL           ;计数器 0 置初值
MOV DX, 01FFH
MOV AL, 76H
OUT DX, AL           ;计数器 1 工作在方式 3
MOV DX, 01FDH
MOV AX, 2000
OUT DX, AL
MOV AL, AH
OUT DX, AL           ;计数器 1 置初值
MOV DX, 01FFH
MOV AL, 0B0H
OUT DX, AL           ;计数器 2 工作在方式 0
MOV DX, 01FEH
MOV AX, 1000
OUT DX, AL
MOV AL, AH
OUT DX, AL           ;计数器 2 置初值

```

例 10 8254 工作室的 I/O 端口地址为 0300H,0301H,0302H,0303H,写一个程序编程计数器 2,使其在 CLK 输入为 4MHz 时产生一个 20kHz 的方波.

答:

分析:

计数器 2 工作在方式 3:10110110

初值 $N=4*10^6/20\ 000=200$

程序:

```

MOV DX,0303H
MOV AL,0B6H
OUT DX,AL ;设定计数器 2 工作在方式 3
DEC DX
MOV AX, 200
OUT DX, AL

```

```
MOV AL, AH
OUT DX, AL ;计数器 2 置初值
```

附录:

; 控制步进电机工作的汇编语言程序

```
mode equ 082h
PortA equ 08000h
contrl equ 08003h
```

```
Astep equ 01h ;A 项
Bstep equ 02h ;B 项
Cstep equ 04h ;C 项
Dstep equ 08h ;D 项
```

```
data segment
dly_c dw 0 ;延时常数
data ends
```

```
code segment
assume cs:code, ds:data
```

```
start proc near
mov ax, data
mov ds, ax
mov dx, contrl ;8255A 控制口地址
mov al, mode ;8255A 方式控制字 82H
out dx, al ;8255A 初始化
mov dx, PortA ;端口 A 地址
mov al, 0
out dx, al ;端口 A 输出 0, 步进电机停止转动.
mov dly_c, 1000h
jmp step4 ;或者 step8, step41
```

; 双四拍工作方式

step4:

```
mov dx, PortA
mov al, Astep + Bstep
out dx, al
```

```

    call delay
    mov al,Bstep+Cstep
    out dx,al
    call delay
    mov al,Cstep+Dstep
    out dx,al
    call delay
    mov al,Dstep+Astep
    out dx,al
    call delay
    mov ax, dly_c
    dec ah          ;提高转速
    cmp ax, 300h ;最快速度
    jne nn3
    inc ah          ;恒速运转
nn3: mov dly_c,ax
      jmp step4
start endp

```

;行扫描法键盘编程

```

COLS EQU 4
ROWS EQU 4
PORTC EQU 52H

```

;获取按键的位置码子程序 getkey

```

GETKEY PROC NEAR USES CX BX AX
    MOV AL, 81H ;端口 C 高 4 位输出,低 4 位输入
    OUT 53H, AL
X1: MOV AL,0
    OUT 52H, AL ;行输出 0000
    IN AL, 52H ;输入低 4 位
    AND AL, 0FH ;提取低 4 位
    CMP AL, 0FH
    JZ X1 ;没键按下,继续查询
    CALL DELAY20ms ;有键按下,延时 20ms 去抖动
;去抖后,重新读取列的值,如果没有键按下,则刚才是干扰,返回 X1 继

```

续查询.否则,逐行扫描.

```
MOV AL,0
OUT 52H, AL ;行输出 0000
IN AL, 52H ;输入低 4 位
AND AL, 0FH ;提取低 4 位
CMP AL, 0FH
JZ X1 ;没键按下转移 X1,刚才是干扰
MOV BL, AL
```

;确认有键按下,逐行扫描

```
MOV CX, ROWS ;设置扫描行数:4
MOV AH, 0FEH
X2: MOV AL, AH ;保存初始行值
ROL AL,4
OUT 52H, AL ;扫描第 0 行 (PC4)
IN AL, 52H
AND AL, 0FH
CMP AL, 0FH
JNZX3 ;找到按键所在的行了,保存行值
ROL AH,1
LOOP X2
MOV AH, 80H ;4 行都扫描完,没有键按下,80H 作为标志.
JMP XEND
```

;AH 的高 4 位是键的行码,AL 中的低 4 位是键的列码

```
X3: AND AH,0FH
SHL AH,4
OR BL, AH ;行码列码组成一个字节
```

```
XEND: NOP
RET ;返回 AL,位置码
```

```
GETKEY ENDP
```

;查表确定键值子程序 KEY-NUM,入口参数 AL

```
KEY-NUM PROC NEAR USES AX BX CX
MOV CL,0
MOV BX,OFFSET TABLE
X4: CMP AL,[BX]
JZ X5
```

```

INC    CL
INC    BX
CMP    CL, 16
JNZX4
MOV     AH, 0FFH    ;没有在表中找到对应的键值,可能出现
;多个键被同时按下的情况,FFH 作为标志.
JMP     XEND0
X5:    MOV  AH, CL
XEND0:NOP
RET
KEY—NUM    ENDP

```

;反转法键盘扫描

```

START:  MOV AL, 10000001B;方式 0,C 口高 4 位输出,低 4 位输入
        OUT 43H,AL
        MOV AL, 0
        OUT 42H,AL      ;C 口高 4 位输出 0
WAIT1:  IN  AL, 42H      ;取低 4 位
        AND AL, 0FH
        CMP AL, 0FH
        JZ  WAIT1
        MOV AH, AL
        MOV AL, 10001000B;方式 0,C 口高 4 位输入,低 4 位输出
        OUT 43H,AL
        MOV AL, AH
        OUT 42H,AL      ;把列值输出
        IN  AL, 42H ;输入行值
        AND AL, 0F0H ;取高 4 位
        OR  AL, AH
        ... ;查表求出按键的键号

```

第八章 数/模转换及模/数转换技术

一、本章考点

1. 小题

- (1) 如果参考电压是 -2.55V ,那么 DAC0832 输出的阶梯电压是多少?
- (2) ADC080X 需要大约多长时间将一个模拟电压转换为数字代码?
- (3) DAC8032 在大约多长时间内将一个 8 位数字输入转换成一个模拟输出?

答:

- (1) 0.01V

DAC0832 是 8 位数据输入, $n=8$,阶梯电压 $=2.55/(2^8 - 1) = 0.01\text{V}$

- (2) $100\mu\text{s}$

- (3) $0.1\mu\text{s}$

2. 大题:考察一道大题

范围:

- (1) 1.4 例 1~例 3 (如果出题基本出原题),例 4 (相似类型题)
- (2) 2.4 例 5

二、基础知识

模 / 数 (Analog to Digital, A / D) 转换是将输入的模拟量转换为数字量,将实现 A / D 转换的电路称为 A / D 转换器 (简称 ADC)。

数/模 (Digital to Analog, D / A) 转换是将计算机处理的数字量转换为能够驱动模拟执行机构的模拟量.将实现 D / A 转换的电路称为 D / A 转换器 (简称 DAC)。

1. 数模转换

(1) 数模转换基本原理

- 数字量由二进制位组成,每个二进制位的权为 2^i ,只要将这些位按权大小转换成相应的模拟量,然后根据叠加原理将这些模拟量相加,总和就是与数字量成正比的模拟量.通常 D / A 转换器将数字量转换为电流,再经运算放大器将电流转换为电压 (一般转换后的模拟信号都是以电压的形式输出)。
- T 型电阻网络:
T 型电阻网由三部分组成:

- ① 由二进制数控制的开关组;
- ② $R-2R$ 电阻网络;
- ③ 由运算放大器构成的电流—电压转换电路.

工作原理:

当开关 K_0, K_1, K_2, K_3 都倒向左边, 相当于输入二进制数 0000 时, 运算放大器得到的输入电流为 0.

当开关 K_0, K_1, K_2, K_3 都倒向右边, 相当于输入二进制数 1111 时, 运算放大器得到的最大输入电流.

运算放大器的输出电压为:

$$V_{OUT} = -I \times R_0 = -\frac{V_{REF}}{2R} \times R_0 \left(\frac{1}{2^0} + \frac{1}{2^1} + \frac{1}{2^2} + \frac{1}{2^3} \right)$$

由此可以看出, 电阻网络会有 16 种不同电流流入运算放大器, 相应的运算放大器输出 16 种不同的电压, 从而将数字量的变化转换成模拟量的变化.

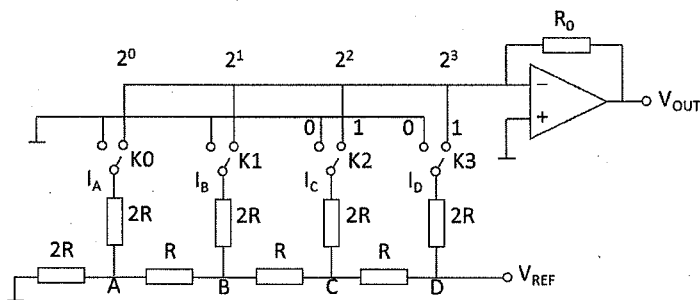


图8-2 T型电阻网络D/A转换器

- D/A 转换器的转换精度与基准电压 V_{ref} 和权电阻的精度以及数字量的位数有关, 所以要求 V_{ref} 为一个足够精度的标准电压, 同时数字量的位数越多, 转换的精度就越高.

(2) 数模转换参数

- ① 分辨率: 也称阶梯电压, 输入数字量的最低有效位发生变化时, 所对应的输出模拟量的变化量, 反映了输出模拟量的最小变化值.
分辨率与输入数字量的位数有关, 分辨率 $= FS / (2^n - 1)$, FS 是满量程输入值, 及参考电压 $-V_{ref}$.
- ② 转换精度: 分为绝对精度与相对精度.
- ③ 建立时间: 也叫转换时间, 指输入数字量后, 输出的模拟量稳定到相应数值范围内所经历的时间, 是描述 D/A 转换速率的一个重要参数.

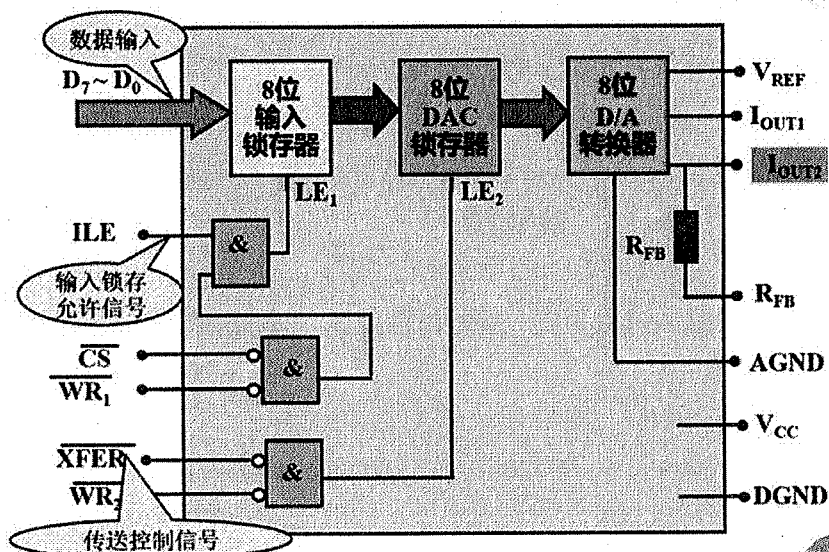
实际建立时间的长短不仅与转换速率有关,还与数字量变化的大小有关.输入数字由全 0 到全 1,建立时间最长,成为满量程的建立时间.

- ④ 线性误差: D/A 转换器理想特性趋于一条直线,实际有误差,模拟量实际值与理想值之间的最大差值称之为线性误差.

(3) DAC0832 数模转换器

DAC0832 为电流输出.逻辑电平输入与 TTL 兼容.它在 1.0us 内将一个数字信号转换为模拟电压.

① 内部结构



该器件包含 2 个锁存器,第一级 8 位数据输入锁存器,第二级 8 位 DAC 寄存器. D/A 转换器的输出不能被禁止,需要在系统初始化时设置安全状态.

② DAC0832 接口电路

- 1° 直通工作方式: 两个寄存器均直通放行,数据可以从输入端直接进入 D/A 转换器.此时 DAC0832 不能与 CPU 的数据总线相连,因为这种方式不使用缓冲器,可以接一片 8255 缓冲.
- 2° 单缓冲工作方式——1 个直通,1 个受控
这种方式是使两个寄存器(输入寄存器和 DAC 寄存器)之一始终处于直通,即 $\text{LE}_1 = 1$ 或 $\text{LE}_2 = 1$,另外一个寄存器处于受控状态.此时只需一次写操作,就开始转换,可以提高 D/A 的数据吞吐量.在不要求多路 D/A 同时输出时,可以采用单缓冲方式.
- 3° 双缓冲工作方式——2 个都受控

适合于多路模拟信号同时输出的应用场合.此时 COU 对 DAC0832 进行两步操作:将数据写入输入寄存器;将寄存器内容写入 DAC 寄存器.这样输入接收与启动转换可以分步进行.

(4) DAC0832 实例

DAC0832 单缓冲方式应用非常广泛,不仅可以对现场执行机构进行控制,还可以用于产生多种周期可调,幅值可变的智能信号(如锯齿波,三角波,方波,正弦波等),可做为信号源使用.假设 DAC0832 的端口地址为 200H.

例 1 锯齿波:

```
MOV DX,200H ;端口地址送至 DX
MOV AL,0 ;赋初值
NEXT1: OUT DX,AL ;输出数字量送至 D/A 转换器
      INC AL ;数字量加 1
      CALL DELAY ;调用延时子程序
      JMP NEXT1 ;循环,到最大值 255 再+1 后自动复位为 0
```

例 2 三角波

```
MOV DX,200H ;端口地址送至 DX
NEXT0: MOV CX,0FFH ;CX 赋循环次数
      MOV AL,0 ;赋初值
NEXT1: OUT DX,AL ;输出数字量送至 D/A 转换器
      INC AL ;数字量加 1
      CALL DELAY ;调用延时子程序
      LOOP NEXT1 ;循环
      MOV CX,0FFH
NEXT2: DEC AL ;数字量减 1
      OUT DX,AL
      CALL DELAY
      LOOP NEXT2
      JMP NEXT0
```

例 3 方波

```
MOV DX,200H ;端口地址送至 DX
NEXT: MOV AL,0 ;赋初值(低电平)
      OUT DX,AL ;输出数字量送至 D/A 转换器
      CALL DELAY ;调用延时子程序
```

```

MOV AL,0FFH ;赋初值（高电平）
OUT DX,AL
CALL DELAY
JMP NEXT

```

例 4 DAC0832 常见连接

利用 DAC0832 输出单极性模拟量电压。

将从 2000H 开始的 50 个字节单元数据依次送到 DAC0832 输出,每个数据输出间隔时间为 1ms,可调用 D1ms 延时 1ms 子程序。

答:读图得端口地址 80H~83H

输出程序编制如下:

```

X1:  MOV SI,2000H
      MOV CX,50
X2:  MOV AL,[SI]
      INC SI
      OUT 80H,AL
      CALL D1 ms
      LOOP X2
      HLT

```

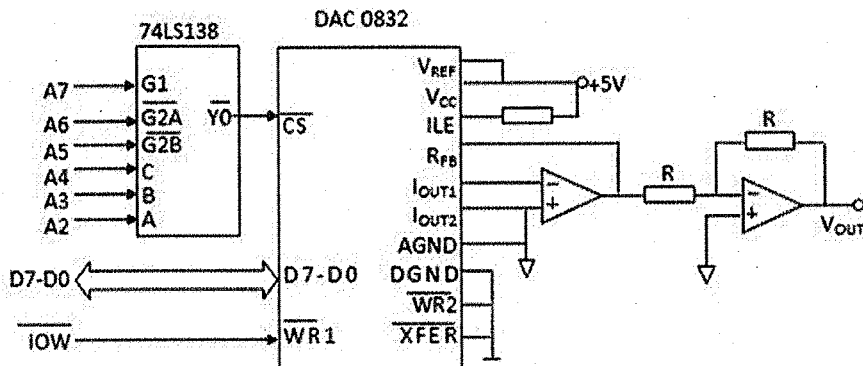


图 8-7 DAC 0832 单缓冲接口

2. 模数转换

(1) 模数转换原理简介

A/D 转换方法:计数式,逐次逼近式,双积分式.计数式简单,速度低;逐次逼近式速度高,价格合适;双积分式转换精度高,速度低。

A/D 转换基本过程:

模拟量在时间上与幅值上都连续.把模拟信号转为数字信号,首先进行采样.采样过程:使用采集电路等距时间间隔采集模拟信号,用保持电路将采集来的信号电平保持一段时间,便于数字转换。

采样后的信号是在时间上离散,幅值上连续的离散信号,需要在幅值上离散化计算机才能处理.这一过程称之为量化.所有采样值量化后化为对应的数字量,成为了整数值.

量化过程中会有误差,是固有误差,最大偏差为量化单位的一半.

量化后的信号在时间与幅值上都离散.

(2) 模数转换参数

- ① 分辨率:指引起输出的二进制数字量最低有效位变动一个数码后,输入的模拟量最小变化量.

分辨率与输入数字量的位数有关,分辨率= $FS/(2^n-1)$,FS 是满量程输入值.

- ② 转换精度:数字量的实际输出一理论输出

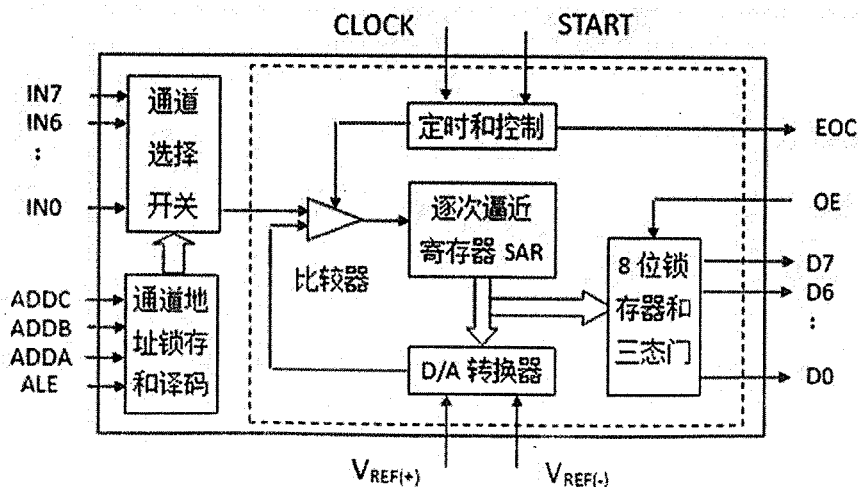
- ③ 转换时间:完成一次转换所用时间,指发出转换控制信号开始,直到输出端得到稳定的数字量输出所用时间,是微秒级.转换时间的倒数是转换速率.

(3) ADC0809 简介

ADC0809 是 8 路输入单片 A/D 转换器,可以与 CPU 总线直接连接.输出是可锁存三态输出,与 TTL 兼容.他在 100us 内将一个模拟信号转换为数字信号.

ADC0809 的模拟量输入是单极性的,范围为 $0 \sim +5V$,若实际输入模拟信号是双极性的,比如为 $-5V \sim +5V$,则需要设计极性转换电路

④ 内部结构



ADC 0809 是一种 CMOS 单片 A/D 转换器,由三个部分组成,即 8 路模拟开关及地址锁存与译码,8 位 A/D 转换器,还有三态输出锁存缓冲器.

ADC 0809 内部没有模拟输入信号采样保持器,处理快速信号时应外加。

⑤ 引脚功能

IN7—IN0:8 路模拟量输入线。

D7—D0:8 位数字量输出线。

ADDC—ADDA:3 位地址线,用来选通 8 路模拟通道中的一路。

ALE:地址锁存允许,在 ALE 的上升沿,ADDC,ADDB,ADDA 三位地址信号被锁存到地址锁存器。

START:启动信号,正脉冲有效。

EOC:转换结束信号,高电平有效,输出。

OE:输出允许,当 OE 为高电平时打开输出三态缓冲器,使转换后的数据进入数据总线。

(4) ADC 0809 与 CPU 的接口方法

A/D 转换器与 CPU 的数据传送控制方式通常有 3 种:等待方式,查询方式,中断方式。

① 等待方式

等待方式又称定时采样方式,或无条件传送方式,这种方式是在向 A/D 转换器发出启动指令(脉冲)后,进行软件延时(等待),此延时时间取决于 A/D 转换器完成 A/D 转换所需要的时间,经过延时后才可读入 A/D 转换数据,速度较慢。

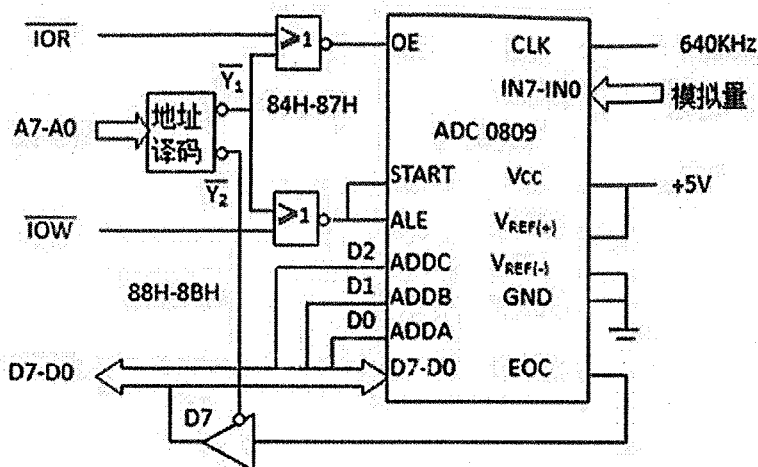
② 查询方式

所谓程序查询方式,就是先选通模拟量输入通道,发出启动 A/D 转换的信号,然后用程序查看 EOC 状态,若 EOC=1,则表示 A/D 转换已结束,可以读入数据;若 EOC=0,则说明 A/D 转换器正在转换过程中,应继续查询,直到 EOC=1 为止。

比等待方式速度快,但实时性差。

例 5 ADC 0809 查询方式接口设计

ADC 0809 与微处理器之间的查询方式连接如图 8-12 所示,利用该接口电路,采用查询方式,对现场 8 路模拟量输入信号循环采集一次,其数据存入数据缓冲区中,进行程序设计



```

DATA    SEGMENT
COUNT  DB    00H                ;采样次数
NUMBER  DB    00H                ;通道号
ADCBUF  DB    8 DUP(?)           ;采样数据缓冲区
DATA    ENDS
ADCC    EQU    84H                ;A/D 控制口地址
ADCS    EQU    88H                ;A/D 状态口地址
CODE    SEGMENT
        ASSUME CS:CODE,DS:DATA
START:  MOV    AX,DATA
        MOV    DS,AX
MOV     BX,OFFSET ADCBUF ;设置 A/D 缓冲区
MOV     CL,COUNT         ;设置采样次数
MOV     DL,NUMBER        ;设置通道号

```

```

X3:      MOV    AL,DL
OUT     ADCC,AL          ;启动 ADC 0809 相应通道
X1:      IN     AL,ADCS    ;读取状态口
TEST    AL,80H           ;析取 EOC
JNZ     X1               ;EOC≠0,ADC 0809 未开始转换,等待
X2:      IN     AL,ADCS
TEST    AL,80H

```

```
JZ    X2                ;EOC≠1,ADC 0809 未转换完成,等待
IN     AL,ADCC           ;读数据
MOV    [BX],AL
INC    BX                ;指向下一个数据缓冲单元
INC    DL                ;指向下一个通道
INC    CL                ;采样次数加 1
```

```
CMP    CL,08H
JNZ    X3
MOV     AX,4C00H
INT     21H
CODE    ENDS
END START
```

③ 中断方式

在这种方式中,CPU 启动 A/D 转换后,即可转而处理其它事情,一旦 A/D 转换结束,则由 A/D 转换器发出转换结束信号,这一信号作为中断请求信号发给 CPU,CPU 响应中断后,便读入数据。

第二部分 模拟题

微机原理模拟试题 1

一、简答题（每小题 5 分,共 40 分）

1. 求 -126 的原码,反码及补码.
2. 8088CPU 的一个标准总线周期是多少个时钟周期? 8088CPU 的标准工作频率是 5 MHz,它的一个标准总线周期需要多少纳秒?
3. 8086/8088CPU 为什么对存储器采用分段管理? 分段管理的方法是什么?
4. 假设 $DS = 2000H$, $SS = 3000H$, $BP = 100H$, $SI = 5$,请确定下列指令中存储器操作数的物理地址.
`mov [bp+si], ax`
`mov dx, [si]`
5. 在内存中,从 Table 单元开始,用伪指令定义 0~15 的平方表.
6. 从键盘输入单个字符可以使用 1 号 DOS 功能调用,请写出调用的指令段.
7. 8088/8086CPU 的 I/O 系统采用独立编址方式,它的优缺点是什么?
8. 请从中断请求输入引脚,触发信号,是否受 IF 标志位控制的角度简述 8088/8086CPU 对于存储器奇偶错,电源掉电等紧急事件如何处理? 对一般的外部中断请求如何处理?

二、完成下面程序（10 分）

在数据段中,从 buff 单元开始存放着 10 个字节类型的无符号数,为找出其中最小的数并存入 min 单元.编程如下,请将程序补充完整.

```
data segment
buff db 35h,12h,23h,08h,65h,51h,42h,89h,73h,81h
min db ?
data ends
code segment
assume cs:code, ds:data
start:  mov ax, data
        mov ds, ax
        mov cx, 9      ;设置循环次数
                        ;数据首地址送 si
```

```

;取第一个数
;调整指针指向第二个数
;与第二个数比较大小
;若 al<[si],则跳转 next 处
;否则替换 al
next:      ;修改地址指针
          loop  con      ;测试循环条件 cx 是否为零?
          mov  min, al
          mov  ah, 4ch
          int  21h
code ends
end start

```

三、存储器应用 (10 分)

如图 1 所示,由 2 片静态随机存储器芯片 62256 (32Kx8) 构成存储器,74LS138 的 $\overline{Y2}$ 作为芯片 U1 的片选信号, $\overline{Y3}$ 作为芯片 U2 的片选信号,请给出芯片的地址范围.

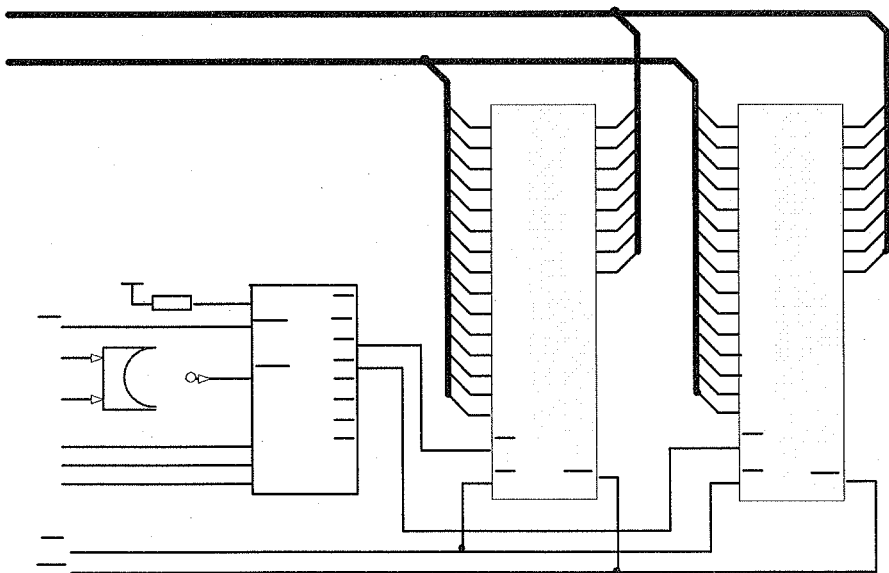


图 1

四、8253 应用 (10 分)

某系统中 8253 的连接如图 2 所示. 编程控制计数器 1,使其在 CLK1 输入脉

冲频率为1 MHz时,输出1000 Hz 的方波信号.请说明 8253 的端口地址,计数器 1 的控制字,时间常数,并将程序补充完整.

```
code segment
    assume cs:code ,ds:data
start:

code ends
end start
```

图 2

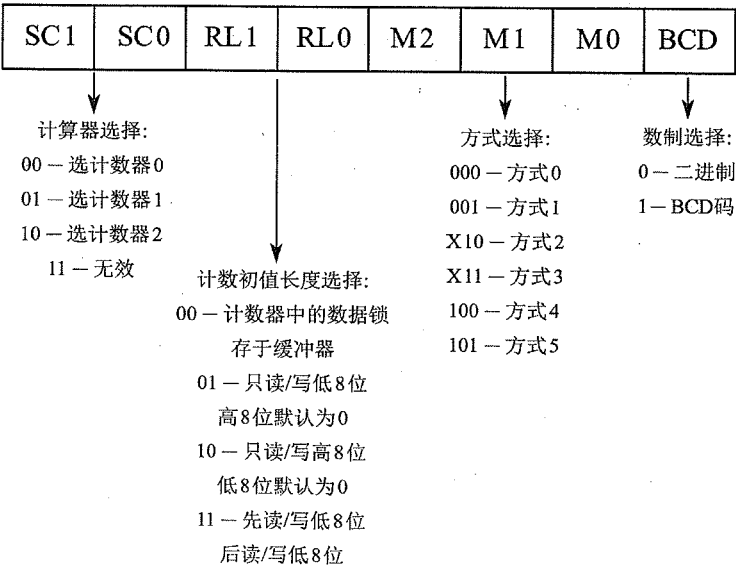
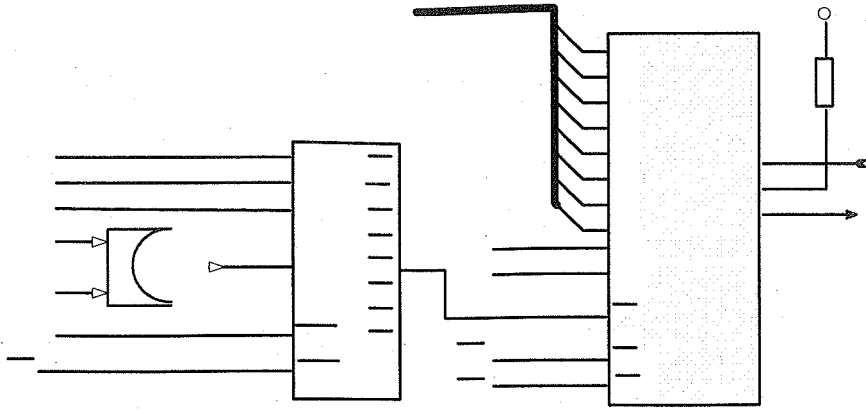


图 3



五、8255 应用 (20 分)

设 8255 的端口地址为 8000H~8003H, 设置 8255 工作在方式 0, 利用它的端口 A 控制 4 相步进电机, 采用双四拍方式驱动电机 (AB—BC—CD—DA—A B), 请说明 8255 的方式控制字是什么, 并将程序补充完整.

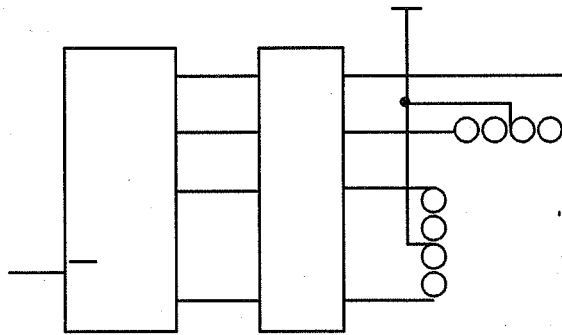


图 4

code segment

assume cs:code, ds:data

start proc near

} ;设置 8255 工作方式

```
    mov dx, 8000H
    mov al, 0
    out dx, al
s1: mov dx, 8000H
    mov al, 3
    out dx, al
    call delay
```

} ;控制电机运转

```
    jmp s1
start endp
```

```
delay proc near
    push cx
    mov cx, 1000h
s2: loop s2
    pop cx
    ret
```

```
    delay endp
code ends
end start
```

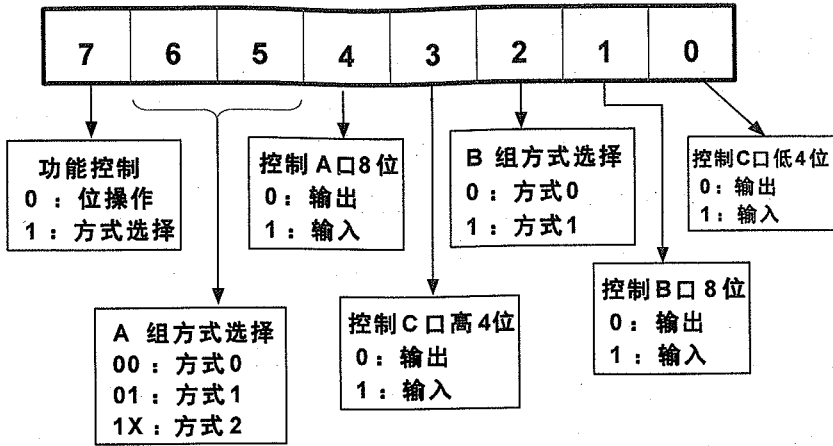


图 5 8255 方式控制字格

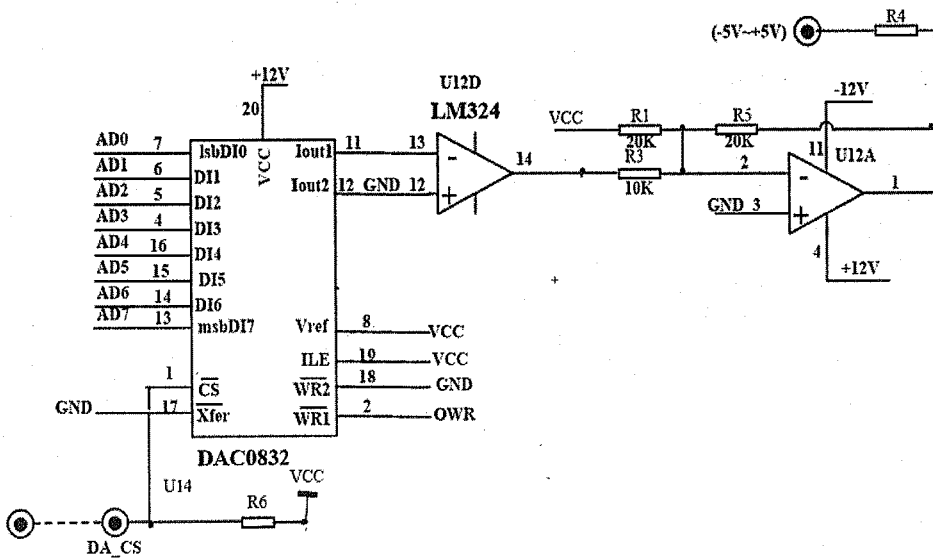


图 6 0832 应用

六、数/模转换 (10 分)

数模转换器 DAC0832 以单缓冲方式工作,端口地址为 A000H,下面程序是产生三角波的

控制程序,请补充完整.

```
cs0832 equ 0a000h
```

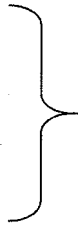
```
code segment
```

```
assume cs:code
```

```
start proc near
```

```
mov dx,cs0832
```

```
next0: mov cx,0ffh ;赋循环次数
```



```
mov cx,0ffh
```

```
next2: dec al
```

```
out dx,al
```

```
call delay
```

```
loop next2
```

```
jmp next0
```

```
start endp
```

```
delay proc near
```

```
push cx
```

```
mov cx,0fh
```

```
loop $
```

```
pop cx
```

```
ret
```

```
delay endp
```

```
code ends
```

```
end start
```

模拟试题 1 参考答案

一、简答题（每小题 5 分,共 40 分）

1. (5 分) 答:FEH, 81H, 82H

2. (5 分) 答:4, 800ns

3. (5 分) 答:8088CPU 内部寄存器的宽度为 16 位,单个寄存器不能寻址 1M 的地址空间,所以它采用了分段寻址的管理方法,使用两个寄存器表示物理地址。(2 分)

8088CPU 将 1M 的内存空间划分成一些连续的区域,称为段.每段的最大长度为 64K,并要求段的起始地址必须能被 16 整除.段的起始地址的高 16 位称为段基址,存于段寄存器中.段内各存储单元地址相对于段起始地址的位移量,称为段内偏移量。(3 分)

物理地址=段基址 X10H+段内偏移量.

4. (5 分) 答:30105H, 20005H

5. (5 分) 答:table db 0,1,4,9,16,25,36,49,64,81,100,121,144,169,196,225

6. (5 分) 答:mov ah,1

int 21h

7. (5 分) 答:优点:不占用内存空间,接口的地址译码电路简单.缺点:需要输入输出专用的控制信号和专用指令.

8. (5 分) 答:中断处理.采用不可屏蔽中断方式处理,中断请求信号通过 8088CPU 的 NMI 引脚输入,上升沿触发,不受 IF 标志位控制。(2 分)

采用可屏蔽中断处理,中断请求信号通过 8088CPU 的 INTR 引脚输入,高电平触发,受 IF 标志位控制。(3 分)

二、完成下面程序 (10 分)

lea si, buff;或 mov si, offset buff (从整体上看,逻辑上错一条指令减 2 分)

mov al, [si]

inc si

con: cmp al, [si]

jc next

mov al, [si]

next: inc si

三、存储器应用 (10 分)

U1:D0000H~D7FFFH,U2:D8000H~DFFFFH (错一个数字减 2.5 分)

四、8253 应用 (10 分)

8253 的端口地址:B0H~B3H,计数器 1 的控制字:01110110B,时间常数 1000
(各 2 分)

start:mov al,76h (编程共 4 分)

out 0b3h,al

mov ax,1000

out 0b1h,al

mov al,ah

out 0b1h,al

jmp \$

五、8255 应用 (20 分)

mov dx,8003h (6 分)

mov al,1000xxxxB

out dx,al

mov al,6 (共 14 分,其中 6,0ch,9
各的 3 分,

out dx,al 程序得 5 分)

call delay

mov al,0ch

out dx,al

call delay

mov al,9

out dx,al

call delay

六、数/模转换器 0832 应用 (10 分) (每条指令得 2 分)

mov al,0 ;赋初值

next1: out dx,al ;输出数字量送 D/A 转换器

inc al ;数字量加 1

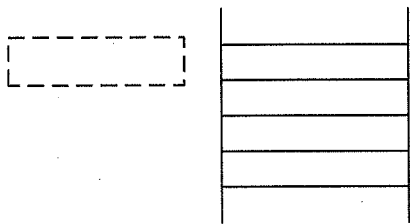
call delay ;调用延时子程序

loop next1 ;循环

微机原理模拟试题二

一、简答题（每小题 5 分,共 40 分）

1. 8086/8088CPU 内部寄存器都是 16 位的,如何管理 1MB 的存储空间? 指令中的逻辑地址如何转换为物理地址?
2. 假设 $DS = 0200H$, $BX = 0300H$, $SI = 0400H$,请确定下面每条指令寻址的存储器物理地址.
MOV AL, [1040H]
ADD [BX+SI], AX
MOV DX, [SI]
3. 8088CPU 在最小模式下读写内存使用的控制信号是什么? 在最大模式下端口输入输出使用的控制信号是什么?
4. 在 String1 单元开始的存储区域中,连续存放着 100 个字符型数据,请编写程序段,将它们传送到 String2 开始的存储区域中.
5. 8255 有几种工作方式? 如果工作在方式 1 下,哪个端口用来输出控制信号或者输入外设的状态信号?
6. 8088/8086CPU 有两个与中断有关的输入引脚,分别支持哪类中断? 哪类中断受标志寄存器 IF 标志位的控制? 如何控制?
7. 8088/8086CPU 的 I/O 系统采用独立编址方式,它的优缺点是什么?
8. 中断类型码为 30H 的中断服务程序,存放在 0300H:2900H 开始的存储区域中,请将该中断类型在中断向量表中的存放地址和内容,填入下图中.



二、完成下面程序（10 分）

下面程序的功能是:把 BUFF 中的二进制数转换为 ASCII 码,依次存入 RESULT 开始的存储单元中并显示在屏幕上.例如,将 2AB9H 转换为 ASCII 码并存入 RESULT 中,RESULT 开始的 4 个单元的内容依次为 32H,41H,42H,39H.请将程序补充完整.

DATA SEGMENT

```

        BUFF DW 2AB9H
        RESULT DB 4 DUP(0)
DATA ENDS
CODE SEGMENT
        ASSUME CS:CODE, DS:DATA
START: MOV AX, DATA
        MOV DS, AX
        MOV AX, BUFF      ;取 BUFF 中的数据
        MOV CX, 4
        LEA BX, RESULT
        MOV SI, 3          ;SI 指向 RESULT+3
A1:     MOV DX, AX          ;备份数据
                                ;提取 AX 的低 4 位
                                ;与 0AH 比较
                                ;大于等于 0AH 则转移到 A2
                                ;在 0~9 之间就加 30H
                                ;跳转到 A3
                                ;A2,在 A~F 之间就加 37H
                                ;A3,保存 ASCII 值
                                ;调整指针 SI
                                ;取出备份的数据
        SHR AX, 4          ;右移 4 位,准备转换下一位 ASCII 数据
        LOOP A1            ;4 位 ASCII 都转换完了吗? 没有则返回 A1 继续处理
        MOV CX, 4          ;显示 4 位 ASCII 数据
        MOV SI, 0
A4:     MOV DL, [BX+SI]
        INC SI
        MOV AH, 2
        INT 21H
        LOOP A4
        MOV AH, 4CH        ;返回 DOS
        INT 21H
CODE ENDS
        END START

```

三、存储器应用（10分）

若用存储器芯片 SRAM6264（8Kx8）和 EPROM2764（8Kx8）构成 32Kx8bit 位的存储器系统（如图 1 所示），请根据图示回答各个存储芯片的地址范围是多少？

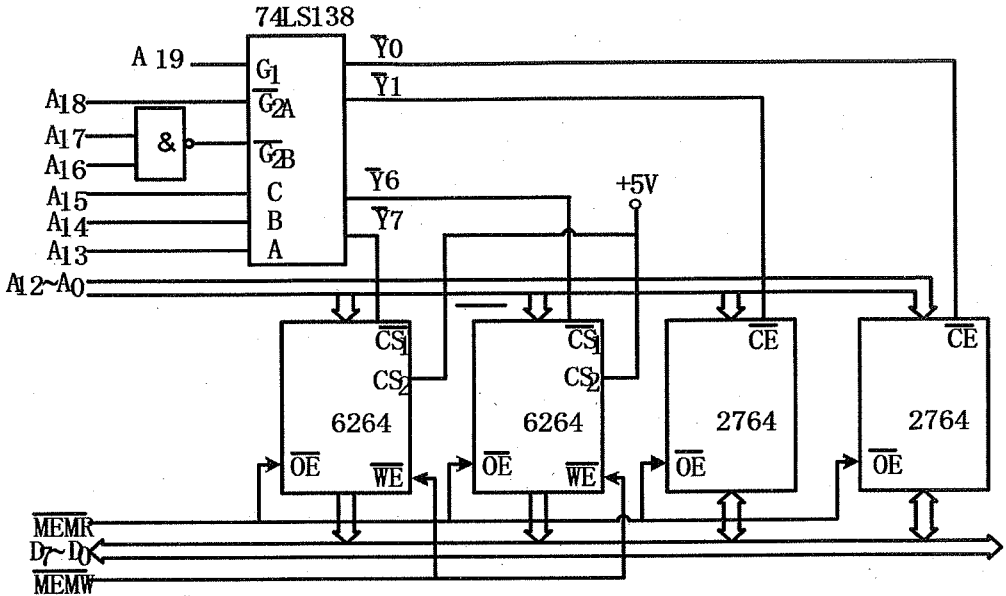


图 1

四、8253 应用（10分）

某系统中 8253 计数器 0 的端口地址为 0200H,CLK0 输入脉冲频率为 1MHz,要求输出高电平时间为 100μs,低电平时间为 1μs 的连续脉冲信号.请说明计数器 0 的控制字和计数初值,并编程控制.

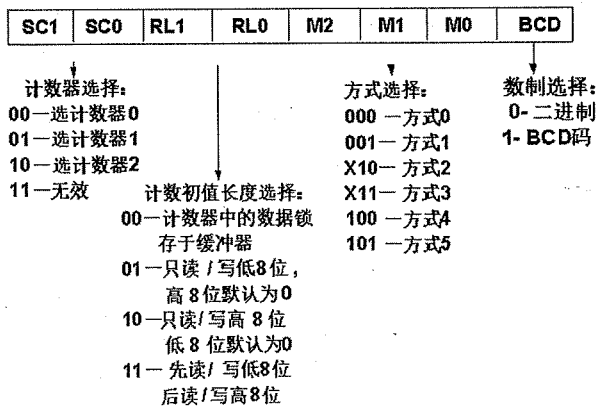


图 28358 控制字

五、8255 应用（10 分）

利用 82C55 芯片实现对七段数码管（共阴极）的动态方式显示控制,82C 55 芯片的 PB0~PB7 输出 8 位段码,接七段数码管的 a~gDp 段,C 端口输出位码,PC0 经反相器后接七段数码管的公共端 COM,如图 2 所示.七段数码管的字型代码表如表 1 所示.请将程序补充完整,并说明各端口地址的具体数值和 8255 的方式控制字.

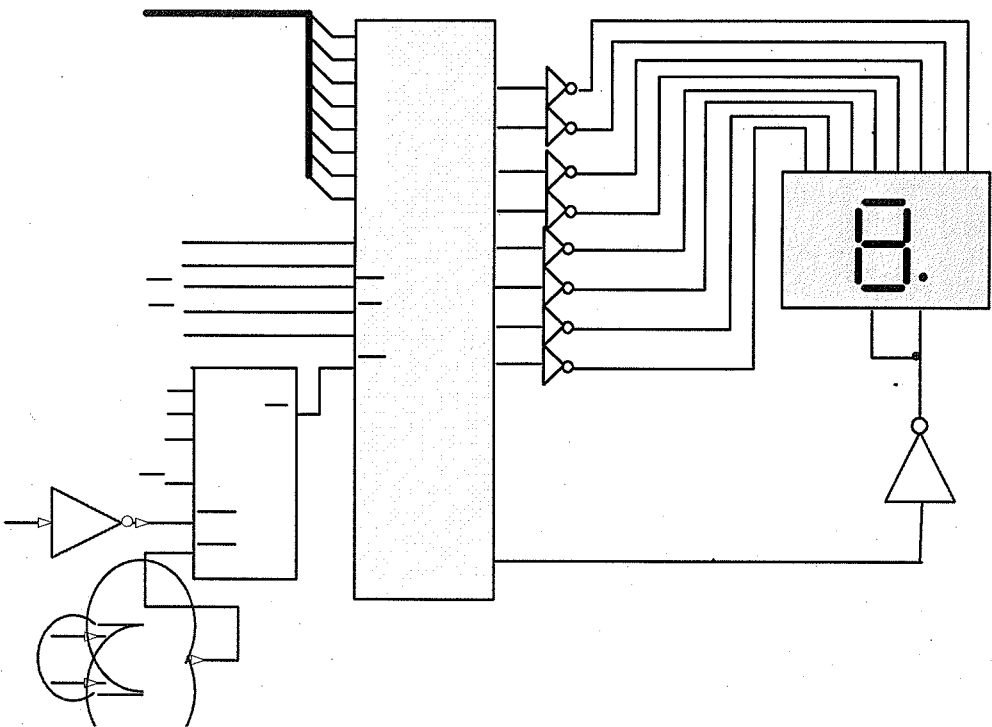


图 2 82C55 应用

表 1 共阴极七段码显示字形码表

显示字形	g	f	e	d	c	b	a	段码
0	0	1	1	1	1	1	1	3fh
1	0	0	0	0	1	1	0	06h
2	1	0	1	1	0	1	1	5bh
3	1	0	0	1	1	1	1	4fh
4	1	1	0	0	1	1	0	66h
5	1	1	0	1	1	0	1	6dh
6	1	1	1	1	1	0	1	7dh
7	0	0	0	0	1	1	1	07h
8	1	1	1	1	1	1	1	7fh
9	1	1	0	1	1	1	1	6fh
A	1	1	1	0	1	1	1	77h
b	1	1	1	1	1	0	0	7ch
C	0	1	1	1	0	0	1	39h
d	1	0	1	1	1	1	0	5eh
E	1	1	1	1	0	0	1	79h
F	1	1	1	0	0	0	1	71h

data segment

```
array db 3fh,06h,5bh,4fh,66h,6dh,7dh,07h,7fh,6fh,77h,7ch,39h,5eh,79h,71h
```

```
data ends
```

code segment

```
assume cs:code, ds:data
```

```
start: mov ax,data
```

```
mov ds, ax
```

```
;设置 8255 的工作方式
```

```
a1: lea si, array ;设置指针 SI,指向字形码数据串
```

```
mov bh,16 ;循环显示 0~F 共 16 个数字
```

```
a2: ;取字形码
```

```
;字形码取反
```

```
;修改指针
```

```
;输出字形码
```

```
;输出位码
```

```

mov cx,0ffffh          ;延时
a3:  loop a3
     dec bh
     jnz a2              ;显示下一个数字
     jmp a1              ;跳转到 a1,重新装入初值
code ends
end start

```

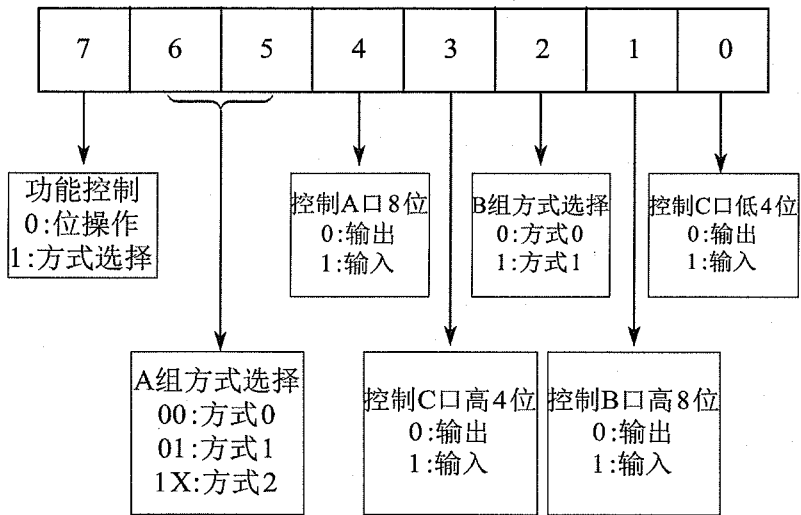


图3 8255方式控制字格

六、数/模转换 (20 分)

如图 4 所示,应用数模转换器 DAC0832 单缓冲方式产生三角波,延时子程序 DELAY 可直接调用.说明它的端口地址,写出它的控制程序.

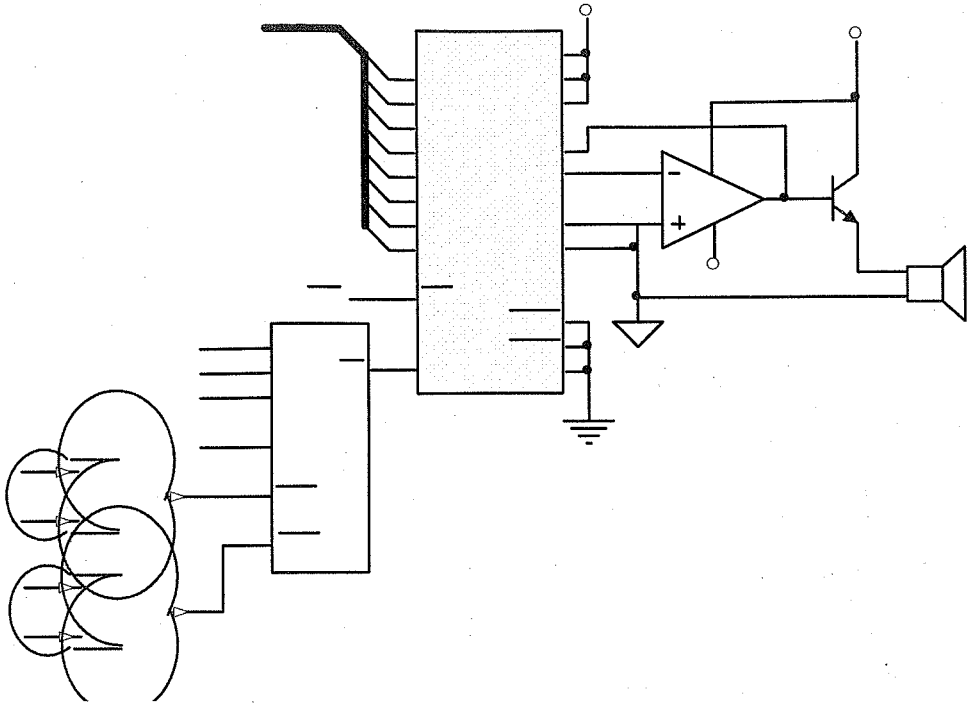


图 4 0832 应用

模拟试题 2 答案部分

一、简答题（每小题 5 分,共 40 分）

1. 答:

8086 和 8088CPU 对 1MB 的存储器采用存储器分段的方法进行管理,每段的长度最大 64KB.

物理地址=段基址 X10H+段内偏移地址. (2 分)

2. 答:03040H,02700H,02400H (5 分);错一个扣 2 分,错 2 个扣 3 分.

3. 答:

$\overline{IO/\overline{M}}, \overline{RD} \quad \overline{WR} \quad (3 \text{ 分}) \quad \overline{IOW} \quad \overline{IOR} \quad (2 \text{ 分})$

4. 答: (5 分)

MOV SI, OFFSET STRING1

MOV DI, OFFSET STRING2

MOV CX, 100

REP MOVSB

5. 参考答案:

3 种工作方式:方式 0,方式 1,方式 2. (3 分) 端口 C (2 分)

6. 参考答案:

8088/8086CPU 有两个与中断有关的输入引脚 INTR,NMI,分别支持可屏蔽中断与非可屏蔽中断 (3 分),其中可屏蔽中断受标志寄存器 IF 标志位的控制,当 IF=1 时允许中断,当 IF=0 时屏蔽中断. (2 分)

7. 参考答案:

8088/8086CPU 的 I/O 系统采用独立编址方式,其优点是不占用内存,节约内存空间;输入输出地址线根数少,I/O 译码电路简单. (3 分)

缺点是需要专门的 I/O 指令,专用控制线. (2 分)

8. 参考答案: (5 分)

000C0H	
	00
	29
	00
	03

二、完成下面的程序 (10 分)

逻辑正确就得分

```
AND  AX, 0FH      ;AND  AL, 0FH
CMP  AL, 0AH
JNC  A2            ;JA  A2
ADD  AL, 30H
JMP  A3
A2:  ADD  AL, 37H
A3:  MOV  [BX+SI], AL
      DEC  SI
      MOV  AX, DX
```

三、存储器应用（10 分）

B0000H~B1FFFFH,
B2000H~B3FFFFH,
BC000H~BDFFFFH,
BE000H~BFFFFFFH

四、8253 应用（10 分）

（BCD 计数也得分）

计数器 0 的计数初值为 101,控制字为 00110100B=34H,00010100B=14H
.....（4 分）

参考程序段:.....（6 分）

```
MOV  DX, 0203H
      MOV  AL, 14H
      OUT  DX, AL
MOV  DX, 0200H
      MOV  AL, 101
      OUT  DX, AL
```

或:

```
MOV  DX, 0203H
      MOV  AL, 34H
      OUT  DX, AL
MOV  DX, 0200H
      MOV  AX, 101
      OUT  DX, AL
      MOV  AL, AH
```

OUT DX,AL

五、8255 应用 (10 分)

端口地址:24H 25H 26H 27H..... (1 分)

方式控制字:1xxxx000B..... (1 分)

MOV AL,80H

OUT 27H,AL..... (2 分)

AGAIN:MOV AL,[SI]..... (6 分)

INC SI

NOT AL

OUT 25H,AL

MOV AL,1

OUT 26H,AL

六、数/模转换 (20 分)

端口地址为 42H (2 分)

CODE SEGMENT	;端口地址送至 DX
ASSUME CS:CODE	;赋循环次数
START: MOV DX,42H	;赋初值
NEXT0: MOV CX,0FFH	;输出数字量送 D / A 转换器
MOV AL,0	;数字量加 1
NEXT1: OUT DX,AL	;调用延时子程序
INC AL	;循环
CALL DELAY	;赋循环次数
LOOP NEXT1	;数字量减 1
MOV CX,0FFH	;输出数字量送 D / A 转换器
NEXT2: DEC AL	;调用延时子程序
OUT DX,AL	;循环
CALL DELAY	;达到最低重新循环
LOOP NEXT2	
JMP NEXT0	

CODE ENDS

END START